自然语言处理（NLP）

1. NLP基础

1.1 定义

NLP（Natural Language Processing，自然语言处理）是计算机学科及人工智能领域一个重要的子学科，它研究如何把人类自然使用的语言转化为计算机可以表示、计算、建模和生成的信息，并进一步完成分类、标注、检索、抽取、匹配、问答、翻译和生成等任务。

1.2 任务范式的演进

1.2.1 基于规则

- **时间段**：1950s ~1980s
- **时代背景**：计算机刚诞生不久，数据量极少，计算能力有限
- **标志性事件**：1966年，Joseph Weizenbaum 开发的 ELIZA 系统， 一个通过模式匹配模拟心理医生对话的程序，是最早的人机对话系统之一
- **主要方法**：专家手工编写语法规则、构建词典，通过模式匹配和规则推导来处理文本
- **典型任务**：早期的句法分析、词法分析、简单问答系统
- **局限性**：
  - 规则的构建和维护成本极高，语言现象千变万化，规则永远写不完
  - 无法处理规则之外的"例外"情况
  - 不同语言、不同领域之间的规则难以复用

1.2.2 统计方法

- **时间段**：90年代初至2000年代
- **标志性事件**：1993年，IBM的统计机器翻译（SMT）
- **主要方法**：概率模型、统计学习
- **任务范式**：随着计算能力的提升和大量标注数据的出现，NLP任务逐渐转向统计方法。机器学习模型开始替代手工规则，通过统计分析大量文本数据中的语言规律来进行任务处理。常见的方法包括：

    - **隐马尔可夫模型（HMM）**：用于序列标注任务，如词性标注、命名实体识别（NER）。
    - **最大熵模型、条件随机场（CRF）**：用于文本标注和序列预测任务。
    - **基于统计的机器翻译（Statistical Machine Translation, SMT）**。
- **局限性**：
    - 严重依赖人工设计的特征工程（feature engineering）
    - 需要大量标注数据
    - 模型只能捕捉浅层的统计规律，难以理解深层语义

1.2.3 深度学习

- **时间段**：2010年代初至今
- **标志性事件**：2014年，**Tomas Mikolov**等人提出了**Word2Vec**模型，标志着深度学习在NLP中已经获得了广泛应用
- **主要方法**：神经网络、深度学习模型
- **任务范式**：深度学习的崛起彻底改变了NLP的范式，尤其是神经网络和深度神经网络（DNN）在大规模数据和计算能力的支持下取得了突破。NLP任务逐渐转向通过深度学习模型来自动学习特征，并且这些模型能够捕捉到语言的复杂结构和语义信息。
    - **卷积神经网络（CNN）**：在文本分类等任务中，CNN被用于学习文本的局部特征。
    - **循环神经网络（RNN）**：用于序列数据处理，特别是在机器翻译、情感分析等任务中。
    - **长短期记忆（LSTM）**：解决了RNN在长序列中的梯度消失问题，广泛应用于语音识别、机器翻译等领域。
- **局限性**：
    - 仍然需要大量标注数据来训练特定任务模型
    - 每个任务都要单独训练一个模型，迁移能力有限
    - 模型可解释性差

1.2.4 预训练语言模型

- **时间段**：2018年起
- **标志性事件**：2018年，Google 发布 **BERT** 模型，在 11 项 NLP 基准任务上刷新纪录，标志着"预训练 + 微调"范式的确立

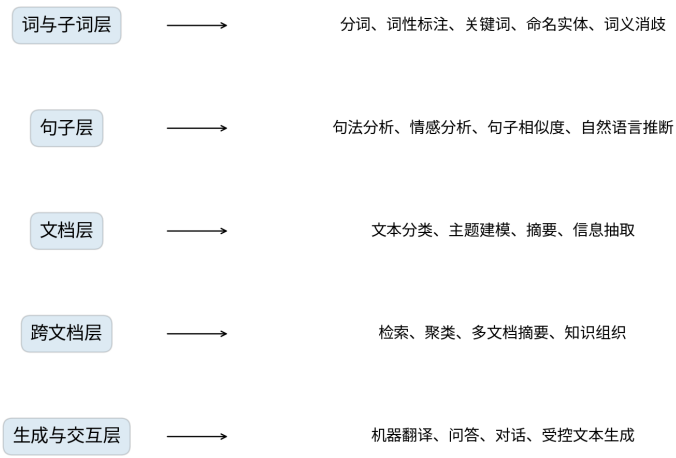
- **主要方法**：预训练+微调（Pretraining + Fine-tuning）
  - **预训练阶段**：在数十亿甚至更多的无标注文本上，通过自监督任务（如掩码语言模型）学习通用语言知识
  - **微调阶段**：在具体任务的少量标注数据上，对预训练模型进行轻量调整
- **任务范式**：在这个阶段，**预训练+微调** 成为NLP的标准范式，模型通过预训练学习大量的语言模式，再通过微调完成具体任务，如文本分类、命名实体识别、文本摘要等。
- **意义**：
  - 极大提升了小样本任务的效果
  - 一个预训练模型可以适配多种任务，迁移能力大幅增强
  - 降低了每个任务的数据需求和训练成本

1.2.5 提示工程与大模型

- **时间段**：2020年起
- **时代背景**：模型规模突破千亿参数，涌现出强大的零样本/少样本能力
- **标志性事件**：2020年，OpenAI 发布 **GPT-3** —— 1750亿参数的大语言模型，展示了惊人的"提示即任务"能力：不需要任何梯度更新，只需设计合适的提示词（prompt），模型就能完成各种任务
- **主要方法**：提示学习（Prompt Learning）/ 提示工程（Prompt Engineering）
  - 通过设计自然语言提示来引导模型输出，而不是修改模型参数
  - 模型从"任务专用"变成了"通用任务求解器"
- **局限性**：
  - 提示的设计依赖经验和试错，效果不稳定
  - 模型可能产生幻觉（hallucination），生成看似合理但事实错误的内容
  - 推理成本高，对算力要求极高

1.3 NLP的主要任务

NLP任务层级：粒度越往下，所需上下文和系统复杂度越高



1.3.1 词级任务

词级任务是自然语言处理的基础层级，以 "词语" 为基本处理单元，解决的是 "文本最基本的构成要素是什么" 这一问题。

所有更高层级的 NLP 任务几乎都建立在词级处理的结果之上。

- **中文分词**：将连续的汉字序列切分成有意义的词语。

拉丁语系（如英语、法语）中词语之间天然有空格分隔，计算机可以直接识别词边界；

但中文、日文等语言的文本是字与字紧密相连的，没有显性分隔符，因此分词成为理解文本语义的第一道关卡。

分词质量的高低，会直接影响后续词性标注、命名实体识别乃至整句理解的准确性。例如：

```
文本：深度学习正在重塑医疗诊断的格局
分词：深度学习/正在/重塑/医疗/诊断/的/格局
```

- **词性标注（POS Tagging）**：为句子中的每个单词标注其词性。

词性既包括名词、动词、形容词、副词、介词、连词等基础词类，也可以细分为专有名词、普通名词、及物动词、不及物动词、动名词等子类。

词性标注是句法分析的前置步骤，帮助模型理解词语在句子中的语法功能。

```
源示例：小猫在阳光下懒洋洋地睡觉。
词性标注：小猫（名词） 在（介词） 阳光（名词） 下（方位词） 懒洋洋（形容词） 地（副词） 睡觉（动词）。
```

- **命名实体识别（NER）**：从文本中识别出具有特定意义的实体，并将其归类到预定义的实体类型中。

常见的实体类型包括人名、地名、组织机构名、时间、日期、货币、百分比、产品名等。

NER 是信息抽取、问答系统、知识图谱构建等任务的核心基础技术。

```
源示例：张桂梅于2008年在云南丽江创办了华坪女高，帮助上千名山区女孩圆了大学梦。
实体标注：
  张桂梅（人名）  于（其他）  2008年（时间）  在（其他）
  云南丽江（地点）  创办了（其他）  华坪女高（组织机构名）  ，（其他）
  帮助（其他）  上千名（其他）  山区（其他）  女孩（其他）
  圆了（其他）  大学梦（其他）  。（其他）
```

- **词义消歧（Word Sense Disambiguation）**：根据上下文确定多义词的具体含义。

自然语言中大量词汇具有多重含义（一词多义），消歧的目标就是结合上下文将模糊的词义锁定到具体的语义上。这是语义理解的关键步骤，直接影响机器翻译、信息检索、问答系统等应用的效果。

```
词：苹果
句子1：我昨天在水果店买了一个苹果（水果），打算作为午餐后的甜点。
句子2：乔布斯在1976年推出了第一款苹果（公司）电脑，彻底改变了个人电脑行业。
```

1.3.2 句子级任务

句子级任务以 "句子" 为处理单元，关注的是词语如何组合成完整的句子、句子表达了什么含义、句子之间有什么关系等问题。

- **句法分析**：分析句子的语法结构，确定词语之间的语法关系和层次组织

句法分析是自然语言处理中的一项技术，旨在分析句子的结构，以确定词语之间的语法关系。其主要目标是理解句子的组成部分及其如何组合成更复杂的结构。

```
源示例：科学家在南极冰层下发现了未知的微生物群落。

依存句法分析结果：
  主语：科学家
  谓语：发现
```

```
   宾语：微生物群落
   定语（修饰宾语）：未知的
   地点状语：在南极冰层下
   补语：了（体标记，表示动作完成）

短语结构分析（简化）：
   [S [NP 科学家]
      [VP [PP 在南极冰层下]
          [VP 发现了
              [NP [AP 未知的] 微生物群落]]]]
```

- **情感分析**：自动识别和判断文本所表达的情感倾向与情感强度。

情感分析的粒度可以从词语级、句子级到文档级不等。

最常见的任务是判断情感极性（积极 / 消极 / 中性），更精细的分析可以识别具体的情绪类别（喜悦、愤怒、悲伤、惊讶等），还可以进行情感强度打分（1-5 分或 0-1 的连续值）。

```
【正面情感分析】
源示例：这款新发布的降噪耳机音质通透，续航持久，佩戴也很舒适，性价比超高！
情感分析：强烈积极情感。多个维度均表达正面评价——
   "音质通透"（音质维度：正面）
   "续航持久"（续航维度：正面）
   "佩戴舒适"（舒适度维度：正面）
   "性价比超高"（价格维度：正面，且程度副词"超"强化了情感强度）
综合情感得分：0.92（满分1.0）

【负面情感分析】
源示例：等了半个月才发货，结果收到的商品有明显划痕，客服还一直推诿不处理，太失望了。
情感分析：强烈消极情感。负面信号叠加——
   "等了半个月才发货"（物流维度：负面）
   "有明显划痕"（品控维度：负面）
   "客服推诿不处理"（服务维度：负面）
   "太失望了"（整体情绪总结：强烈负面）
综合情感得分：0.08（满分1.0，越低越消极）
```

- **句子相似度计算**：评估两个句子之间的相似程度。

句子相似度计算的核心是将句子映射到高维向量空间，然后通过向量之间的距离或夹角来衡量语义相似度。常用的相似度度量方法包括余弦相似度、欧氏距离、曼哈顿距离等。该技术是信息检索、文本去重、问答匹配、paraphrase 识别等任务的基础。

```
句子相似度计算示例

句子A：人工智能技术正在深刻改变医疗行业的诊断方式。
句子B：AI 技术正在彻底革新医疗领域的疾病诊断模式。

【相似度分析】
  词汇层面："人工智能"≈"AI"、"深刻改变"≈"彻底革新"、
            "医疗行业"≈"医疗领域"、"诊断方式"≈"诊断模式"
  语义层面：两句话表达的核心命题完全一致——
            智能技术对医疗诊断领域产生了重大变革性影响
  句法层面：句式结构略有不同，但主干成分（主语+谓语+宾语）对应关系清晰

【相似度评分】
  余弦相似度：0.89
  判定结果：高度语义相似

【对比组】
  句子C：人工智能技术正在深刻改变教育行业的教学方式。
  与句子A的余弦相似度：0.68
  判定结果：中等相似（领域不同，但句式和结构高度平行）
```

1.3.3 文档级任务

文档级任务以 "整篇文档" 为处理单位，关注的是文档整体的主题、内容结构和信息价值。

- **文本分类**：将整篇文档自动分配到一个或多个预定义的类别中。

文本分类是从文本中提取特征，训练分类器将文本映射到预设标签。

分类可以是二分类（如垃圾邮件 / 正常邮件）、多分类（如新闻的体育 / 财经 / 科技 / 娱乐分类），也可以是多标签分类（一篇文章同时属于多个类别）。

```
文本：今天在东京举行了奥运会的开幕式，各国运动员齐聚一堂，展现了团结与友谊的精神。
分类任务：新闻报道分类

分类结果：体育
```

- **文本摘要**：从较长的文本中自动提取核心信息，生成简洁、连贯、保留原意的简短摘要。

  文本摘要主要分为两大类：

  抽取式摘要（Extractive Summarization）：直接从原文中挑选最重要的句子组合成摘要；

  生成式摘要（Abstractive Summarization）：让模型用自己的话重新组织和表达原文的核心内容，更接近人类写摘要的方式。

```
【原文】
   近年来，量子计算领域取得了令人瞩目的进展。IBM、谷歌、亚马逊等科技巨头纷纷加大投入，
   量子比特数量从几年前的几十位迅速攀升至上百位甚至千位级别。2023年，IBM 推出了拥有
   1121个量子比特的 Condor 处理器，标志着量子计算进入了"量子优势"的实用化探索阶段。
   然而，量子纠错、退相干时间短、系统稳定性等核心难题仍未得到根本解决，距离大规模
   商用还有相当长的路要走。学术界普遍认为，未来5到10年将是量子计算从实验室走向
   实际应用的关键窗口期，在药物研发、材料科学、密码学等领域有望率先实现突破。

【生成式摘要】
   量子计算近年来发展迅速，科技巨头持续加码，量子比特数量已达千位级别，IBM 的 Condor
   处理器更是突破千位大关。但量子纠错、退相干等核心技术瓶颈仍制约着大规模商用，
   学界预计未来5到10年是关键窗口期，药物研发、材料科学等领域有望率先落地应用。

（摘要字数：约110字 | 原文压缩率：约35% | 核心信息覆盖率：主要进展+现存挑战+未来展望）
```

- **信息检索**：从大规模文档集合中快速找到与用户查询最相关的文档或信息片段。

信息检索是搜索引擎、问答系统、推荐系统等应用的核心技术。

经典的检索模型包括布尔模型、向量空间模型、BM25 等，近年来，基于语义向量的稠密检索（Dense Retrieval）逐渐成为主流，能够更好地理解查询和文档之间的语义匹配关系。RAG（检索增强生成）技术的核心正是信息检索。

```
【用户查询】
  长时间久坐对腰椎有什么危害？

【检索结果（按相关性排序）】

  文档1（相关性得分：0.91）
  久坐会使腰椎间盘承受的压力增加约40%，长期可导致腰椎间盘退变、
  椎间盘突出等问题。建议每坐45分钟起身活动5分钟，保持正确坐姿。

  文档2（相关性得分：0.83）
  久坐人群腰肌劳损的发生率是普通人群的3倍以上，腰部肌肉长期处于
  紧张状态，容易引发慢性腰痛和腰椎曲度变直。

  文档3（相关性得分：0.72）
  除了腰椎问题，久坐还与肥胖、心血管疾病、糖尿病等多种健康风险
  相关，建议每天保持至少30分钟的中等强度运动。

【检索策略说明】
  第一阶段（稀疏检索 / BM25）：基于关键词匹配召回初步候选集（约100篇）
  第二阶段（稠密检索 / 向量相似度）：基于语义嵌入对候选集精排
  第三阶段（重排序 / Cross-Encoder）：对 Top-K 结果做精细排序后返回
```

1.3.4 跨文档级任务

（1）知识图谱构建：从多个文档中自动抽取实体、关系和属性，构建结构化的知识网络。

知识图谱是自然语言处理和人工智能领域的一种结构化表示，用于描述实体及其之间的关系。其主要目标是将信息以图的形式组织，使计算机能够更好地理解和推理知识。

```
【输入文本（模拟跨文档）】

```

```
  文档1：屠呦呦，1930年12月30日出生于浙江宁波，是中国著名药学家，
        2015年获得诺贝尔生理学或医学奖。

  文档2：屠呦呦团队从青蒿中提取出青蒿素，这种药物能有效降低疟疾患者的死亡率，
        挽救了全球特别是发展中国家数百万人的生命。

  文档3：2019年，屠呦呦被授予"共和国勋章"，以表彰她在中医药科技创新和
        人类健康事业中做出的杰出贡献。

【抽取结果（三元组）】

  (屠呦呦, 出生日期, 1930年12月30日)
  (屠呦呦, 出生地, 浙江宁波)
  (屠呦呦, 职业, 药学家)
  (屠呦呦, 获得奖项, 诺贝尔生理学或医学奖)
  (诺贝尔生理学或医学奖, 获奖年份, 2015年)
  (屠呦呦, 发现, 青蒿素)
  (青蒿素, 来源, 青蒿)
  (青蒿素, 治疗, 疟疾)
  (屠呦呦, 获得荣誉, 共和国勋章)
  (共和国勋章, 授予年份, 2019年)

【知识图谱可视化】

  [屠呦呦] —(出生日期)→ [1930年12月30日]
      |
      ├(出生地)→ [浙江宁波]
      |
      ├(职业)→ [药学家]
      |
      ├(获得奖项)→ [诺贝尔生理学或医学奖] —(获奖年份)→ [2015年]
      |
      ├(发现)→ [青蒿素] —(来源)→ [青蒿]
      |                  └(治疗)→ [疟疾]
      |
      └(获得荣誉)→ [共和国勋章] —(授予年份)→ [2019年]
```

（2）信息抽取结果

从非结构化或半结构化的文本中自动提取出结构化的信息（如事件、关系、属性、实体等）。

信息抽取的目标是将 "人能读懂但机器读不懂" 的自然语言文本，转化为 "机器可以直接处理" 的结构化数据（表格、数据库条目、JSON 等），是连接非结构化文本与结构化数据的桥梁。

【输入文本】
文本1：2024年11月，中国商飞在珠海航展上与多家航空公司签署了C919大型客机的
批量采购协议，订单总数超过300架。

文本2：C919是中国自主研发的单通道干线客机，最大载客量192人，航程可达5555公里，
已于2023年5月实现首次商业飞行。

文本3：据业内人士分析，C919的量产化将逐步打破波音和空客在单通道客机市场的
长期垄断格局。

【信息抽取结果】

实体类型	实体内容	关系/属性	关联实体
组织机构	中国商飞	签署协议	多家航空公司
产品	C919大型客机	订单数量	超过300架
事件	珠海航展	举办时间	2024年11月
产品	C919	研发主体	中国（自主研发）
产品	C919	飞机类型	单通道干线客机
产品	C919	最大载客量	192人
产品	C919	最大航程	5555公里
事件	C919首次商业飞行	发生时间	2023年5月
组织机构	波音、空客	市场地位	长期垄断
市场	单通道客机市场	竞争格局变化	有望被打破

（3）多文档摘要：

针对同一主题的多篇文档，自动整合、提炼出一篇覆盖全面、信息不重复的综合性摘要。

多文档摘要是自然语言处理中的一种技术，旨在从多个相关文档中提取和整合关键信息，生成一个简洁的摘要。其主要目标是帮助用户快速获取多个来源的核心内容，而无需逐一阅读所有文档。

```
【输入文档】

  文档1：2024年全球半导体市场规模达到6200亿美元，同比增长19%。
        AI 芯片需求爆发是主要驱动力，英伟达数据中心业务收入同比翻倍。

  文档2：受 AI 热潮推动，2024年 GPU 市场规模突破800亿美元，
        英伟达占据超过80%的市场份额，AMD 和英特尔紧随其后。

  文档3：2024年全球半导体行业资本开支约1500亿美元，台积电、三星、
        英特尔三大厂商合计占比超过70%，先进制程竞争日趋激烈。

  文档4：分析机构预测，2025年全球半导体市场将继续增长，但增速可能
        放缓至10%左右，地缘政治风险和供应链不确定性仍是主要隐忧。

【多文档摘要】
  2024年全球半导体市场规模达6200亿美元，同比增长19%，AI 芯片需求是核心驱动力。
  GPU 市场突破800亿美元，英伟达以超80%份额领跑。行业资本开支约1500亿美元，
  台积电、三星、英特尔主导先进制程竞争。展望2025年，市场预计继续增长但增速
  可能放缓至10%，地缘政治和供应链风险仍需关注。

（整合维度：市场规模 / 细分市场格局 / 资本开支 / 未来展望）
```

1.3.5 更高级任务

- **对话系统**：

能够与人类进行自然语言交互的智能系统，包括用户意图理解、对话状态跟踪、知识检索、回复生成等多个核心模块。

对话系统按任务类型可分为：任务型对话（帮助用户完成特定任务，如订票、订餐、查询）、问答型对话（回答用户的知识类问题）、闲聊型对话（开放域的自由聊天）。现代大语言模型的出现，极大地推动了对话系统的能力边界。

```
【对话场景：智能客服--退换货咨询】

用户：你好，我上周买的那双跑鞋尺码不合适，想换大一码的。

  系统理解：
    意图识别：申请换货
    实体抽取：商品类型=跑鞋，换货原因=尺码不合适，目标=大一码
    对话状态：{intent: exchange, product: 跑鞋, reason: 尺码问题, target_size: 大一号}

  系统回复：
    您好！很抱歉尺码不合适给您带来不便。我帮您查询一下订单信息，
    请问您的订单号是多少呢？另外大一号的同型号目前有货，可以为您安排换货。

用户：订单号是202408150032，换货的话运费谁出啊？

  系统理解：
    意图识别：提供订单号 + 询问运费承担方
    实体抽取：订单号=202408150032，新话题=运费
    对话状态更新：{order_id: 202408150032, question: 运费承担方}

  系统回复：
    已查到您的订单。因尺码问题属于7天无理由换货范畴，首重运费由我们承担，
    您寄回后提供快递单号，我们会为您报销运费。换货申请已帮您提交，
    预计3-5个工作日完成换货流程，还有其他问题吗？

用户：没有了，谢谢。

  系统理解：
    意图识别：结束对话
    情感分析：积极（"谢谢"表达满意）

  系统回复：
    不客气！祝您生活愉快，有任何问题随时联系我们～
```

- **机器翻译**：

利用计算机自动将一种语言（源语言）的文本翻译成另一种语言（目标语言），同时保持语义的准确性和表达的自然流畅。

机器翻译是 NLP 历史最悠久的任务之一，其技术发展经历了多个阶段：基于规则的机器翻译 → 基于统计的机器翻译（SMT） → 基于神经网络的机器翻译（NMT） → 大语言模型驱动的翻译。当前神经机器翻译已在许多语言对上达到接近人类专业译者的水平。

```
示例
源文本（中文）：我喜欢学习新的语言。
机器翻译结果（英文）：I enjoy learning new languages.
```

- **文本生成**：根据上下文生成连贯的文本（如自动编写故事或新闻）。

文本生成是自然语言处理中的一项技术，旨在自动创建符合特定要求或风格的文本内容。其主要目标是生成自然流畅且有意义的文本，以满足用户需求。

```
【生成主题：以"秋夜山居"为题，创作一首七言律诗】

秋夜山居

山深月冷夜沉沉，松影摇窗梦未深。
石上清泉流古韵，篱边黄菊抱秋心。
一灯如豆书千卷，万籁俱寂酒半斟。
坐忘人间烟火事，唯闻涧水送清音。

【生成策略说明】
  体裁约束：七言律诗（八句，每句七字，对仗工整，押韵）
  意象选择：山月、松影、清泉、黄菊、孤灯、涧水等典型"山居秋夜"意象
  情感基调：清幽、宁静、超然物外
  结构安排：首联点题（山深月冷）→ 颔联写景（泉与菊，对仗）
            → 颈联写人（读书饮酒，对仗）→ 尾联抒情（坐忘超然）
```

1.4 困难与挑战

1.4.1 语言歧义

(1)同音词

有一次，纪晓岚与和珅一同在朝为官，纪晓岚任侍郎，和珅任尚书。两人在一起喝酒时，和珅指着一只狗问纪晓岚:

```
和珅（尚书）：纪侍郎，纪大人，这是何物？是狼是狗？(侍郎是狗)
纪晓岚（侍郎）：和尚书，和大人，这好办。看尾巴尖呀，下垂是狼，上竖是狗，记住了。(尚书是狗)
```

(2)词性歧义

同一个词可能是不同词性，导致语义不同

```
动物保护警察
```

(3)分词导致的歧义

中文没有空格分隔词语，切分方式不同，意思完全不同

```
例1：自动化研究所取得的成就
理解一：自动化 / 研究 / 所 / 取得 / 的 / 成就
理解二：自动化 / 研究所 / 取得 / 的 / 成就

例2：武汉市长江大桥
分词1：武汉市/长江大桥
分词2：武汉/市长/江大桥
```

(4)语法结构歧义

```
喜欢乡下的孩子
```

(5)语境歧义

```
你太棒了，你简直是个天才！（真心夸赞）
你太棒了，你简直是个天才！（阴阳师）
```

1.4.2 不同语言结构差异

不同语言的"底层逻辑"，在语法、语序、形态上差异巨大，给机器翻译、跨语言理解带来了根本性挑战。

比如：

- 中文是**孤立语**，靠语序和虚词表达语法关系
- 英语是**屈折语**，动词有时态变化，名词有单复数
- 日语、韩语是**黏着语**，靠助词和词尾变化表达语法

这些结构差异导致很多语言现象无法直接对应翻译。

1.4.3 未知语言不可预测性

语言是活的，每年都有大量新词语、新表达出现，而 NLP 系统的训练数据和词典总是滞后的。

新词	含义	来源
city不city	洋气不洋气、时髦不时髦	源于 citywalk 流行
钝感艺术	自嘲"缺心眼"，指不计较、不敏感的态度	网络衍生
边界感	对个人隐私和空间的尊重与保护意识	心理学概念大众化
泰裤辣	太酷了	"too cool" 谐音
PPT你	给你画饼，只说不做	职场梗
嘴替	帮自己说出心里话的人/言论	网络造词
配享太庙	极高赞誉，夸某人说得太对了	历史梗网络化

1.4.4 语言表达的复杂性

```
甲：你这是什么意思？
乙：没什么意思，意思意思。
甲：你这就不够意思了。
乙：小意思，小意思。
甲：你这人真有意思。
乙：其实也没有别的意思。
甲：那我就不好意思了。
```

1.4.5 潜台词

表面说法	潜台词
"这件事要走流程，我也很难办"	不给点好处，别想空手套白狼
"这个事情怎么样了，你抽空落实一下"	别墨迹了，赶紧办，办好给我回话
"你最近身体怎么样？是不是家里有困难？"	你最近工作状态不行，质量也不行
"小伙子很有性格"	你太锋芒毕露了，不听话迟早要收拾你
"这件事你看着办"	这么简单的事都办不好？办不好我就办你

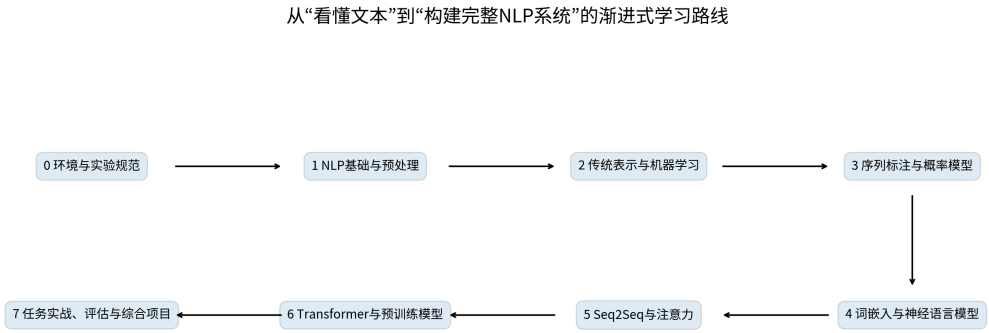
1.4.6 背景与常识

```
中国国家队比赛最没悬念的是乒乓球和足球，他们一个谁也打不过，另一个谁也打不过

请计算：大锤减小锤加宫廷玉液酒等于多少？

穿越者表明身份：奇变偶不变；天王盖地虎；你的益达；今年过节不收礼；
```

1.5 整体主线



- 先处理肉眼可见的字符、词和句子。
- 再学习稀疏向量和经典分类器。
- 再处理具有顺序关系的标签和序列。
- 再学习稠密词向量和上下文表示。

- 再学习输入序列到输出序列的映射。
- 最后进入 Transformer 和预训练模型。

1.5.1 理论线

负责回答“为什么”：

- 为什么文本需要数值表示？
- 为什么 One-Hot 不能表达语义？
- 为什么 TF-IDF 能突出关键词？
- 为什么 HMM 适合序列标注？
- 为什么 Word2Vec 能学习语义邻近？
- 为什么 RNN 会梯度消失？
- 为什么注意力能够缓解固定上下文瓶颈？
- 为什么 Transformer 需要位置编码和掩码？
- 为什么 BERT 适合理解任务，而自回归解码器适合生成？
- 等....

1.5.2 代码线

负责回答怎样实现：

- NumPy 手算
- scikit-learn 建立传统基线。
- Gensim 训练 Word2Vec / FastText。
- Transformers 完成 tokenizer、模型加载、数据整理、训练和推理。
- 等....

1.5.3 项目线

负责回答“怎样解决真实任务”：

- 数据定义。
- 标签设计。
- 训练集、验证集和测试集拆分。
- 指标选择。

- 基线与改进模型对比。
- 推理接口。
- 模型保存和复用。
- 等.....

2. 基于规则的任务范式

2.1 概述

2.1.1 核心思想

基于规则的方法是 NLP 最古老的范式，它的核心逻辑很直观：既然语言有规律，那就把规律写出来，让机器按规则办事。

具体来说，规则范式依赖语言学专家手动编写规则和词典，通过模式匹配和规则推导来处理文本。系统的表现完全取决于规则的覆盖度和质量。

2.2.2 局限性

规则范式有三个根本性的局限：

规则爆炸：语言现象千变万化，例外永远比规则多。写了 100 条规则，总能找到第 101 个例外。

维护成本高：每新增一条规则，都要专家介入，而且新规则可能和旧规则冲突。

迁移性差：中文的规则不能直接用到英文，新闻领域的规则搬到医疗领域就失效。

正因如此，随着数据量和计算能力的增长，统计方法和深度学习逐渐取代了规则方法的主流地位

2.2.3 当今地位

虽然规则方法不再是主流，但它依然有不可替代的价值：

- **高精度场景**：在法律、金融等对准确率要求极高的领域，规则能保证 100% 的可控性
- **可解释性**：规则的每一步判断都可以追溯，不像深度学习是"黑盒"
- **冷启动**：没有标注数据时，规则是最快能跑起来的方案
- **混合系统**：现代 NLP 系统常常把规则作为前处理或后处理模块，和深度学习模型配合使用

> 规则方法不是"过时了"，而是从"主角"变成了"配角"，在合适的场景下，它依然高效可靠。

2.2 中文分词

2.2.1 为什么需要?

英文文本中，单词之间天然有空格分隔，计算机很容易知道"哪些字符构成一个词"。

但中文不一样，句子是连续的汉字串，没有显式的词边界：

```
英文：I love natural language processing
中文：我爱自然语言处理
```

这就是中文分词的任务， 将连续的汉字序列切分成有意义的词语序列。分词几乎是所有中文 NLP 任务的前置步骤：文本分类需要分词后提取特征，信息检索需要分词后建立索引，机器翻译需要分词后对齐。分词的质量直接影响后续所有任务的上限。

2.2.2 正向最大匹配法

正向最大匹配（Forward Maximum Matching, FMM） 是最经典的分词算法之一，它的思路非常朴素：从左往右，每次尽量匹配最长的词。

算法步骤：

1. 加载分词词典，确定词典中最长词的长度 `max_len`
2. 从句子左端开始，取长度为 `max_len` 的子串，去词典中查找
3. 如果找到了，就把这个词切分出来，指针右移 `max_len` 位
4. 如果没找到，就把子串长度减 1，再去词典中查找
5. 重复步骤 3-4，直到匹配成功或子串长度减到 1
6. 继续处理剩余文本，直到整个句子切分完毕

代码

```python
"""
正向最大匹配法（FMM）
"""

# 分词词典
dictionary = ["我们", "野生", "动物", "动物园", "玩", "野生动物园", "在", "是", "你"]
dict_set = set(dictionary)
max_word_len = max(len(w) for w in dictionary)
```

```
9
10 # 待切分的句子
11 sentence = "我们在野生动物园玩"
12
13
14 def forward_matching(sentence, dict_set, max_word_len):
15     result = []
16     i = 0
17     n = len(sentence)
18
19     while i < n:
20         window_end = min(i + max_word_len, n)
21         matched_word = None
22
23         for j in range(window_end, i, -1):
24             word = sentence[i:j]
25             if word in dict_set:
26                 matched_word = word
27                 break
28
29         if matched_word is None:
30             matched_word = sentence[i]
31             i += 1
32         else:
33             i += len(matched_word)
34         result.append(matched_word)
35     return result
36
37
38 print("/".join(forward_matching(sentence, dict_set,
   max_word_len)))
39
```

2.2.3 逆向最大匹配法

逆向最大匹配（Reverse Maximum Matching, RMM）的思路和 FMM 完全一样，只是方向反过来，从右往左，每次尽量匹配最长的词。

代码

```
1 """
2 逆向最大匹配法（RMM）
3 """
4
```

```
dictionary = ["我们", "野生", "动物", "动物园", "玩", "野生动物园", "在", "是", "你"]
dict_set = set(dictionary)
max_word_len = max(len(w) for w in dictionary)
sentence = "我们在野生动物园玩"


def backward_matching(sentence, dict_set, max_word_len):
    result = []
    i = len(sentence)
    while i > 0:
        # 取从 i 往左、最长为 max_word_len 的窗口
        window_start = max(i - max_word_len, 0)
        matched_word = None

        # 从长到短尝试匹配
        for j in range(window_start, i):
            word = sentence[j:i]
            if word in dict_set:
                matched_word = word
                break
        if matched_word is None:
            matched_word = sentence[i - 1]
            i -= 1
        else:
            i -= len(matched_word)
        result.insert(0, matched_word)
    return result


print("/".join(backward_matching(sentence, dict_set, max_word_len)))
```

2.2.4 双向最大匹配法

双向最大匹配法（Bi-directional Maximum Matching，Bi-MM）是将正向最大匹配法得到的分词结果和逆向最大匹配法得到的结果进行比较，然后按照最大匹配原则，选取词数切分最少的作为结果。

判断规则：

1. 如果正向和逆向的分词结果词数不同 → 选词数更少的那个

2. 如果词数相同：

   - 结果完全一样 → 没有歧义，随便返回一个
   - 结果不一样 → 返回单字（单个字成词）数量更少的那个

代码

```python
"""
双向最大匹配法（BiMM）
"""

# 分词词典
dictionary = ["我们", "野生", "动物", "动物园", "玩", "野生动物园", "在", "是", "你"]
dict_set = set(dictionary)
max_word_len = max(len(w) for w in dictionary)

# 待切分的句子
sentence = "我们在野生动物园玩"


def forward_matching(sentence, dict_set, max_word_len):
    result = []
    i = 0
    n = len(sentence)

    while i < n:
        window_end = min(i + max_word_len, n)
        matched_word = None

        for j in range(window_end, i, -1):
            word = sentence[i:j]
            if word in dict_set:
                matched_word = word
                break

        if matched_word is None:
            matched_word = sentence[i]
            i += 1
        else:
            i += len(matched_word)
        result.append(matched_word)
    return result

```

```python
def backward_matching(sentence, dict_set, max_word_len):
    result = []
    i = len(sentence)
    while i > 0:
        # 取从 i 往左、最长为 max_word_len 的窗口
        window_start = max(i - max_word_len, 0)
        matched_word = None

        # 从长到短尝试匹配
        for j in range(window_start, i):
            word = sentence[j:i]
            if word in dict_set:
                matched_word = word
                break
        if matched_word is None:
            matched_word = sentence[i - 1]
            i -= 1
        else:
            i -= len(matched_word)
        result.insert(0, matched_word)
    return result


def bidirectional_max_matching(sentence, dict_set, max_word_len):
    """双向最大匹配"""
    forward = forward_max_matching(sentence, dict_set, max_word_len)
    backward = backward_max_matching(sentence, dict_set, max_word_len)

    # 规则1：词数不同 → 选词数少的
    if len(forward) != len(backward):
        return forward if len(forward) < len(backward) else backward

    # 规则2：词数相同
    if forward == backward:
        return forward  # 完全一致，无歧义
    else:
        # 选单字数量少的
        f_single = sum(1 for w in forward if len(w) == 1)
```

```
        b_single = sum(1 for w in backward if len(w) == 1)
        return forward if f_single <= b_single else backward

print("/".join(bidirectional_max_matching(sentence, dict_set, max_word_len)))
```

2.2.5 jieba分词

jieba 是最流行的中文分词库之一，它结合了基于前缀词典的高效分词和 HMM 模型，兼顾速度和准确率。

jieba做中文分词有三种模式：

(1)全模式：

- **切分方式**：将句子中的所有可能词语都切分出来，包括一些较小的词和组合词。这意味着同一个句子可能会被切分成多个词语。
- **适用场景**：适合用于关键词提取或快速获取文本中的所有潜在词汇。这个模式适合初步分析文本，了解文本的主题。

(2)精确模式：

- **切分方式**：将句子精确地切分成词语，尽量准确地还原出语义。这个模式会根据词典和语言规则，选择最可能的分词结果。
- **适用场景**：适合用于文本分析和语义理解，常用于处理需要分析句子意思的任务，如文本分类和情感分析。

(3)搜索引擎模式：

- **切分方式**：在精确模式的基础上，对词语进行更细粒度的切分，特别是长词和短词的组合，使得搜索时能更好地匹配用户输入。
- **适用场景**：适合用于搜索引擎的词语匹配，尤其是在构建搜索引擎时，可以提高用户输入查询时的匹配准确性。

总的来说，选择哪种模式取决于具体的应用需求：

如果需要准确理解文本，使用精确模式；

如果需要获取所有可能的词汇，使用全模式；

如果需要优化搜索结果，使用搜索引擎模式。

```
"""
```

```
02_jieba相关
单独安装分词: pip install jieba
"""
import jieba

text = '我来到北京清华大学'

# 全模式
seg_list = jieba.cut(text, cut_all=True)
for word in seg_list:
    print(word, end="/")
print()

# 精确模式
seg_list = jieba.cut(text, cut_all=False)
for word in seg_list:
    print(word, end="/")

print()

# 搜索引擎模式
seg_list = jieba.cut_for_search(text)
for word in seg_list:
    print(word, end="/")
print()

```

输出结果：

```
我/来到/北京/清华/清华大学/华大/大学/
我/来到/北京/清华大学/
我/来到/北京/清华/华大/大学/清华大学/
```

2.3 词性标注

2.3.1 定义

词性标注（Part-of-Speech Tagging, POS Tagging） 是自然语言处理中的一项重要任务：给文本中的每个词，标注上它的词性（名词、动词、形容词等）。准确的词性标注能够为后续的任务提供可靠的输入。

例如：

```
我    爱    自然语言    处理
r     v     l          v
```

其中 `r` 代表代词，`v` 代表动词，`l` 代表习用语。

2.3.2 为什么需要?

由于中文和其他语言在句子结构和词义上存在一定的复杂性，进行有效的词性标注对于理解句子的语法和语义至关重要。因此，词性标注被视为处理中文文本的重要步骤。

- **句法分析的基础**：知道了每个词的词性，才能进一步分析句子的语法结构
- **消歧的手段**：很多词有多个词性（比如"计划"可以是名词也可以是动词），词性标注可以帮助确定具体含义
- **下游任务的特征**：在命名实体识别、关系抽取等任务中，词性是重要的特征

2.3.3 标注方法

(1)基于规则的方法

词典查找：每个词在词典中都有一个或多个可能的词性，先查词典得到候选词性集合

规则消歧：当一个词有多个候选词性时，通过上下文规则来确定最终词性

常见的消歧规则举例：

- "的"前面通常是形容词或名词
- "了"后面通常不是动词的进行时
- 数词后面通常跟量词

(2) 基于统计模型的方法

- 隐马尔可夫模型（HMM）：HMM是一种常用的序列模型，能够通过训练数据学习词的转移概率和发射概率，从而进行词性标注。

2.3.4 标注规范

词性标注要有一定的标注规范，如将名词、形容词、动词表示为"n", "adj", "v"等。

中文领域尚无统一的标注标准，中文领域主流的有两套：

- **北大词性标注集**：北京大学计算语言学研究所制定，中文领域使用广泛，地址: https://opendata.pku.edu.cn/dataverse/icl
- **宾州词性标注集**：基于英文宾州树库扩展，常用于中文树库标注

标记	词性	说明
h	前接成分	取英语 head 的第 1 个字母
i	成语	取英语成语 idiom 的第 1 个字母
j	简称略语	取汉字“简”的声母
k	后接成分	
l	习用语	习用语尚未成为成语，有点“临时性”，取“临”的声母
m	数词	取英语 numeral 的第 3 个字母，n、u 已有他用
ng	名语素	名词性语素。名词代码为 n，语素代码 g 前面置以 n
n	名词	取英语名词 noun 的第 1 个字母
nr	人名	名词代码 n 和“人（ren）”的声母并在一起
ns	地名	名词代码 n 和处所词代码 s 并在一起
nt	机构团体	“团”的声母为 t，名词代码 n 和 t 并在一起
nz	其他专名	“专”的声母的第 1 个字母为 z，名词代码 n 和 z 并在一起
o	拟声词	取英语拟声词 onomatopoeia 的第 1 个字母
p	介词	取英语介词 prepositional 的第 1 个字母
q	量词	取英语 quantity 的第 1 个字母
r	代词	取英语代词 pronoun 的第 2 个字母，因 p 已用于介词
s	处所词	取英语 space 的第 1 个字母
tg	时语素	时间词性语素。时间词代码为 t，在语素的代码 g 前面置以 t
t	时间词	取英语 time 的第 1 个字母
u	助词	取英语助词 auxiliary
vg	动语素	动词性语素。动词代码为 v。在语素的代码 g 前面置以 v
v	动词	取英语动词 verb 的第一个字母
vd	副动词	直接作状语的动词。动词和副词的代码并在一起
vn	名动词	指具有名词功能的动词。动词和名词的代码并在一起
w	标点符号	
x	非语素字	非语素字只是一个符号，字母 x 通常用于代表未知数、符号
y	语气词	取汉字“语”的声母
z	状态词	取汉字“状”的声母的前一个字母

2.2.5 Jieba库词性标注

Jieba库提供了词性标注功能，采用结合规则和统计的方式，具体为在词性标注的过程中，词典匹配和HMM共同作用。

代码

```
from jieba import posseg

def pos(text):
    words = posseg.cut(text)
    res = ''
    for word, flag in words:
        res += f"{word}/{flag} "
    return res
```

```
9
10 text = "我爱自然语言处理"
11 print(pos(text))
12
13 text = '杰弗里辛顿2024年获得了诺贝尔物理学奖'
14 print(pos(text))
```

```
1 我/r 爱/v 自然语言/l 处理/v
2 杰弗里/nrt 辛顿/nr 2024/m 年/m 获得/v 了/ul 诺贝尔物理学奖/nz
```

词性标注流程如下：

1. 先用正则表达式判断文本是否为汉字
2. 如果是汉字，构建 HMM 模型计算最大概率路径，同时在词典中查找词性
3. 如果词典中找不到该词，标记为"未知"（`x` 或其他默认标签）
4. 非汉字部分（数字、英文等）通过正则表达式分别赋予数词（`m`）、英文（`eng`）等标签

2.4 命名实体识别（NER）

2.4.1 定义

命名实体识别（Named Entity Recognition, NER） 的任务是：从文本中识别出具有特定意义的实体，并将它们归类到预定义的类别中，通常包括人名、地名、组织名、日期、时间、货币、百分比等。NER 的核心任务是将文本中的这些实体进行标注，使其更易于分析和理解。

常见的实体类别包括：

类别	含义	示例
人名（PER）	人物姓名	张三、爱因斯坦
地名（LOC）	地理位置	北京、长江、珠穆朗玛峰
组织名（ORG）	机构、团体	清华大学、联合国、阿里巴巴
时间（TIME）	日期、时间	2024年6月、下周一、下午三点
百分比（PERCENT）	百分比数值	30%、百分之五十
货币（MONEY）	金额	100元、5万美元

NER 的应用非常广泛：

- 信息抽取：从新闻中抽取出事件的人物、地点、时间
- 问答系统：回答"谁在哪里做了什么"这类问题
- 知识图谱构建：实体是知识图谱的节点
- 金融风控：识别合同中的关键实体

2.4.2 难点

- **没有明确的分词**：中文文本没有空格分隔词语，这使得分词变得困难。命名实体通常由多个汉字组成，分词的准确性直接影响到实体的识别效果。
- **同名实体**：在中文中，可能存在同名的实体（如“张三”可以指多个不同的人），需要上下文信息来判断具体指代哪个实体。
- **多义词**：某些词语可能在不同上下文中具有不同的含义，增加了识别的复杂性。
- **新词识别**：随着社会的发展，新的命名实体不断出现（如新兴公司、人物等），NER系统需要及时更新以识别这些新词。
- **专业术语**：特定领域的命名实体（如医学、法律等）可能不在训练数据中，需要系统具备一定的领域适应能力。

2.4.3 方法

(1)基于规则的方法

- 通过手工定义的规则和词典来识别命名实体，通常使用正则表达式和语言学知识。
- 优点包括实现简单、速度快和可解释性强，但缺点是覆盖面有限，灵活性差，且需要较高的人工维护成本。

(2)基于统计的方法

- 使用统计模型（如隐马尔可夫模型）进行实体识别，依赖于标注数据进行训练。
- 其主要思想是基于人工标注的语料，将命名实体识别任务作为序列标注问题来解决。基于统计的方法对语料库的依赖比较大，而可以用来建设和评估命名实体识别系统的大规模通用语料库又比较少，这是该方法的一大制约。

(3)基于深度学习的方法

- 采用深度学习模型（如LSTM、BERT等）进行特征自动学习和实体识别，能够捕捉复杂的上下文信息。
- 虽然性能优越，但对计算资源和标注数据的需求较高，且模型的可解释性较差。

2.5 NLTK

NLTK（Natural Language Toolkit）是 Python 最经典的英文 NLP 库。

```
pip install nltk
```

首次使用需要下载数据包：

```
import nltk
nltk.download()  # 弹出GUI界面，选择 all 下载全部
```

或者下载指定包：

```
nltk.download('punkt')        # 分词器
nltk.download('averaged_perceptron_tagger')  # 词性标注器
nltk.download('maxent_ne_chunker')  # 命名实体识别
nltk.download('words')        # 词表
nltk.download('stopwords')    # 停用词
nltk.download('wordnet')      # WordNet 词形还原
```

2.5.1 分词

1.句子分词（把段落拆成句子）

```
from nltk.tokenize import sent_tokenize

text = "Hello everyone. Welcome to NLP class. NLTK is easy to learn!"
sentences = sent_tokenize(text)

print(sentences)
# 输出：
# ['Hello everyone.', 'Welcome to NLP class.', 'NLTK is easy to learn!']
```

2. 单词分词（把句子拆成单词）

```python
from nltk.tokenize import word_tokenize

sentence = "NLTK is a powerful tool for natural language processing."
words = word_tokenize(sentence)

print(words)
# 输出:
# ['NLTK', 'is', 'a', 'powerful', 'tool', 'for', 'natural', 'language', 'processing', '.']
```

3. 其他分词器

```python
from nltk.tokenize import WordPunctTokenizer, RegexpTokenizer

# 按标点切分
tokenizer = WordPunctTokenizer()
print(tokenizer.tokenize("I can't believe it!"))
# ['I', 'can', "'", 't', 'believe', 'it', '!']

# 正则分词（只保留字母单词）
tokenizer = RegexpTokenizer(r'\w+')
print(tokenizer.tokenize("I can't believe it!"))
# ['I', 'can', 't', 'believe', 'it']
```

2.5.2 词性标注

```python
from nltk import word_tokenize, pos_tag

sentence = "Geoffrey Hinton won the Nobel Prize in Physics in 2024."
words = word_tokenize(sentence)
tagged = pos_tag(words)

print(tagged)
[
    ('Geoffrey', 'NNP'),    # 专有名词（单数）
    ('Hinton', 'NNP'),      # 专有名词（单数）
    ('won', 'VBD'),         # 动词（过去式）
    ('the', 'DT'),          # 限定词
    ('Nobel', 'NNP'),       # 专有名词
    ('Prize', 'NNP'),       # 专有名词
```

```
    ('in', 'IN'),           # 介词
    ('Physics', 'NNP'),     # 专有名词
    ('in', 'IN'),           # 介词
    ('2024', 'CD'),         # 数词
    ('.', '.')              # 标点
]

# 查看所有标签含义
import nltk
nltk.download('tagsets')
nltk.help.upenn_tagset('NNP')
```

2.5.3 NER

```
import nltk
from nltk import word_tokenize, pos_tag, ne_chunk

nltk.download('maxent_ne_chunker_tab')  # NER 模型（注意是 _tab 结尾）
nltk.download('words')  # 词典，NER 也依赖

sentence = "Geoffrey Hinton works at Google in California and won the Nobel Prize."
words = word_tokenize(sentence)
tagged = pos_tag(words)
entities = ne_chunk(tagged)

print(entities)
```

2.6 正则表达式

2.6.1 定义

正则表达式（简称 regex /re）是一套用特殊符号描述文本模式的规则语言，用来在字符串中查找、匹配、替换、提取符合特定模式的文本。

2.6.2 常用函数

函数	作用	返回值	特点
re.match()	从字符串开头匹配	匹配对象 / None	只看开头，开头不匹配直接失败

函数	作用	返回值	特点
re.search()	在整个字符串中找第一个匹配	匹配对象 / None	找到第一个就停
re.findall()	找出所有匹配的内容	列表	最常用，返回所有结果
re.sub()	替换匹配到的内容	替换后的字符串	文本清洗神器

代码 re.match()

```python
import re

# re.match ()从开头匹配
text = "2024年诺贝尔物理学奖揭晓"
result = re.match(r'\d+', text)  # 匹配开头的数字
print(result.group())  # 2024
print(result.start())  # 0（起始位置）
print(result.end())  # 4（结束位置）
```

代码 re.search()

```python
import re

# re.search()找第一个匹配
text = "杰弗里·辛顿出生于1947年，2024年获得诺贝尔奖"
# search 是全文搜索，找到第一个就返回。
result = re.search(r'\d{4}年', text)
print(result.group())  # 1947年
```

代码re.findall()

```python
import re

# re.findall() 找所有匹配
text = "杰弗里·辛顿出生于1947年，2024年获得诺贝尔奖，奖金1100万克朗"
results = re.findall(r'\d+', text)
# ['1947', '2024', '1100']
print(results)
```

代码 re.sub()

```python
import re

# re.sub ()替换匹配内容
text = "我的电话是 13812345678，备用号 13987654321"

# 把所有手机号替换成 [隐私信息]
result = re.sub(r'1[3-9]\d{9}', '[隐私信息]', text)
print(result)
```

2.6.3 字符类

普通字符和元字符

字符类用来定义: 这个位置可以出现哪些字符。

类型	说明	例子
普通字符	字面匹配，写啥找啥	`abc` 匹配 "abc"
元字符	有特殊含义的符号	`. * + ? [] () {} ^ $ \

字符集

方括号里的字符，任意匹配一个

写法	含义	匹配示例
`[abc]`	匹配 a、b、c 中任意一个	a / b / c
`[a-z]`	匹配任意小写字母	a ~ z
`[A-Z]`	匹配任意大写字母	A ~ Z
`[0-9]`	匹配任意数字	0 ~ 9
`[a-zA-Z]`	匹配任意字母	大小写都行
`[^abc]`	匹配**不是** a、b、c 的字符	d、e、1、中.....

预定义字符

符号	含义	等价写法
`\d`	数字（digit）	`[0-9]`
`\D`	非数字	`[^0-9]`

符号	含义	等价写法
`\w`	单词字符（字母、数字、下划线）	`[a-zA-Z0-9_]`
`\W`	非单词字符	`[^a-zA-Z0-9_]`
`\s`	空白字符（空格、制表符、换行）	`[ \t\n\r\f\v]`
`\S`	非空白字符	`[^ \t\n\r\f\v]`
`.`	任意字符（除换行符）	

转义符 \

元字符本身有特殊含义，想匹配元字符字面值，前面加 `\` 转义。

写法	匹配的是
`\.`	真正的点号 `.`
`\*`	真正的星号 `*`
`\\`	真正的反斜杠 `\`
`\?`	真正的问号 `?`

量词

量词用来定义前面的字符 / 模式可以出现多少次

量词	含义	等价于
`\*`	0 次或多次（可有可无，多多益善）	`{0,}`
`+`	1 次或多次（至少一次）	`{1,}`
`?`	0 次或 1 次（有也行没有也行）	`{0,1}`
`{n}`	恰好 n 次	—
`{n,}`	至少 n 次	—
`{n,m}`	n 到 m 次	—

2.6.4 贪婪模式

量词默认是**贪婪**的，能匹配多长就匹配多长。

在量词后面加 `?` 变成**非贪婪**，能匹配多短就匹配多短。

```python
import re

text = "<div>内容1</div><div>内容2</div>"

# 贪婪
res = re.findall(r'<div>.*</div>', text)
# ['<div>内容1</div><div>内容2</div>']
print(res)

# 非贪婪（加?）
res = re.findall(r'<div>.*?</div>', text)
# ['<div>内容1</div>', '<div>内容2</div>']
print(res)
```

2.6.5 综合项目

前言

基于正则表达式的信息抽取, 信息抽取（Information Extraction, IE）：从非结构化文本中自动提取出结构化信息。

要素	英文	定义	例子
实体	Entity	文本中具有特定意义的事物	人名、地名、机构名、时间、数字
属性	Attribute	实体的特征或性质	人的年龄、公司的地址、产品的价格
关系	Relation	实体之间的联系	A 任职于 B、A 毕业于 B
事件	Event	发生的事情，包含时间、地点、参与者等	获奖、并购、发布产品

方法一：关键词匹配

最简单粗暴，看文本里有没有指定的关键词。

```python
def keyword_extract(text, keywords):
    """关键词匹配：返回文本中出现的关键词"""
    found = []
    for kw in keywords:
        if kw in text:
            found.append(kw)
    return found

text = "杰弗里·辛顿因深度学习方面的贡献获得诺贝尔奖"
keywords = ["深度学习", "神经网络", "Transformer", "诺贝尔奖"]
print(keyword_extract(text, keywords))
# ['深度学习', '诺贝尔奖']
```

优点：简单、快、可控

缺点：只能精确匹配，变体、同义词、模糊表达都搞不定

方法二：正则表达式匹配

用模式匹配，比关键词灵活得多，能处理有规律但不固定的文本。

项目名称

招聘信息结构化抽取系统，基于正则表达式的信息抽取，信息抽取（Information Extraction, IE）：从非结构化文本中自动提取出结构化信息。

项目任务

企业发布的招聘信息通常是非结构化文本，例如：

星海科技有限公司现招聘Python开发工程师2名，工作地点为上海，月薪15000-25000元，要求本科及以上学历，3年以上开发经验，熟悉Python、MySQL和Linux。请于2026年9月30日前将简历发送至hr@xinghai.com。

```json
{
  "公司名称": "星海科技有限公司",
  "招聘岗位": "Python开发工程师",
  "招聘人数": 2,
  "工作地点": "上海",
  "最低薪资": 15000,
  "最高薪资": 25000,
  "学历要求": "本科及以上",
  "经验要求": "3年以上",
  "技能要求": ["Python", "MySQL", "Linux"],
  "截止日期": "2026年9月30日",
  "联系邮箱": "hr@xinghai.com"
}
```


核心知识结构

实体抽取

实体是文本中具有明确意义的对象。

本项目需要抽取的实体包括：

实体类型	示例
公司实体	星海科技有限公司
岗位实体	Python开发工程师
地点实体	上海
技能实体	Python、MySQL、Linux
时间实体	2026年9月30日
联系方式实体	hr@xinghai.com

属性抽取

属性是对实体特征的描述。

实体	属性	示例
招聘岗位	招聘人数	2名
招聘岗位	最低薪资	15000元
招聘岗位	最高薪资	25000元
招聘岗位	学历要求	本科及以上

实体	属性	示例
招聘岗位	工作经验	3年以上
招聘事件	截止日期	2026年9月30日

关系抽取

关系表示两个实体之间的联系。

本项目重点识别以下关系：

公司 ——招聘——> 岗位
岗位 ——工作地点——> 城市
岗位 ——要求掌握——> 技能
岗位 ——要求学历——> 学历
岗位 ——要求经验——> 工作年限

例如：

星海科技有限公司 ——招聘——> Python开发工程

Python开发工程师 ——要求掌握——> Python

Python开发工程师 ——工作地点——> 上海

事件抽取

事件是由多个实体和属性共同组成的完整业务行为。

本项目中的核心事件是：

招聘事件

招聘事件由以下要素组成：事件不是某一个字段，而是多个实体、属性和关系组成的完整信息集合。

事件要素	内容
事件主体	招聘公司
事件动作	招聘
事件对象	招聘岗位
招聘数量	招聘人数
发生地点	工作地点
薪资条件	薪资范围

事件要素	内容
任职条件	学历、经验、技能
截止时间	简历投递截止日期
联系方式	邮箱或电话

项目代码

```
"""
正则信息抽取项目
"""
import re

texts = [
    """星海科技有限公司现招聘Python开发工程师3名，
    工作地点为北京市，月薪15000-20000元，
    要求本科及以上学历，具有2年以上Python开发经验，
    熟悉Python、Django和MySQL。
    请于2026年8月20日前将简历发送至hr@xinghai.com。""",

    """华星数据有限公司诚聘数据分析师2名，工作地点为上海市，
    薪资12000至18000元/月，要求本科及以上学历，
    具有1年以上数据分析经验，熟练掌握SQL、Excel和Python。
    简历请发送至job@huaxing.com，截止日期为2026年9月10日。""",

    """未来智能集团招聘算法工程师若干名，工作地点：深圳市，
    月薪20000-30000元，要求硕士及以上学历，具有3年以上算法开发经验，
    熟悉PyTorch、TensorFlow等深度学习框架。
    请于2026年10月15日前联系hr@future-ai.cn。""",
]


def extract_and_print(text):
    result = {}

    # 1. 公司名称
    pattern = r"([\u4e00-\u9fa5A-Za-z0-9（）()]+?(?:有限责任公司|股份有限公司|有限公司|集团|公司))"
    matches = re.findall(pattern, text)
    result["公司名称"] = matches[0] if matches else None
```

```python

    # 2. 招聘岗位
    pattern = r"(?:招聘|诚聘)\s*([\u4e00-\u9fa5A-Za-z0-9+#.]+?)\s*(?:\d+\s*名|若干名)"
    matches = re.findall(pattern, text)
    result["招聘岗位"] = matches[0] if matches else None

    # 3. 招聘人数
    pattern = r"(\d+|若干)\s*名"
    matches = re.findall(pattern, text)
    if matches:
        num = matches[0]
        result["招聘人数"] = int(num) if num.isdigit() else num
    else:
        result["招聘人数"] = None

    # 4. 工作地点
    pattern = r"工作地点\s*(?:为|是|：|:)?\s*([\u4e00-\u9fa5]{2,10}?(?:省|市|区|县)?)"
    matches = re.findall(pattern, text)
    result["工作地点"] = matches[0] if matches else None

    # 5. 薪资范围
    pattern = r"(?:月薪|薪资|工资)\s*(?:为|是|：|:)?\s*(\d{4,6})\s*(?:-|—|~|至|到)\s*(\d{4,6})"
    matches = re.findall(pattern, text)
    if matches:
        result["最低薪资"] = int(matches[0][0])
        result["最高薪资"] = int(matches[0][1])
    else:
        result["最低薪资"] = result["最高薪资"] = None

    # 6. 学历要求
    pattern = r"((?:高中|中专|大专|本科|硕士|博士)(?:及以上|以上)?)"
    matches = re.findall(pattern, text)
    result["学历要求"] = matches[0] if matches else None

    # 7. 工作经验
    pattern = r"(\d+\s*年以上)[\u4e00-\u9fa5]{0,10}?经验"
    matches = re.findall(pattern, text)
```

```
    result["经验要求"] = matches[0].replace(" ", "") if matches else None

    # 8. 技能要求
    pattern = r"(?:熟悉|掌握|精通)\s*([A-Za-z0-9+#.\-、，,/ 和]+?)(?:等|。|，|；|请于|$)"
    matches = re.findall(pattern, text)
    if matches:
        skills_str = matches[0].strip().replace("和", "、")
        result["技能要求"] = [s.strip() for s in re.split(r"[、，,/]", skills_str) if s.strip()]
    else:
        result["技能要求"] = None

    # 9. 截止日期
    pattern = r"(\d{4}年\d{1,2}月\d{1,2}日)"
    matches = re.findall(pattern, text)
    result["截止日期"] = matches[0] if matches else None

    # 10. 联系邮箱
    pattern = r"([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})"
    matches = re.findall(pattern, text)
    result["联系邮箱"] = matches[0] if matches else None

    # 结构1：结构化抽取结果（字典）
    structured = result

    # 结构2：实体列表 [(实体类型, 实体值), ...]
    entities = [
        ("公司实体", result["公司名称"]),
        ("岗位实体", result["招聘岗位"]),
        ("地点实体", result["工作地点"]),
        ("时间实体", result["截止日期"]),
        ("联系方式实体", result["联系邮箱"]),
    ]
    if result["技能要求"]:
        for skill in result["技能要求"]:
            entities.append(("技能实体", skill))

    # 结构3：属性列表 [(主体, 属性名, 属性值), ...]
    attributes = [
```

```
        ("招聘岗位", "招聘人数", result["招聘人数"]),
        ("招聘岗位", "最低薪资", result["最低薪资"]),
        ("招聘岗位", "最高薪资", result["最高薪资"]),
        ("招聘岗位", "学历要求", result["学历要求"]),
        ("招聘岗位", "工作经验", result["经验要求"]),
    ]

    # 结构4：关系三元组列表 [(主体, 关系, 客体), ...]
    relations = [
        (result["公司名称"], "招聘", result["招聘岗位"]),
        (result["招聘岗位"], "工作地点", result["工作地点"]),
        (result["招聘岗位"], "要求学历", result["学历要求"]),
        (result["招聘岗位"], "要求经验", result["经验要求"]),
    ]
    if result["技能要求"]:
        for skill in result["技能要求"]:
            relations.append((result["招聘岗位"], "要求掌握", skill))

    # 结构5：招聘事件字典
    event = {
        "事件主体(招聘公司)": result["公司名称"],
        "事件动作": "招聘",
        "事件对象(招聘岗位)": result["招聘岗位"],
        "招聘数量": result["招聘人数"],
        "发生地点": result["工作地点"],
        "薪资条件": f"{result['最低薪资']}-{result['最高薪资']}" if result["最低薪资"] else None,
        "任职条件": {
            "学历": result["学历要求"],
            "经验": result["经验要求"],
            "技能": result["技能要求"],
        },
        "截止时间": result["截止日期"],
        "联系方式": result["联系邮箱"],
    }

    # 打印（全部用循环，没有一条硬编码）
    print("【结构化抽取结果】")
    for k, v in structured.items():
```

```
        print(f"  {k}: {v}")
    print()

    print("【实体抽取】")
    for ent_type, ent_val in entities:
        print(f"  {ent_type}: {ent_val}")
    print()

    print("【属性抽取(岗位实体的属性)】")
    for subj, attr, val in attributes:
        print(f"  {subj} --{attr}--> {val}")
    print()

    print("【关系三元组】")
    for subj, rel, obj in relations:
        print(f"  ({subj}) --{rel}--> ({obj})")
    print()

    print("【招聘事件】")
    for k, v in event.items():
        print(f"  {k}: {v}")


for i, text in enumerate(texts, 1):
    print("=" * 60)
    print(f"第 {i} 条招聘信息")
    print("=" * 60)
    extract_and_print(text)
    print()

```

2.7 关键词提取

关键词提取是提取出代表文章重要内容的一组词，对文本聚类、分类、自动摘要起到重要作用。关键词提取还能使人们便捷地浏览和获取信息。

关键词提取包括有监督学习、无监督学习方法两类。

2.7.1 有监督关键词提取

该方法主要通过分类方式进行，通过构建一个较为丰富完整的词表，然后通过判断每个文档与词表中每个词的匹配程度，以类似打标签的方式，达到关键词提取的效果。

该方法能获取较高的精度，但需要对大量样本进行标注，人工成本过高。

2.7.2 无监督关键词提取

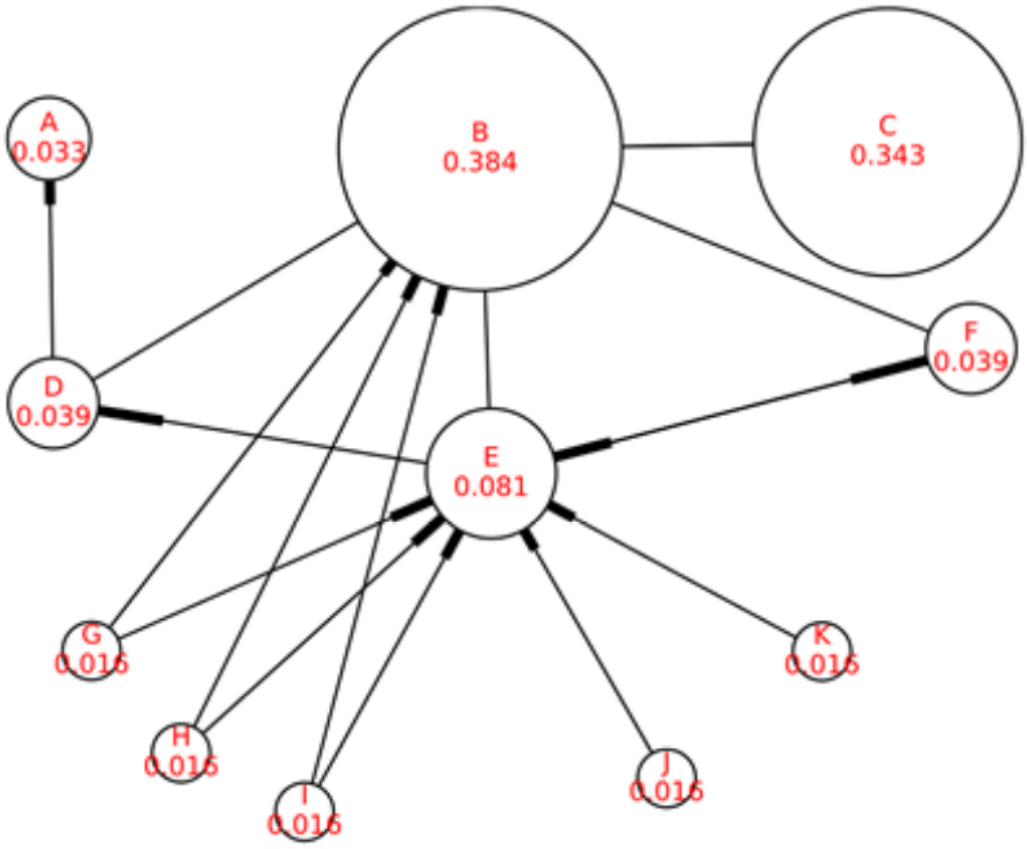
相对于有监督关键词提取，无监督方法对数据要求低得多，既不需要人工维护词表，也不需要人工标注语料辅助训练。无监督关键词提取算法，包括TF-IDF算法(后面会讲)， TextRank算法

2.7.3 TextRank算法

定义

TextRank算法可以脱离于语料库，仅对单篇文档进行分析就可以提取该文档的关键词，这也是 TextRank算法的一个重要特点。

TextRank算法最早用于文档的自动摘要，基于句子维度的分析，利用算法对每个句子进行打分，挑选出分数最高的n个句子作为文档的关键句，以达到自动摘要的效果。



公式

$$WS(V_i) = (1-d) + d \sum_{V_j \in In(V_i)} \frac{WS(V_j)}{|Out(V_j)|}$$

- d：阻尼系数，一般取 0.85
- $In(V_i)$：所有指向词i的词集合
- $|Out(V_j)|$：词j向外连出的边的总数量

3. 基于统计的任务范式

3.1 任务流程

基于统计的任务范式（Statistical Paradigm in NLP）通常遵循数据驱动的流程，依赖于大量的语料库和统计模型来处理和分析自然语言任务。整个流程可以分为多个步骤，每一步都涉及到数据处理、特征提取、模型训练和评估。以下是基于统计的NLP任务范式的典型流程：

3.1.1 数据收集与预处理

统计方法依赖于大量的标注数据，因此数据的收集和预处理是一个非常重要的步骤。

- **数据收集**：首先需要收集大量的文本数据，数据可以来自不同的来源，如网络、社交媒体、书籍、新闻、问答系统等。
- **文本清洗**：包括去除HTML标签、特殊字符、非标准符号，处理换行符、空格等。
- **分词**：将文本分解成更小的单位（如单词或词语），对于中文等语言需要使用分词工具。
- **去停用词**：去除不携带有用信息的常见词（如“的”、“了”、“是”等）。
- **词形还原与词干提取**：将单词转化为标准形式，减少词汇表的复杂度。

3.1.2 特征工程

特征工程（Feature Engineering）是指从原始数据中选择、转换、构造出适用于机器学习模型的特征的过程。

在 NLP 中，原始数据是非结构化的自然语言文本，而模型（尤其是传统机器学习模型）无法直接理解文本，必须将其转化为数值化表示才能进行计算。

3.1.3 学习过程(大致)

基于统计的方法通常依赖于传统的机器学习算法。选择合适的模型并进行训练是整个流程的关键步骤。

1. 选择模型：根据任务的性质选择合适的统计模型。例如：

- 朴素贝叶斯Naive Bayes：适用于分类任务，特别是文本分类和情感分析。
- 支持向量机SVM：用于高维特征空间的分类任务，常用于文本分类和情感分析。
- 隐马尔科夫HMM：HMM主要用于解决序列标注问题，即给定一个观测序列，预测每个观测的标签。
- 条件随机场CRF：适用于序列标注任务，如命名实体识别（NER）和词性标注（POS tagging）。

2. 训练模型：使用训练数据集来训练模型。训练过程包括最大化目标函数或最小化损失函数（如最大似然估计）。

3.评估模型：　评估模型的性能是保证其准确性和鲁棒性的重要步骤。

3.2 文本表示

文本表示，又称文本向量化，是 NLP 特征工程的核心环节，指将非结构化的自然语言文本转换为机器可理解的数值向量的过程，也是把文字变成数字的所有方法总称。

3.2.1 独热编码（One-Hot Encoding)

One-hot 独热编码是一种最简单的文本表示方式。

定义与原理

每个单词表示为一个稀疏的向量，其中向量的维度等于词汇表的大小。若某个位置对应的单词出现，则该位置为1，其他位置为0。

如果有一个大小为V的词表，对于第i个词 ，可以用一个长度为V的向量来表示

例如：

```
减肥：[1, 0, 0, 0, 0]
瘦身：[0, 1, 0, 0, 0]
增重：[0, 0, 1, 0, 0]
减脂：[0, 0, 0, 1, 0]
塑形：[0, 0, 0, 0, 1]
```

优点

One-hot词向量构建简单，简单，易于实现

缺点

- 空间浪费，无法捕捉词语间的相似性，且不适合大规模数据集。
- 维度过高: 如果词数量较多，每个词需要使用更长的向量表示，造成维度灾难；
- 稀疏矩阵: 每个词向量，其中只有一位为1，其它位均为零
- 语义鸿沟: 词语之间的相似度、相关程度无法度量、不表达词的顺序。

```python
"""
独热编码
"""
from sklearn.preprocessing import OneHotEncoder
import numpy as np

words = np.array(["减肥", "瘦身", "增重", "减脂", "塑形"]).reshape(-1, 1)
encoder = OneHotEncoder(sparse_output=False,dtype=int)
encoder.fit(words)

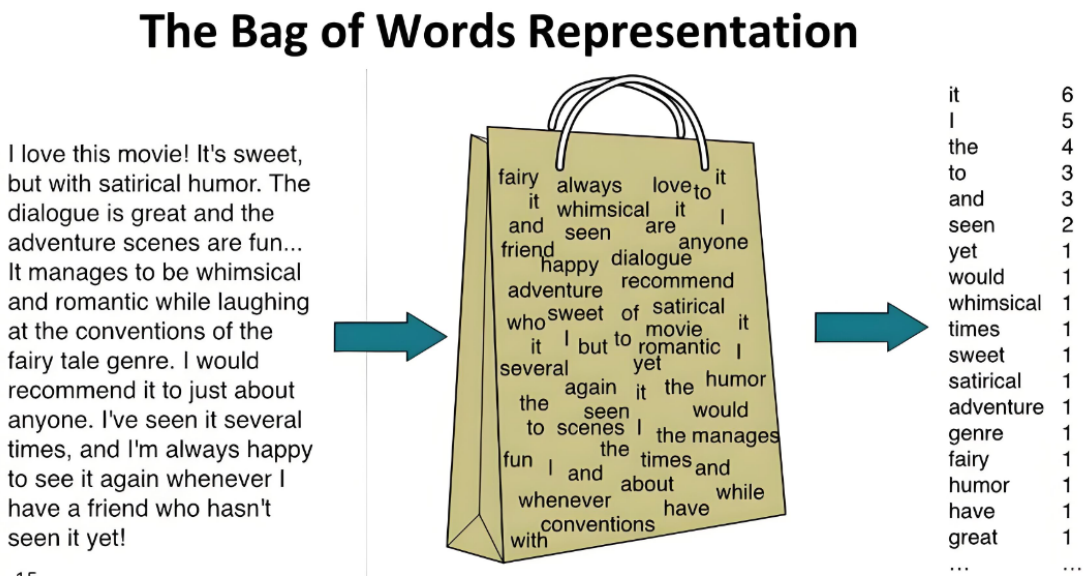
# 把词转成独热向量
encoded = encoder.transform(words)
for i, word in enumerate(["减肥", "瘦身", "增重", "减脂", "塑形"]):
    print(f"{word}: {encoded[i].tolist()}")

```

运行结果

```
减肥：[1, 0, 0, 0, 0]
瘦身：[0, 0, 0, 0, 1]
增重：[0, 0, 0, 1, 0]
减脂：[0, 1, 0, 0, 0]
塑形：[0, 0, 1, 0, 0]
```

3.2.2 词袋模型（Bag-of-Words, BoW）



定义与原理

词袋模型(Bag-of-words model，BOW)，BOW模型假定对于一个文档，忽略它的单词顺序和语法、句法等要素，将其仅仅看作是若干个词汇的集合，文档中每个单词的出现都是独立的，不依赖于其它单词是否出现。

代码

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

documents = [
    "我 爱 减肥 减肥",
    "减肥 瘦身 都 要 运动",
    "增肌 增重 吃 蛋白"
]

# 创建词袋模型
vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")

```

```
X = vectorizer.fit_transform(documents)

# 查看词表
print(vectorizer.get_feature_names_out())

# 用DataFrame展示，更直观
df = pd.DataFrame(
    X.toarray(),
    columns=vectorizer.get_feature_names_out(),
    index=["文档1", "文档2", "文档3"]
)

print(df)
```

运行结果

```
词表:
['减肥' '吃' '增肌' '增重' '我' '爱' '瘦身' '蛋白' '要' '运动' '都']

      减肥  吃  增肌  增重  我  爱  瘦身  蛋白  要  运动  都
文档1   2  0   0   0  1  1   0   0  0   0  0
文档2   1  0   0   0  0  0   1   0  1   1  1
文档3   0  1   1   1  0  0   0   1  0   0  0
```

3.2.3 二元特征（Binary Feature）

定义与原理

二元特征（Binary Feature），也叫**二元词袋模型**，是词袋模型的简化版本。

公式

$$\text{Binary}(t,d) = \begin{cases} 1 & \text{如果词} t \text{ 在文档 } d \text{ 中出现} \\ 0 & \text{如果词} t \text{ 在文档 } d \text{ 中未出现} \end{cases}$$

代码

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

documents = [
```

```
    "我 爱 减肥 减肥 减肥",
    "减肥 瘦身 都 要 运动",
    "增肌 增重 吃 蛋白",
]

# 词袋模型
bow_vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X_bow = bow_vectorizer.fit_transform(documents)

df_bow = pd.DataFrame(
    X_bow.toarray(),
    columns=bow_vectorizer.get_feature_names_out(),
    index=["文档1", "文档2", "文档3"]
)

# 二元特征
binary_vectorizer = CountVectorizer(
    token_pattern=r"(?u)\b\w+\b",  # 分词规则：保留单个字
    binary=True  # 二元模式：非0的值都变成1
)
X_binary = binary_vectorizer.fit_transform(documents)

df_binary = pd.DataFrame(
    X_binary.toarray(),
    columns=binary_vectorizer.get_feature_names_out(),
    index=["文档1", "文档2", "文档3"]
)
print(df_binary)

# 对比两者的区别
print("对比：文档1中'减肥'这个词")
print(f"  词袋模型：{df_bow.loc['文档1', '减肥']} 次")
print(f"  二元特征：{df_binary.loc['文档1', '减肥']}")
```

3.2.4 词频（Term Frequency, TF）

定义与原理

词频（TF）是指某个词在文档中出现的次数，它简单地衡量了词在文档中的重要性。

为了避免文档长度的影响，词频是通过词在文档中的出现次数除以文档中总词数来计算

公式

$$\mathrm{TF}(t,d) = \frac{\text{词 } t \text{ 在文档 } d \text{ 中出现的次数}}{\text{文档 } d \text{ 中所有词的总数}}$$

代码

```python
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

data = [
    'I love programming with Python and I love machine learning',
    'I love Python as well'
]
vec = CountVectorizer()
X = vec.fit_transform(data)

# 转化成特征矩阵
df = pd.DataFrame(X.toarray(), columns=vec.get_feature_names_out())
print(df)
print()
norm_df = df.div(df.sum(axis=1), axis=0,)
print(norm_df)
```

运行结果

```
   and  as  learning  love  machine  programming  python  well  with
0    1   0         1     2        1            1       1     0     1
1    0   1         0     1        0            0       1     1     0

      and    as  learning  love  machine  programming  python  well   with
0  0.125  0.00     0.125  0.25    0.125        0.125   0.125  0.00  0.125
1  0.000  0.25     0.000  0.25    0.000        0.000   0.250  0.25  0.000
```

3.2.5 逆文档频率（Inverse Document Frequency, IDF）

定义

IDF 是一种权衡方法，用来衡量某个词对文档的区分性。

IDF 的思想是，如果一个词出现在很多文档中，它的区分性就较低，因此它的重要性较低。

IDF 通过减少频繁出现的词的权重来强调那些较少出现的词。

$$\text{IDF}(t) = \log\left(\frac{N}{df(t)}\right)$$

- N：文档集合中的文档总数。
- $df(t)$：包含词语 t 的文档数量。

计算步骤

假设有 4 个文档：

```
Document 1: "I love programming with Python"
Document 2: "Python programming is great"
Document 3: "I love machine learning with Python"
Document 4: "Machine learning is fun"
```

我们计算词语 "Python" 和 "machine" 的 **IDF**。

- 总文档数 (N = 4)

- 词 "Python" 出现的文档数 df("Python") = 3（出现在 Document 1, 2, 3）
- 词 "machine" 出现的文档数 df("machine") = 2（出现在 Document 3, 4）
- 计算的时候可以使用python的math计算

```
import math
print(math.log10(1.33))
print(math.log10(2))
print(math.log10(4))
```

- 计算 "Python" 的 IDF：

$$\text{IDF}(\text{Python}) = \log\left(\frac{4}{3}\right) \approx 0.12$$

- 计算 "machine" 的 IDF：

$$\text{IDF}(\text{machine}) = \log\left(\frac{4}{2}\right) \approx 0.30$$

代码

```
from sklearn.feature_extraction.text import
CountVectorizer, TfidfTransformer
import pandas as pd

# 准备文档
documents = [
    "我 爱 减肥",
    "减肥 瘦身 都 要 运动",
    "增肌 增重 吃 蛋白",
    "减肥 增肌 都 可以",
    "运动 减肥 瘦身",
]

# 第一步：用词袋模型统计词频
vec = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X_count = vec.fit_transform(documents)

# 第二步：用TfidfTransformer计算IDF
tfidf_transformer = TfidfTransformer(smooth_idf=True)
```

```
tfidf_transformer.fit(X_count)

# 获取IDF值
idf_values = tfidf_transformer.idf_
feature_names = vec.get_feature_names_out()

# 用DataFrame展示
df_idf = pd.DataFrame({
    '词语': feature_names,
    'IDF值': idf_values
}).sort_values('IDF值', ascending=False).reset_index(drop=True)
```

3.2.6 TF-IDF（Term Frequency-Inverse Document Frequency）

定义与原理

TF-IDF 是将词频和逆文档频率结合起来的指标(TF × IDF，两个值乘起来)，常用于评估一个文档集中一个词对某份文档的重要程度(语义贡献度)。

它的思想是，如果一个词在某个文档中频繁出现，并且在语料库中出现频率较低，那么这个词在该文档中就具有较高的权重。

公式

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

公式推导

词频:

$$TF_{ij} = \frac{n_{ji}}{\sum_k n_{kj}}$$

逆文档率:

$$IDF_i = \log\left(\frac{|D|}{|D_i|+1}\right)$$

词频逆文档率:

$$TF-IDF = TF_{ij} \times IDF_i = \frac{n_{ji}}{\sum_k n_{kj}} \times \log\left(\frac{|D|}{|D_i|+1}\right)$$

三者关系

指标	衡量什么	单独用的问题
TF	词在本文档里的出现频率	停用词频率最高，但没用
IDF	词的通用程度 / 辨识度	只看全局，不看本文档
TF-IDF	词对本文档的重要程度	解决了上面两个的问题

例如

```
世界献血日，学校团体、献血服务志愿者等可到血液中心参观检验加工过程，我们会对检验结果进行公示，同时血液的价格也将进行公示。
```

以上文本词语总数为30，计算几个词的词频：

$$TF_{\text{献血}} = 2/30 \approx 0.067$$
$$TF_{\text{血液}} = 2/30 \approx 0.067$$
$$TF_{\text{进行}} = 2/30 \approx 0.067$$
$$TF_{\text{公示}} = 2/30 \approx 0.067$$

假设语料库共1000个文档，出现献血、血液、进行、公示文档数量分别为10、15、100、50，根据TF-IDF计算公式：

$$TF-IDF_{\text{献血}} = 0.067 * log(1000/10) = 0.067 * 2 = 0.134$$
$$TF-IDF_{\text{血液}} = 0.067 * log(1000/15) = 0.067 * 1.824 = 0.1222$$
$$TF-IDF_{\text{进行}} = 0.067 * log(1000/100) = 0.067 * 1 = 0.067$$
$$TF-IDF_{\text{公示}} = 0.067 * log(1000/50) = 0.067 * 1.30 = 0.08717$$

“献血”、“血液”的TF-IDF值最高，所以为最适合这篇文档的关键词。

代码

```python
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

# 假设有5篇文档
```

```
documents = [
    "我 爱 减肥",  # 文档1
    "减肥 瘦身 都 要 运动",  # 文档2
    "增肌 增重 吃 蛋白",  # 文档3
    "减肥 增肌 都 可以",  # 文档4
    "运动 减肥 瘦身",  # 文档5
]

# 创建TF-IDF向量化器
vectorizer = TfidfVectorizer(
    token_pattern=r"(?u)\b\w+\b",
    smooth_idf=True,
    norm=None,
    sublinear_tf=False
)

# 拟合+转换
X = vectorizer.fit_transform(documents)

# 查看结果
feature_names = vectorizer.get_feature_names_out()
# 转成DataFrame，方便查看
df_tfidf = pd.DataFrame(
    X.toarray(),
    columns=feature_names,
    index=[f"文档{i + 1}" for i in range(len(documents))]
)
print(df_tfidf.round(4))  # 保留4位小数


# 查看每个词的IDF值
idf_df = pd.DataFrame({
    '词语': feature_names,
    'IDF值': vectorizer.idf_
}).sort_values('IDF值',
ascending=False).reset_index(drop=True)

print(idf_df.to_string(index=False))

```

3.2.7 N-Grams

定义与原理

N-Gram（N 元模型）是一种基于统计的文本表示方法，核心思想是：把连续的 N 个词（或字）当作一个整体单元来统计。一个词出现的概率，只跟它前面的 N-1 个词有关，跟更早的词没关系(马尔可夫假设)

实际上N-Gram模型也是一种基于统计语言模型，语言模型是一个基于概率的判别模型，它的输入是个句子（由词构成的顺序序列），输出是这句话的概率，即这些单词的联合概率。

N-Gram本身也指一个由N个单词组成的集合，各单词具有先后顺序，且不要求单词之间互不相同。常用的有Bi-gram （N=2）和Tri-gram（N=3）

公式

- **统一表示**

$$P(w_1, w_2, \ldots, w_m) = \prod_{i=1}^{m} P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$$

- **N=1** → Unigram（一元）：单个词，就是普通的词袋模型，每个词语出现的概率是独立的

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{m} P(w_i)$$

- **N=2** → Bigram（二元）：连续两个词的组合，计算第二个词时，只考虑前面1个词

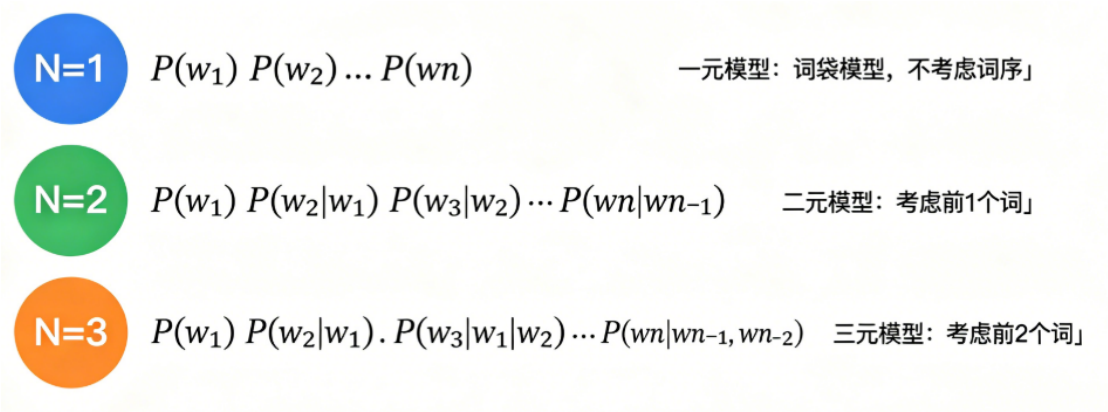
$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{m} P(w_i|w_{i-1})$$

- **N=3** → Trigram（三元）：连续三个词的组合

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{m} P(w_i|w_{i-2}, w_{i-1})$$

- **N=4** → 4-gram（四元）：连续四个词的组合

N值越大，保留的词序信息（上下文）越丰富，计算量呈指数级增长，特征维度爆炸， 数据越稀疏，越容易过拟合



优点

- 简单：就是统计频率，原理简单，实现简单
- 保留词序：比词袋模型多了局部顺序信息
- 有效：在很多任务上，加了 Bigram/Trigram 效果确实更好
- 可解释：每个特征都是可理解的词或短语

缺点

- 维度爆炸：N 越大，特征数量指数级增长（$V \rightarrow V^2 \rightarrow V^3$）
- 数据稀疏：大部分 N-Gram 在语料里从没出现过，都是 0
- 只能看局部：只能看到前面 N-1 个词，长距离依赖搞不定
- 泛化差：没见过的组合就预测不了

代码

```
from sklearn.feature_extraction.text import
CountVectorizer
import pandas as pd

# 准备文档
documents = [
    "我 爱 减肥",
    "我 爱 运动",
    "我 吃 饭",
    "你 爱 减肥",
    "你 爱 运动",
]

```

```python
# Unigram (N=1)
unigram_vec = CountVectorizer(
    token_pattern=r"(?u)\b\w+\b",
    ngram_range=(1, 1)
)
X_unigram = unigram_vec.fit_transform(documents)

df_unigram = pd.DataFrame(
    X_unigram.toarray(),
    columns=unigram_vec.get_feature_names_out(),
    index=[f"文档{i + 1}" for i in range(len(documents))]
)

# Bigram (N=2)
bigram_vec = CountVectorizer(
    token_pattern=r"(?u)\b\w+\b",
    ngram_range=(2, 2)
)
X_bigram = bigram_vec.fit_transform(documents)

df_bigram = pd.DataFrame(
    X_bigram.toarray(),
    columns=bigram_vec.get_feature_names_out(),
    index=[f"文档{i + 1}" for i in range(len(documents))]
)

# Unigram + Bigram 混合
mixed_vec = CountVectorizer(
    token_pattern=r"(?u)\b\w+\b",
    ngram_range=(1, 2)
)
X_mixed = mixed_vec.fit_transform(documents)

df_mixed = pd.DataFrame(
    X_mixed.toarray(),
    columns=mixed_vec.get_feature_names_out(),
    index=[f"文档{i + 1}" for i in range(len(documents))]
)
```

3.2.8 共现矩阵（Co-occurrence Matrix）

定义与原理

共现矩阵是一种基于统计的词表示方法，核心思想是：统计一个事先指定大小的窗口内(窗口大小 = 2当前词左右各 2 个词)的词语共现次数，以词语周边的共现词的次数做为当前词语的向量，表示词的语义。

例如，有语料如下：

```
I like deep learning
I like NLP
I enjoy flying
```

则共现矩阵(两个词相交的地方就是共现矩阵)表示为：

counts	I	like	enjoy	deep	learning	NLP	flying
I	0	2	1	0	0	0	0
like	2	0	0	1	0	1	0
enjoy	1	0	0	0	0	0	1
deep	0	1	0	0	1	0	0
learning	0	0	0	1	0	0	0
NLP	0	1	0	0	0	0	0
flying	0	0	1	0	0	0	0

代码

```
# 语料
corpus = [
    "我 爱 减肥",
    "我 爱 运动",
    "减肥 需要 运动",
]

# 窗口大小
window_size = 1

# 构建词表
vocab = list(set(word for sent in corpus for word in sent.split()))
vocab.sort()

```

```
# 初始化矩阵
co_matrix = {}

for sent in corpus:
    words = sent.split()
    # 遍历每个词当中心词
    for i, center in enumerate(words):
        # 窗口范围：左边 max(0, i-窗口) 到 右边 min(总词数-1, i+窗口)
        start = max(0, i - window_size)
        end = min(len(words) - 1, i + window_size)
        # 遍历窗口内的上下文词
        for j in range(start, end + 1):
            if i != j:  # 排除自己
                context = words[j]
                key = (center, context)  # 词对：(中心词, 上下文词)
                co_matrix[key] = co_matrix.get(key, 0) + 1  # 计数+1

# 打印结果
print("共现矩阵（词对 → 次数）：")
for (center, context), count in sorted(co_matrix.items()):
    print(f"  {center} → {context}: {count} 次")
```

3.2.9 GloVe

定义与原理

GloVe（Global Vectors for Word Representation）是斯坦福大学 2014 年提出的词嵌入方法，核心思想是：基于词的共现矩阵来学习词向量。

优点

- 训练快：基于矩阵运算，比 Word2Vec 的逐窗口训练快
- 利用全局信息：利用了整个语料库的统计信息
- 可解释性强：基于共现统计，原理清晰
- 大语料表现好：语料越大，效果越好

缺点

- 内存占用大：需要构建共现矩阵，大词表很吃内存
- 小语料不如 Word2Vec：小语料上 Word2Vec 的增量学习效果更好
- 新词处理麻烦：加新词需要重新训练整个模型

3.3 实现任务

3.3.1 TF-IDF情感分析

(1) 项目概述

情感分析（Sentiment Analysis）是自然语言处理中的经典任务，目标是判断一段文本表达的情感倾向，是正面的、负面的，还是中性的。

本项目我们将使用统计范式的方法来实现情感分析：

- **文本表示**：使用 TF-IDF 将句子转换为稀疏向量
- **分类模型**：使用朴素贝叶斯（Naive Bayes）统计学习模型进行情感分类
- **任务目标**：给定一个句子，判断它是"正面情感"还是"负面情感"

(2) 项目流程

统计范式情感分析的整体流程可以分为以下几步：

1. 文本预处理：对原始文本进行分词、去停用词等处理

2. 特征提取：使用 TF-IDF 将文本转换为向量表示

3. 模型训练：用标注好的训练数据训练分类器

4. 预测分类：用训练好的模型对新文本进行情感预测

5. 评估效果：用测试数据评估模型的准确率

(3) 代码实现

```
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
import jieba

# 训练数据
train_data = [
```

```
    ("这款手机速度很快，拍照也很好", 1),
    ("性价比高，推荐大家购买", 1),
    ("外观好看，系统流畅", 1),
    ("手机很流畅，拍照清晰", 1),
    ("续航不错，值得入手", 1),
    ("屏幕清晰，手感很好", 1),
    ("音质棒，听歌体验好", 1),
    ("充电快，用着方便", 1),
    ("做工精细，质感高级", 1),
    ("运行流畅，不卡顿", 1),
    ("拍照效果好，夜景清晰", 1),
    ("电池耐用，一天够用", 1),
    ("界面美观，操作简单", 1),
    ("信号稳定，通话清晰", 1),
    ("价格实惠，物超所值", 1),
    ("电池不行，经常卡顿", 0),
    ("质量太差，非常不满意", 0),
    ("发货慢，客服态度不好", 0),
    ("垃圾产品，千万别买", 0),
    ("屏幕易碎，售后差", 0),
    ("用了一周就坏了，差评", 0),
    ("发热严重，烫手", 0),
    ("续航太差，一天三充", 0),
    ("拍照模糊，不如宣传", 0),
    ("系统卡顿，经常死机", 0),
    ("充电慢，充一晚上才满", 0),
    ("做工粗糙，缝隙大", 0),
    ("信号差，经常断网", 0),
    ("价格虚高，不值这个钱", 0),
    ("售后推诿，不给退货", 0),
]

# 测试数据
test_data = [
    ("手机很流畅，拍照清晰", 1),
    ("垃圾产品，千万别买", 0),
    ("续航不错，值得入手", 1),
    ("屏幕易碎，售后差", 0),
]

# 分开文本和标签
train_texts, train_labels = zip(*train_data)
test_texts, test_labels = zip(*test_data)
```

```python

# TF-IDF 特征提取
vectorizer = TfidfVectorizer(
    tokenizer=jieba.lcut,
    token_pattern=None,
    ngram_range=(1, 2),
    min_df=1
)

X_train = vectorizer.fit_transform(train_texts)  # 训练集：学词表+转向量
X_test = vectorizer.transform(test_texts)  # 测试集：用学来的词表转向量

# 训练朴素贝叶斯
classifier = MultinomialNB(alpha=0.5)
classifier.fit(X_train, train_labels)

# 预测
predictions = classifier.predict(X_test)
probabilities = classifier.predict_proba(X_test)

# 输出结果
correct = 0
for i, (text, true_label) in enumerate(test_data):
    pred = predictions[i]
    conf = max(probabilities[i])
    status = "✓" if pred == true_label else "✗"

    print(f"{status} {text}")
    print(f"   真实：{'正面' if true_label else '负面'}  "
          f"预测：{'正面' if pred else '负面'}  "
          f"置信度：{conf:.2%}")
    print()

    if pred == true_label:
        correct += 1

print(f"准确率：{correct / len(test_data):.2%} ({correct}/{len(test_data)})")
```

运行结果示例：

```
✓ 手机很流畅，拍照清晰
    真实：正面   预测：正面   置信度：91.03%

✓ 垃圾产品，千万别买
    真实：负面   预测：负面   置信度：81.69%

✓ 续航不错，值得入手
    真实：正面   预测：正面   置信度：79.45%

✓ 屏幕易碎，售后差
    真实：负面   预测：负面   置信度：83.58%

准确率：100.00% (4/4)
```

(4) 影响效果的因素

情感分析的效果受很多因素影响：

- **数据质量**：训练数据越多、标注越准确，模型效果越好
- **分词效果**：好的分词能提升特征质量（单字分词 vs 词语分词）
- **特征选择**：是否去除停用词、是否加入 N-gram 特征
- **模型选择**：不同的分类器适用于不同的数据分布
- **领域适配**：在手机评论上训练的模型，直接用在电影评论上效果可能下降

(5) 局限性

统计范式的情感分析有一些固有的局限性：

- **无法理解语义**：基于词的匹配，无法理解"好得不能再好了"和"好得我都想扔了"的区别
- **无法处理否定**："不好"和"好"在词袋模型中是两个独立的特征，模型很难学到否定关系
- **依赖特征工程**：需要人工设计特征（如情感词典、否定词处理等）
- **泛化能力有限**：训练数据中没见过的表达方式很难处理

3.3.2 文本相似度计算

(1) 项目概述

文本相似度（Text Similarity）是自然语言处理中的基础任务，目标是衡量两段文本之间的相似程度。它在很多场景中都有应用：

- **信息检索**：查询和文档的相似度排序
- **问答系统**：找到与问题最相似的已有答案
- **文本去重**：找出重复或高度相似的文档
- **推荐系统**：推荐相似的文章、商品描述
- **抄袭检测**：检测文本之间的抄袭程度

本项目我们将实现两种不同粒度的文本相似度计算方法：

方法	适用粒度	原理
Jaccard 相似度	短句级	基于词集合的交集与并集之比
稀疏句向量法	段落/文档级	基于 TF-IDF 向量的余弦相似度

(2) 短句级相似度 Jaccard

Jaccard 相似度（Jaccard Similarity），也叫 Jaccard 系数，是衡量两个集合相似程度的经典指标。

例子

```
句子A：我 爱 自然语言处理
句子B：我 爱 学习

集合A = {我, 爱, 自然语言处理}
集合B = {我, 爱, 学习}

交集 = {我, 爱} → 2个元素
并集 = {我, 爱, 自然语言处理, 学习} → 4个元素

Jaccard 相似度 = 2 / 4 = 0.5
```

Jaccard 相似度的取值范围是 [0, 1]：

- **0**：两个文本完全没有共同的词，完全不相似
- **1**：两个文本的词完全一样，完全相似
- **0.5**：一半的词是共同的

代码

```
import jieba

test_cases = [
```

```
    ("我爱自然语言处理", "我爱学习"),
    ("今天天气很好", "今天天气不错"),
    ("机器学习很有趣", "深度学习很有用"),
    ("苹果香蕉橘子", "猫狗鸟"),
    ("我爱编程", "我爱编程"),
]

for text1, text2 in test_cases:
    set1 = set(jieba.lcut(text1))
    set2 = set(jieba.lcut(text2))

    # 交集 / 并集
    sim = len(set1 & set2) / len(set1 | set2) if (set1 | set2) else 0

    print(f"句子1: {text1}")
    print(f"句子2: {text2}")
    print(f"Jaccard 相似度: {sim:.4f}")
```

运行结果：

```
句子1: 我爱自然语言处理
句子2: 我爱学习
Jaccard 相似度: 0.4000
句子1: 今天天气很好
句子2: 今天天气不错
Jaccard 相似度: 0.2500
句子1: 机器学习很有趣
句子2: 深度学习很有用
Jaccard 相似度: 0.3333
句子1: 苹果香蕉橘子
句子2: 猫狗鸟
Jaccard 相似度: 0.0000
句子1: 我爱编程
句子2: 我爱编程
Jaccard 相似度: 1.0000
```

优缺点分析

优点：

- 极其简单：原理和实现都非常简单，几行代码就能搞定
- 计算快：集合操作效率很高，适合大规模计算

- 不需要训练：无监督方法，不需要标注数据
- 直观易懂：结果容易解释

缺点：

- 丢失词频信息：只考虑词是否出现，不考虑出现多少次。
- 丢失词序信息：词的顺序完全不影响结果。
- 无法捕捉语义相似："快乐"和"高兴"意思很接近，但字面上不一样，Jaccard 相似度为 0
- 对短句效果尚可，长句效果差：句子越长，共同词的比例越低，Jaccard 相似度普遍偏低

(3) 段落/文档级相似度 稀疏句向量法

核心思路

对于较长的文本（段落或文档），Jaccard 方法就不够用了。我们需要一种更精细的方法，能够：

- 考虑词的重要性差异（TF-IDF 权重）
- 考虑词频信息
- 能处理较长的文本

稀疏句向量法的核心思路是：

用 TF-IDF 将每段文本表示为一个稀疏向量, 然后计算两个向量之间的余弦相似度（Cosine Similarity）

余弦相似度原理

余弦相似度衡量两个向量在方向上的相似程度。它的几何意义是：两个向量之间夹角的余弦值。

计算公式：

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|} = \frac{\sum_{i=1}^{n} A_i \cdot B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}$$

其中：

- ‖A‖ 和 ‖B‖ 是两个向量的模长（L2 范数）

余弦相似度的取值范围也是 [-1, 1]：

- **1**：两个向量方向完全相同，完全相似
- **0**：两个向量正交（垂直），没有相似性
- **-1**：两个向量方向完全相反

在文本相似度中，因为 TF-IDF 值都是非负的，所以余弦相似度的范围是 [0, 1]。

代码实现

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 文档
documents = [
    "自然语言处理 是 人工智能 的 重要 分支",
    "机器学习 包括 监督学习 和 无监督学习",
    "深度学习 基于 神经网络 模型",
    "文本分类 是 自然语言处理 的 基础 任务",
    "情感分析 是 文本分类 的 一种",
    "信息检索 研究 如何 查找 相关 文档",
    "计算机视觉 处理 图像 和 视频",
    "推荐系统 帮助 用户 发现 感兴趣 的 内容",
    "数据挖掘 从 大量 数据 中 发现 模式",
    "自然语言生成 可以 自动 生成 文本",
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)

test_pairs = [
    ("自然语言处理 文本分类 情感分析", "自然语言处理 文本分类 信息检索"),
    ("机器学习 深度学习 神经网络", "自然语言处理 文本分类"),
    ("计算机视觉 图像 视频", "推荐系统 数据挖掘"),
    ("情感分析 是 文本分类 的 一种", "情感分析 属于 文本分类 任务"),
]

for text1, text2 in test_pairs:
    vecs = vectorizer.transform([text1, text2])
    sim = cosine_similarity(vecs[0], vecs[1])[0][0]

```

```
    print(f"文本1: {text1}")
    print(f"文本2: {text2}")
    print(f"余弦相似度: {sim:.4f}")

```

运行结果示例：

```
文本1: 自然语言处理 文本分类 情感分析
文本2: 自然语言处理 文本分类 信息检索
余弦相似度: 0.5911

文本1: 机器学习 深度学习 神经网络
文本2: 自然语言处理 文本分类
余弦相似度: 0.0000

文本1: 计算机视觉 图像 视频
文本2: 推荐系统 数据挖掘
余弦相似度: 0.0000

文本1: 情感分析 是 文本分类 的 一种
文本2: 情感分析 属于 文本分类 任务
余弦相似度: 0.6327


```

(4) 找最相似的文档

文本相似度最常见的应用之一就是：给定一个查询，从文档库中找出最相似的文档。这就是信息检索的基本原型。

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 文档库
doc_library = [
    "自然语言处理 是 人工智能 的 重要 分支 研究 人机 交互",
    "机器学习 包括 监督学习 无监督学习 和 强化学习",
    "深度学习 基于 深度 神经网络 能够 自动 学习 特征",
    "文本分类 是 自然语言处理 的 基础 任务 之一",
    "情感分析 判断 文本 的 情感 倾向 正面 或 负面",
    "信息检索 研究 如何 从 大量 文档 中 查找 相关 内容",
    "计算机视觉 让 计算机 理解 图像 和 视频 内容",
    "推荐系统 根据 用户 兴趣 推荐 个性化 内容",
```

```
    "数据挖掘 从 海量 数据 中 发现 隐藏 的 模式",
    "问答系统 自动 回答 用户 提出 的 问题",
]

# 训练 TF-IDF
vectorizer = TfidfVectorizer()
doc_vectors = vectorizer.fit_transform(doc_library)


# 找出与查询最相似的 top_k 个文档
def find_most_similar(query, top_k=3):
    # 查询转向量
    query_vec = vectorizer.transform([query])
    # 一次性算和所有文档的相似度
    similarities = cosine_similarity(query_vec, doc_vectors)[0]
    # 按相似度降序排
    ranked = sorted(enumerate(similarities), key=lambda x: x[1], reverse=True)
    return ranked[:top_k]

# 测试查询
queries = [
    "自然语言处理 文本分类",
    "神经网络 深度学习",
    "用户 推荐 内容",
]

for query in queries:
    results = find_most_similar(query, top_k=3)
    for rank, (doc_idx, sim) in enumerate(results, 1):
        print(f"  {rank}. 相似度: {sim:.4f} | 文档: {doc_library[doc_idx][:30]}...")

```

运行结果示例：

```
查询: 自然语言处理 文本分类
最相似的文档:
  1. 相似度: 0.6040 | 文档: 文本分类 是 自然语言处理 的 基础 任务 之一...
  2. 相似度: 0.2169 | 文档: 自然语言处理 是 人工智能 的 重要 分支 研究 人机 交互...
```

```
   3. 相似度: 0.0000 | 文档: 机器学习 包括 监督学习 无监督学习 和 强化学习...

查询: 神经网络 深度学习
最相似的文档:
   1. 相似度: 0.5089 | 文档: 深度学习 基于 深度 神经网络 能够 自动 学习 特征...
   2. 相似度: 0.0000 | 文档: 自然语言处理 是 人工智能 的 重要 分支 研究 人机 交互...
   3. 相似度: 0.0000 | 文档: 机器学习 包括 监督学习 无监督学习 和 强化学习...

查询: 用户 推荐 内容
最相似的文档:
   1. 相似度: 0.6022 | 文档: 推荐系统 根据 用户 兴趣 推荐 个性化 内容...
   2. 相似度: 0.2053 | 文档: 问答系统 自动 回答 用户 提出 的 问题...
   3. 相似度: 0.1556 | 文档: 计算机视觉 让 计算机 理解 图像 和 视频 内容...

```

(5) 两种方法总结

对比维度	Jaccard 相似度	TF-IDF + 余弦相似度
原理	集合交集/并集	向量夹角余弦
考虑词频	否（只看是否出现）	是（TF 词频）
考虑词重要性	否（所有词同等重要）	是（IDF 权重）
计算复杂度	低	中
适用文本长度	短句	段落、长文档
精度	较低	较高
是否需要语料库	不需要	需要（计算 IDF）

3.3.3 信息检索系统

(1) 项目概述

信息检索（Information Retrieval, IR）是研究如何从大量文档中找到与用户查询相关的信息的学科。

本项目我们将实现一个简单的信息检索系统，涵盖两种不同的实现方式：

实现方式	核心技术	特点
基于规则	布尔检索模型	精确匹配、基于集合运算、结果要么相关要么不相关
基于统计范式	向量空间模型	相似度排序、基于 TF-IDF、结果按相关度排序

两种方式都依赖于同一个核心数据结构: 倒排索引（Inverted Index）。

(2) 倒排索引

原理

要理解"倒排索引"，先想想什么是"正排索引"。

正排索引（Forward Index）：从文档到词的映射。

```
文档1 → [我，爱，自然语言处理]
文档2 → [自然语言处理，很，有趣]
文档3 → [我，爱，学习]
```

倒排索引（Inverted Index）：从词到文档的映射。

```
我 → [文档1，文档3]
爱 → [文档1，文档3]
自然语言处理 → [文档1，文档2]
很 → [文档2]
有趣 → [文档2]
学习 → [文档3]
```

结构

一个完整的倒排索引通常包含两部分：

词典（Dictionary / Vocabulary）：所有出现过的词的列表。每个词指向它的倒排列表。

倒排列表（Posting List）：每个词对应的文档列表。列表中通常包含文档ID、词频、位置信息等。

更完整的倒排列表还会记录词在文档中的位置（用于短语查询）、词频（用于相关性排序）等信息。

构建

构建倒排索引的过程很直观：

1. 遍历每一篇文档
2. 对文档进行分词
3. 对每个词，把当前文档的 ID 添加到该词的倒排列表中
4. 所有文档处理完后，对每个倒排列表按文档 ID 排序（方便后续的集合运算）

代码

```
from sklearn.feature_extraction.text import CountVectorizer

# 文档
documents = {
    "doc1": "自然语言处理 是 人工智能 的 重要 分支",
    "doc2": "机器学习 包括 监督学习 和 无监督学习",
    "doc3": "深度学习 基于 神经网络 模型",
    "doc4": "文本分类 是 自然语言处理 的 基础 任务",
    "doc5": "情感分析 是 文本分类 的 一种",
    "doc6": "信息检索 研究 如何 查找 相关 文档",
}

doc_ids = list(documents.keys())
doc_texts = list(documents.values())

vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X = vectorizer.fit_transform(doc_texts)

# 转置：行=词，列=文档
X_T = X.T.tocsr()

vocab = vectorizer.get_feature_names_out()

# 构建倒排索引字典
inverted_index = {}
for i, word in enumerate(vocab):
    doc_indices = X_T[i].nonzero()[1]
    inverted_index[word] = [doc_ids[idx] for idx in doc_indices]
```

```
29
30 # 输出
31 print(f"倒排索引（共 {len(documents)} 篇文档，{len(vocab)}
   个词）：")
32 for word in sorted(inverted_index.keys()):
33     print(f"  {word}: {inverted_index[word]}")
34
35 # 查询倒排列表
36 for word in ["自然语言处理", "深度学习", "文档"]:
37     print(f"{word}: {inverted_index.get(word, [])}")
38
```

(3) 基于规则实现 布尔检索模型

原理

布尔检索模型（Boolean Retrieval Model）是最早的信息检索模型，也是规则范式的典型代表。

它的核心思想是：用户的查询由关键词和布尔运算符（AND、OR、NOT）组成，系统通过集合运算找到匹配的文档。

布尔运算符有三种基本操作：

- **AND（与）**：文档必须同时包含所有关键词 → 对应集合的交集
- **OR（或）**：文档包含任意一个关键词即可 → 对应集合的并集
- **NOT（非）**：文档不能包含关键词 → 对应集合的补集

例子：

```
1 查询：自然语言处理 AND 文本
2 含义：找同时包含"自然语言处理"和"文本"的文档
3
4 自然语言处理 → {doc1, doc4}
5 文本 → {doc4, doc5}
6 交集 → {doc4}
7 结果：doc4
```

代码

```
1 from sklearn.feature_extraction.text import
  CountVectorizer
2
3 # 文档
```

```
documents = {
    "doc1": "自然语言处理 是 人工智能 的 重要 分支",
    "doc2": "机器学习 包括 监督学习 和 无监督学习",
    "doc3": "深度学习 基于 神经网络 模型",
    "doc4": "文本分类 是 自然语言处理 的 基础 任务",
    "doc5": "情感分析 是 文本分类 的 一种",
    "doc6": "信息检索 研究 如何 查找 相关 文档",
}

doc_ids = list(documents.keys())
all_docs = set(doc_ids)

# 建倒排索引
vectorizer = CountVectorizer(token_pattern=r"(?u)\b\w+\b")
X = vectorizer.fit_transform(documents.values())
X_T = X.T.tocsr()
vocab = vectorizer.get_feature_names_out()

# 词 -> 文档集合
index = {word: set(doc_ids[idx] for idx in X_T[i].nonzero()[1]) for i, word in enumerate(vocab)}


# 查询函数
def search(query):
    # NOT A
    if query.startswith("NOT "):
        return sorted(all_docs - index.get(query[4:], set()))

    # A AND B
    if " AND " in query:
        a, b = query.split(" AND ")
        return sorted(index.get(a, set()) & index.get(b, set()))

    # A OR B
    if " OR " in query:
        a, b = query.split(" OR ")
        return sorted(index.get(a, set()) | index.get(b, set()))

```

```
    # 单词
    return sorted(index.get(query, set()))


# 测试
test_queries = [
    "自然语言处理",
    "深度学习",
    "自然语言处理 AND 文本分类",
    "学习 OR 深度学习",
    "NOT 是",
    "文本分类 AND 情感分析",
]

for query in test_queries:
    results = search(query)
    print(f"查询: {query}")
    print(f"结果: {results}")
    if results:
        for doc_id in results:
            print(f"  {doc_id}: {documents[doc_id]}")
    print()
```

优缺点

优点:

- 简单直观：查询语义清晰，容易理解
- 精确可控：用户可以精确控制查询条件
- 效率高：集合运算速度快，适合大规模数据
- 结果确定：同样的查询总是返回同样的结果

缺点:

- 二元判断：文档要么相关要么不相关，没有相关度的概念，无法排序
- 查询难写：普通用户不会写布尔表达式，需要学习查询语法
- 容易空结果：AND 条件太多时，可能没有匹配的文档
- 无法处理模糊匹配：必须精确匹配关键词，同义词、近义词无法处理
- 没有排序：返回的结果没有按相关度排序，用户不知道先看哪篇

(4) 基于统计范式实现 向量空间模型

原理

向量空间模型（Vector Space Model, VSM）是统计范式下最经典的信息检索模型。它的核心思想是：把查询和文档都表示成向量，然后用向量的相似度来衡量查询和文档的相关度。

具体来说：

1. 用 TF-IDF 将查询和每篇文档都表示为向量
2. 计算查询向量与每个文档向量的余弦相似度
3. 按相似度从高到低排序，返回最相关的文档

统计范式更好

相比布尔检索，向量空间模型有几个关键优势：

- 相关度排序：文档按相关度排序，最相关的排在前面
- 部分匹配：不需要所有查询词都出现，出现部分词也能有一定相关度
- 词重要性：TF-IDF 自动区分了常见词和稀有词的权重
- 结果更自然：符合用户"找最相关的"的直觉需求

代码

```
"""
向量空间模型检索
"""
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# 文档
documents = {
    "doc1": "自然语言处理 是 人工智能 的 重要 分支",
    "doc2": "机器学习 包括 监督学习 和 无监督学习",
    "doc3": "深度学习 基于 神经网络 模型",
    "doc4": "文本分类 是 自然语言处理 的 基础 任务",
    "doc5": "情感分析 是 文本分类 的 一种",
    "doc6": "信息检索 研究 如何 查找 相关 文档",
}

doc_ids = list(documents.keys())
```

```
doc_texts = list(documents.values())

# 建 TF-IDF 索引
vectorizer = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
doc_vectors = vectorizer.fit_transform(doc_texts)


# 检索函数
def search(query, top_k=3):
    query_vec = vectorizer.transform([query])  # 查询转向量
    scores = cosine_similarity(query_vec, doc_vectors)[0]  # 算所有文档的相似度
    ranked = sorted(zip(doc_ids, scores), key=lambda x: x[1], reverse=True)
    return [(doc_id, score) for doc_id, score in ranked if score > 0][:top_k]


# 测试
test_queries = [
    "自然语言处理 文本分类",
    "深度学习 神经网络",
    "学习 机器学习",
    "文档 查找",
]

for query in test_queries:
    results = search(query, top_k=3)
    print(f"\n查询: {query}")
    print("最相关的文档:")
    for rank, (doc_id, score) in enumerate(results, 1):
        print(f"  {rank}. 相关度: {score:.4f} | {doc_id}: {documents[doc_id]}")

```

(5) 两种检索模型的对比

对比维度	布尔检索模型（规则范式）	向量空间模型（统计范式）
核心思想	精确匹配 + 集合运算	向量相似度 + 排序
结果形式	文档集合（无排序）	按相关度排序的文档列表

对比维度	布尔检索模型（规则范式）	向量空间模型（统计范式）
匹配方式	二元：匹配/不匹配	连续：相关度 0~1
词的权重	所有词同等重要	TF-IDF 区分重要性
查询语法	需要布尔表达式	自然语言关键词即可
部分匹配	不支持	支持
用户体验	需要学习语法	直观易用
适用场景	精确查找、专业数据库	通用搜索引擎、文本检索

3.4 统计范式落幕

统计范式在 NLP 领域统治了二十多年，取得了巨大的成功。但随着研究的深入，它的局限性也越来越明显。

3.4.1 语义鸿沟

统计范式最根本的局限是无法捕捉语义相似性。

在稀疏表示（独热编码、词袋模型、TF-IDF）中，每个词都是一个独立的维度，词和词之间没有任何关联。"快乐"和"高兴"意思非常接近，但它们的向量是正交的，相似度为 0。

这导致了一系列问题：

- 同义词问题：用户搜"开心"，但文档里用的是"快乐"，无法检索
- 泛化能力差：训练数据中没见过的表达方式，模型无法处理
- 数据稀疏：词表越大，数据越稀疏，很多特征只有很少的样本

3.4.2 特征工程瓶颈

统计模型的效果很大程度上取决于特征工程，你设计什么样的特征，模型就能学到什么样的规律。

但特征工程是一件非常耗时耗力的事情：

- 需要领域专家人工设计特征
- 不同任务需要不同的特征
- 特征组合爆炸，很难穷举
- 特征的上限就是模型的上限

这就是所谓的"特征工程瓶颈"，统计模型本身很简单，但要让它工作好，需要大量人工设计特征。

3.4.3 无法捕捉长距离依赖

N-gram 只能捕捉局部的词序信息（N 个词以内），对于长距离的依赖关系完全无能为力。

3.4.4 维度灾难

随着词汇表的增大，向量维度急剧增长。

10 万词汇表就是 10 万维的向量，存储和计算的成本都很高，而且大部分维度都是 0（稀疏）。

3.5 新技术的发展

就在统计范式遇到瓶颈的时候，深度学习技术正在悄然崛起。

深度学习的核心优势是表示学习（Representation Learning）：不需要人工设计特征，让神经网络自动从数据中学习特征表示。

这直接解决了统计范式的几个核心痛点：

- 稠密向量：词被表示为低维的稠密向量（如 100 维、300 维），语义相似的词在向量空间中也接近
- 自动特征学习：不需要人工设计特征，神经网络自动学习层次化的特征表示
- 序列建模能力：RNN、LSTM 等模型能处理序列数据，捕捉长距离依赖
- 端到端学习：从输入到输出端到端训练，不需要中间的特征工程步骤

3.6 统计学习的遗珠

虽然深度学习范式已经成为主流，但统计学习并没有完全退出历史舞台。在很多场景下，统计方法仍然有它的价值：

小数据场景：当训练数据很少时，简单的统计模型（如朴素贝叶斯）往往比复杂的深度学习模型效果更好

可解释性要求高的场景：统计模型的可解释性很强，而深度学习模型像一个黑盒，很难解释它为什么做出某个判断。

快速原型验证：统计模型训练快、实现简单，非常适合做 baseline（基线），快速验证想法是否可行。

低资源场景：在计算资源有限的设备上（如嵌入式设备），轻量级的统计模型仍然有它的用武之地。

TF-IDF + 线性模型：即使在今天，在很多文本分类任务上，TF-IDF + 逻辑回归/SVM 的组合仍然是一个非常强的 baseline，很多复杂的深度学习模型都不一定能轻易超过它。

4. 基于深度学习的任务范式

4.1 概述

4.1.1 新旧范式交替

2010 年代初期，深度学习开始在 NLP 领域崭露头角。标志性的事件包括：

- **2013 年**：Word2Vec 问世，将词嵌入（Word Embedding）技术推向主流，让"稠密词向量"成为 NLP 的标配
- **2014-2015 年**：RNN / LSTM 在序列建模任务上取得突破，机器翻译、语言模型等任务效果大幅提升
- **2014 年**：Seq2Seq 模型提出，端到端的序列到序列学习成为新范式
- **2015 年**：Attention 机制引入，解决了长序列的信息瓶颈问题
- **2017 年**：Transformer 架构提出，自注意力机制成为新的核心
- **2018 年**：BERT 问世，预训练 + 微调成为新范式，NLP 进入"预训练语言模型"时代
- **2020 年至今**：GPT-3、ChatGPT 等大语言模型涌现，NLP 进入"大模型时代"

4.1.2 深层原因解析

第一，数据量的爆发式增长。

互联网的普及带来了海量的文本数据。

统计模型在数据量达到一定程度后，效果就会趋于饱和，再增加数据也不会有明显提升。而深度学习模型能从更多数据中学到更复杂的模式。数据越多，深度学习的优势越明显。

第二，计算能力的飞跃。

GPU 的普及让大规模神经网络的训练成为可能。深度学习模型的训练需要大量的矩阵运算，而 GPU 正好擅长。

第三，表示学习的突破。

统计范式的核心瓶颈是"特征工程"，需要人工设计特征。深度学习的核心优势是"表示学习"，自动从数据中学习特征表示。这把人从繁琐的特征工程中解放出来，也让模型的上限大大提高。

第四，端到端学习的优势。

统计范式的流水线通常是：分词 → 特征提取 → 分类器 → 结果。每一步都可能引入误差，而且误差会累积。

深度学习的端到端学习直接从输入到输出，中间不需要人工设计的步骤——整个系统一起优化，效果自然更好。

4.2 稠密词向量

在统计范式中，词的表示是稀疏的，独热编码、词袋模型、TF-IDF，都是高维稀疏向量。这些方法有一个根本缺陷：无法捕捉词与词之间的语义关系。

答案是稠密词向量（Dense Word Vector），也叫词嵌入（Word Embedding）。

稠密词向量的核心思想是：把每个词表示为一个低维的、稠密的实数向量（比如 100 维、300 维）。语义相似的词，在向量空间中的距离也相近。

举个直观的例子：

```
国王: [0.8, 0.2, 0.9, 0.1, ...]
王后: [0.7, 0.8, 0.9, 0.2, ...]
男人: [0.9, 0.1, 0.3, 0.2, ...]
女人: [0.1, 0.9, 0.3, 0.1, ...]
```

这些向量的每一维都代表某种语义特征（虽然我们不一定能解释每一维具体是什么）。语义相似的词，向量在空间中的位置也接近。

更神奇的是，稠密词向量还能进行向量运算。最经典的例子：

$$\text{国王} - \text{男人} + \text{女人} \approx \text{王后}$$

这说明词向量中蕴含了丰富的语义关系，性别关系、君臣关系等，都被编码在了向量空间中。

4.3 词嵌入(文本表示)

4.3.1 定义

词嵌入（Word Embedding），也叫词向量，是把词映射成低维稠密向量的技术。

```
独热向量（又大又稀）：
  减肥：[1, 0, 0, 0, 0, 0, ...]  ← 10000维，只有1个1，其他都是0
  瘦身：[0, 1, 0, 0, 0, 0, ...]
  增肌：[0, 0, 1, 0, 0, 0, ...]

词嵌入（又小又密）：
  减肥：[0.8, 0.6, 0.1, 0.3]  ← 4维，每个位置都有值
  瘦身：[0.75, 0.65, 0.05, 0.28]  ← 跟减肥很像，因为意思接近
  增肌：[0.2, 0.1, 0.9, 0.7]  ← 跟减肥差很远，因为意思不一样
```

例子

	Man	Women	King	Queen	Apple	Orange
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97

我们用一个四维向量来表示man，Women，King，Queen，Apple，Orange等词语（在实际中使用更高维度的表示，例如100~300维），这些向量能进行语义的表示和计算。

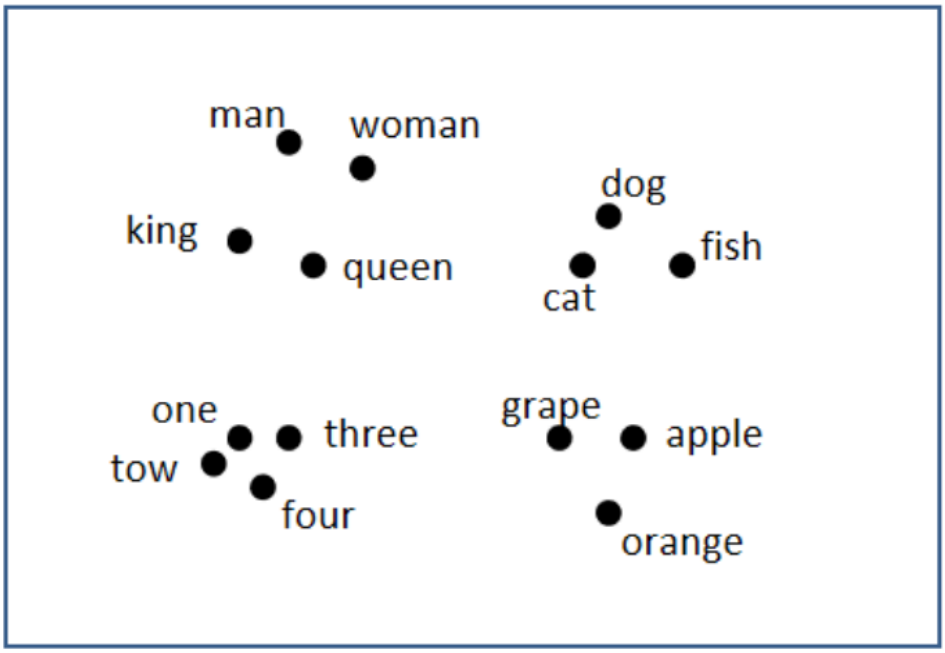
例如，用Man的向量减去Woman的向量值：

$$e_{man} - e_{woman} = \begin{bmatrix} -1 \\ 0.01 \\ 0.03 \\ 0.09 \end{bmatrix} - \begin{bmatrix} 1 \\ 0.02 \\ 0.02 \\ 0.01 \end{bmatrix} = \begin{bmatrix} -2 \\ -0.01 \\ 0.01 \\ 0.08 \end{bmatrix} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

类似地，如果用King的向量减去Queen的向量，得到相似的结果

$$e_{king} - e_{queen} = \begin{bmatrix} -0.95 \\ 0.93 \\ 0.70 \\ 0.02 \end{bmatrix} - \begin{bmatrix} 0.97 \\ 0.95 \\ 0.69 \\ 0.01 \end{bmatrix} = \begin{bmatrix} -1.92 \\ -0.02 \\ 0.01 \\ 0.01 \end{bmatrix} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

我们可以通过某种降维算法，将向量映射到低纬度空间中，相似的词语位置较近，不相似的词语位置较远，这样能帮助我们更直观理解词嵌入对语义的表示：



实际任务中，词汇量较大，表示维度较高，因此，我们不能手动为大型文本语料库开发词向量，而需要设计一种方法来使用一些算法或者神经网络或者语言模型自动找到好的词嵌入，以便有效地执行这项繁重的任务。

4.3.2 三大性质

语义相似性

意思相近的词，向量也相近（余弦相似度高）

```
减肥 ↔ 瘦身   →  相似度 0.92  ← 很高
猫   ↔ 狗     →  相似度 0.78  ← 中等（都是动物）
减肥 ↔ 电脑   →  相似度 0.12  ← 很低（完全不相关）
```

类比推理

词向量可以做加减法，捕捉词语之间的关系。

```
国王 - 男人 + 女人 ≈ 女王
北京 - 中国 + 日本 ≈ 东京
大 - 更大 + 小     ≈ 更小
```

线性可加性

句子的意思 ≈ 句子里所有词的向量加起来（或取平均）

```
"我爱减肥"的向量 ≈ 我 + 爱 + 减肥
```

4.3.3 相关概念

概念	说明
维度	词向量的长度，通常 50/100/200/300 维。维度越高，表达能力越强，越容易过拟合
窗口大小	统计共现时，左右各看几个词。窗口小→语法信息，窗口大→语义信息
词表大小	有多少个词有向量。词表越大，覆盖越全，但内存占用越大
未登录词（OOV）	词表里没有的词。FastText 用子词解决，BERT 用 WordPiece 解决
预训练	先在大语料上训练好词向量，再用到下游任务上
微调	用下游任务的数据，继续调整词向量

4.5 Word2Vec

Word2vec是Goolge发布的、应用最广泛的词嵌入表示学习技术，其主要作用是高效获取词语的词向量，目前被用作许多NLP任务的特征工程。

Word2Vec 的理论基础是分布式假设（Distributional Hypothesis）：出现在相似上下文中的词，它们的语义也相似。

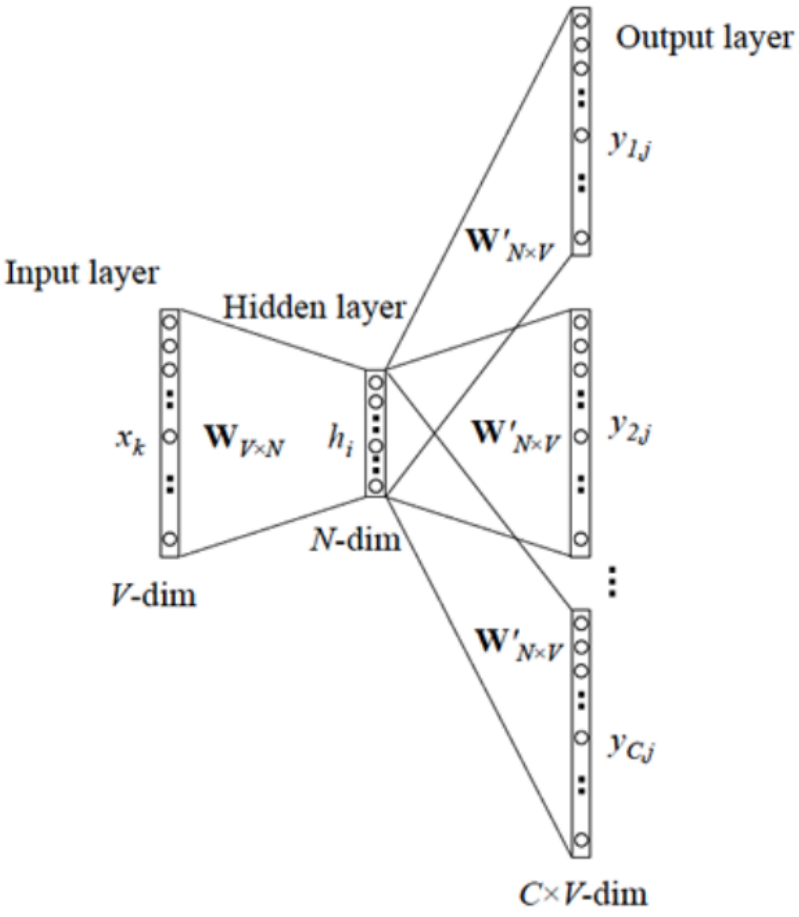
Word2vec 可以根据给定的语料库，通过优化后的训练模型快速有效地将一个词语表达成向量形式，为自然语言处理领域的应用研究提供了新的工具，包含Skip-gram（跳字模型）和CBOW（连续词袋模型）来建立词语的词嵌入表示。

Skip-gram的主要作用是根据当前词，预测背景词（前后的词）；CBOW的主要作用 是根据背景词（前后的词）预测当前词。

4.5.1 Skip-gram

定义

Skip-gram的主要作用是通过中心词预测其周围上下文词，从而学习词的低维稠密向量表示（词嵌入）。



输入的长度为v的向量, 和词表大小一样, 经过神经网络, 输出的n个长度为v的概率,前几个词后几个词的概率

核心思想

- 输入：中心词（1 个）
- 输出：上下文的 C 个词
- 每个输出都是一个 V 维的概率分布，表示词汇表里每个词是上下文词的概率

工作流程

```
句子：「我 爱 深度学习」

如果中心词是「爱」，窗口大小 = 1：
```

- 输入：「爱」（one-hot 向量）
- 输出：
  - 左边上下文：「我」
  - 右边上下文：「深度学习」
- 也就是 C=2，输出两个词的概率分布

训练完之后，我们就得到了每个词的向量表示（就是权重矩阵 W）。

示例

```
呼_ _尔大草原
_伦_ _大草原
_ _贝_ _草原
呼_ _尔_ _原
呼伦_ _大_ _
呼伦贝_ _草_
呼伦贝尔_ _原

```

预测出前后词的数量，称为window_size（以上示例中windows_size为2），实际是要将以下概率最大化：条件概率表示:

```
P(伦|呼)P(贝|呼)
P(呼|伦)P(贝|伦)P(尔|伦)
P(呼|贝)P(伦|贝)P(尔|贝)P(大|贝)
P(伦|尔)P(贝|尔)P(大|尔)P(草|尔)
P(贝|大)P(尔|大)P(草|大)P(原|大)
P(尔|草)P(大|草)(原|草)
P(大|原)P(草|原)
```

可以写出概率的一般化表达式，设有文本Text，由N个单词组成：

$$Text = w_1, w_2, w_3, \ldots, w_n$$

目标函数可以写作(似然公式)：

$$argmax \prod_{w \in Text} \prod_{c \in c(w)} P(c|w;\theta)$$

其中， w是当前词(中心词), c 为 w 的上下文词， 为要优化的参数，这个参数即每个词（或字）的稠密向量表示：

$$\begin{bmatrix} 呼：\theta_{11} & \theta_{12} & \theta_{13} \dots & \theta_{1n} \\ 伦：\theta_{21} & \theta_{22} & \theta_{23} \dots & \theta_{2n} \\ 贝：\theta_{31} & \theta_{32} & \theta_{33} \dots & \theta_{3n} \\ 尔：\theta_{41} & \theta_{42} & \theta_{43} \dots & \theta_{4n} \\ 大：\theta_{51} & \theta_{52} & \theta_{53} \dots & \theta_{5n} \\ 草：\theta_{61} & \theta_{62} & \theta_{63} \dots & \theta_{6n} \\ 原：\theta_{71} & \theta_{72} & \theta_{73} \dots & \theta_{7n} \end{bmatrix}$$

该参数能够使得目标函数最大化。因为概率均为0~1之间的数字，连乘计算较为困难，会越来越小, 所以转换为对数相加形式(对数似然公式)：

$$argmax \sum_{w \in Text} \sum_{c \in c(w)} logP(c|w;\theta)$$

再表示为softmax形式(需要计算 $P(c \mid w)$)：

$$argmax \sum_{w \in Text} \sum_{c \in c(w)} log\left(e^{u_c \cdot v_w} / \sum_{c' \in vocab} e^{u_{c'} \cdot v_w}\right)$$

其中，U 为上下文单词矩阵，V 为同样大小的中心词矩阵，因为每个词可以作为上下文词，同时也可以作为中心词， 表示上下文词和中心词向量的内积（内积表示向量的相似度），相似度越大，概率越高；

分母部分是以为中心词，其它所有上下文词内积之和，再将上一步公式进行简化：

$$\begin{aligned} &= argmax \sum_{w \in Text} \sum_{c \in c(w)} \left(log(e^{u_c \cdot v_w}) - log(\sum_{c' \in vocab} e^{u_{c'} \cdot v_w})\right) \\ &= argmax \sum_{w \in Text} \sum_{c \in c(w)} \left(u_c \cdot v_w - log \sum_{c' \in vocab} e^{u_{c'} \cdot v_w}\right) \end{aligned}$$

上式中，由于需要在整个词汇表中进行遍历，如果词汇表很大，计算效率会很低。

所以，真正进行优化时，采用另一 种优化形式(负采样格式)。

例如有如下语料库：

```
1 呼伦贝尔大草原
```

将window_size设置为1，构建正案例词典、负案例词典（一般来说，负样本词典比正样本词典大的多）：

```
正样本：
D = {(呼，伦)，(伦，呼)，(伦，贝)，(贝，伦),(贝，尔),(尔，贝)，
(尔，大)，(大，尔)，(大，草) (草，大)，(草，原)，(原，草)}

负样本：
D'= {(呼，贝),(呼，尔),(呼，大)，(呼，草)，(呼，原)，(伦，尔)，
(伦，大),(伦，草),(伦，原)，(贝，呼),(贝，大),(贝，草),(贝，原)，
(尔，呼),(尔，伦)(尔，草),(尔，原),(大，呼),(大，伦),(大，原)，
(草，呼)，(草，伦)，(草，贝)，(原，呼)，(原，伦)，(原，贝)，(原，
尔)，(原，大)}
```

词向量优化的目标函数定义为正样本、负样本公共概率最大化函数：

$$argmax(\prod_{w,c\in D} logP(D=1|w,c;\theta)\prod_{w,c\in D'} P(D=0|w,c;\theta))$$

$$=argmax(\prod_{w,c\in D}\frac{1}{1+exp(-U_c\cdot V_w)}\prod_{w,c\in D'}[1-\frac{1}{1+exp(-U_c\cdot V_w)}])$$

$$=argmax(\sum_{w,c\in D} log\sigma(U_c\cdot V_w)+\sum_{w,c\in D'} log\sigma(-U_c\cdot V_w))$$

在实际训练时，负样本不需要全用，每次随机抽几个就行，这就是 "采样" 的含义。从而降低运算量，要训练词向量，还需要借助于语言模型

代码

```python
import numpy as np

corpus = [
    ["我", "爱", "自然语言处理"],
    ["自然语言处理", "很", "有趣"],
    ["我", "爱", "学习"],
    ["学习", "自然语言处理", "很", "重要"],
    ["机器学习", "很", "有趣"],
    ["深度学习", "很", "有用"],
    ["我", "喜欢", "机器学习"],
    ["深度学习", "是", "机器学习", "的", "分支"],
]

vocab = sorted(set(w for s in corpus for w in s))
word2idx = {w: i for i, w in enumerate(vocab)}
V, N, lr = len(vocab), 10, 0.025

# 训练样本
data = []
```

```python
for s in corpus:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                data.append((word2idx[w], word2idx[s[j]]))

# 初始化
W1 = np.random.uniform(-1 / V, 1 / V, (V, N))
W2 = np.random.uniform(-1 / N, 1 / N, (N, V))

# 训练
for epoch in range(5000):
    loss = 0
    for c, ctx in data:
        h = W1[c]
        u = h @ W2
        y = np.exp(u - u.max())
        y /= y.sum()

        loss += -np.log(y[ctx] + 1e-10)

        y[ctx] -= 1
        W2 -= lr * np.outer(h, y)
        W1[c] -= lr * (W2 @ y)

    if (epoch + 1) % 1000 == 0:
        print(f"Epoch {epoch + 1}, Loss: {loss / len(data):.4f}")

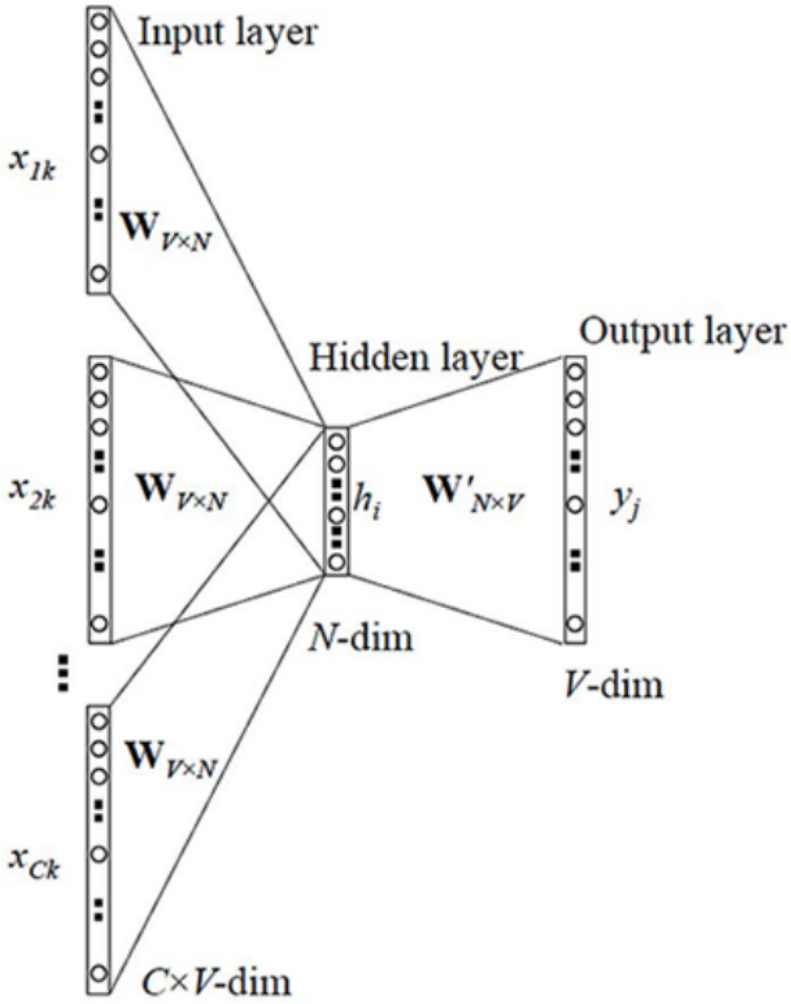

# 测试
def most_similar(w, k=3):
    v = W1[word2idx[w]]
    sims = []
    for x in vocab:
        if x == w: continue
        xv = W1[word2idx[x]]
        sims.append((x, np.dot(v, xv) / (np.linalg.norm(v) * np.linalg.norm(xv) + 1e-10)))
    return sorted(sims, key=lambda x: x[1], reverse=True)[:k]
```

```
58
59
60 print()
61 for w in ["学习", "自然语言处理", "机器学习"]:
62     print(f'"{w}": {most_similar(w)}')
63
```

4.5.2 CBOW

定义

CBOW模型全称为Continous Bag of Words（连续词袋模型），其核心思想是用上下文来预测中心词



核心思想

> 输入上下文的一堆词，预测中间那个词是什么。

- 输入：上下文词（C 个）
- 输出：中心词（1 个）

- 输出是一个 V 维的概率分布，表示词汇表里每个词是中心词的概率

工作流程

```
句子：「呼 伦 贝 尔 大 草 原」

如果中心词是「贝」，窗口大小 = 2：
```

- 输入：
  - 左边上下文：「呼」、「伦」
  - 右边上下文：「尔」、「大」
  - 也就是 C=4，输入 4 个词的 one-hot 向量
- 输出：「贝」（中心词的概率分布）

训练完之后，我们就得到了每个词的向量表示（就是权重矩阵 W）。

4.5.3 训练词向量

```python
import pandas as pd
import jieba
from gensim.models import Word2Vec

# 1.加载数据
train_df = pd.read_csv("data/dianping_train.csv")
dev_df = pd.read_csv("data/dianping_dev.csv")
test_df = pd.read_csv("data/dianping_test.csv")

df = pd.concat([train_df, dev_df, test_df], ignore_index=True)

# 2.中文分词
def cut_words(text):
    return [w for w in jieba.lcut(str(text)) if len(w) >= 2]
sentences = df["review"].apply(cut_words).tolist()

# 3.训练 Word2Vec
model = Word2Vec(
    sentences=sentences,
    vector_size=200,  # 词向量维度
    window=5,  # 窗口大小
    min_count=5,  # 忽略词频低于5的词
```

```
    sg=1,  # 1=Skip-Gram, 0=CBOW
    workers=4,  # 并行线程
    epochs=20  # 训练轮数
)

# 保存模型
model.save("model/dianping_word2vec.model")

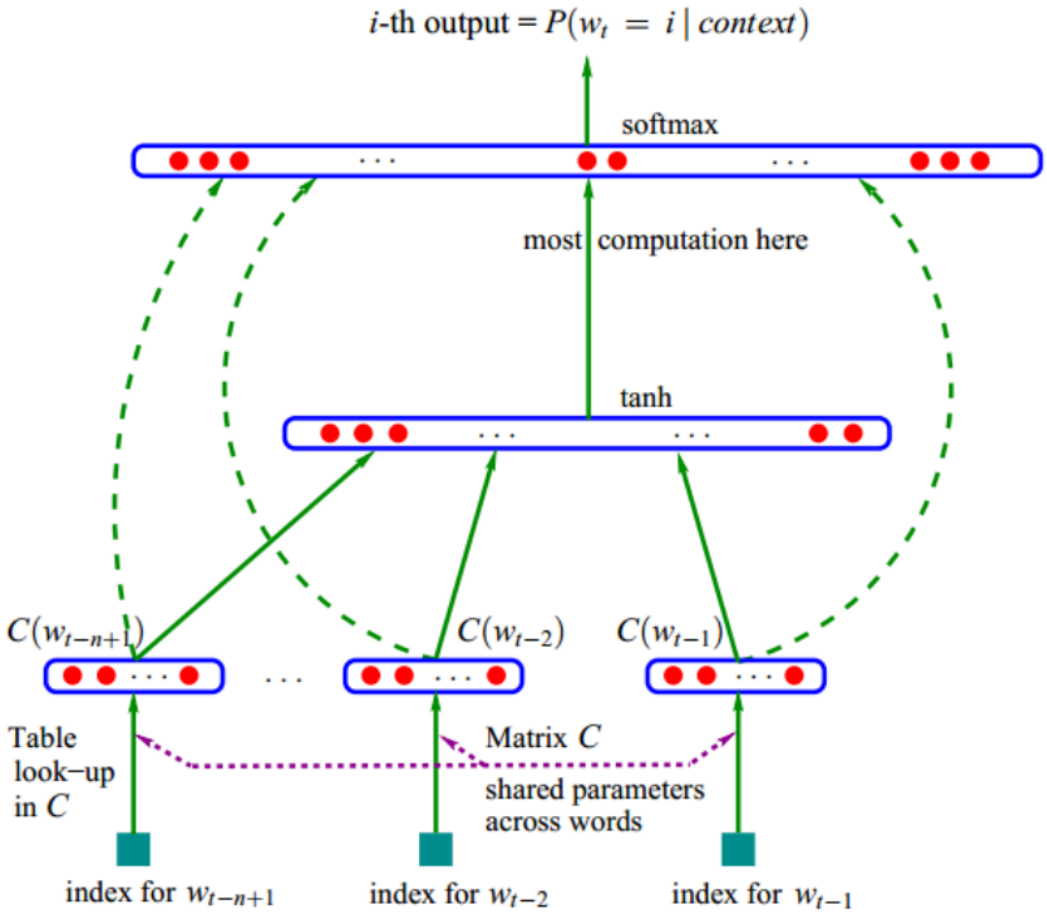
# 4.测试词向量
test_words = ["好吃", "服务", "环境", "价格", "推荐", "难吃", "排队", "味道"]
for word in test_words:
    if word in model.wv:
        similar = model.wv.most_similar(word, topn=5)
        print(f'"{word}" 最相似的词:')
        for w, sim in similar:
            print(f"  {w}: {sim:.4f}")
        print()

analogies = [
    (["好吃", "服务"], ["难吃"]),  # 好吃:难吃 = 服务:?
    (["便宜", "环境"], ["贵"]),  # 便宜:贵 = 环境:?
    (["美味", "排队"], ["难吃"]),  # 美味:难吃 = 排队:?
]
for pos, neg in analogies:
    if all(w in model.wv for w in pos + neg):
        result = model.wv.most_similar(positive=pos, negative=neg, topn=3)
        print(f"{neg[0]} : {pos[0]} = ? : {pos[1]}")
        for w, sim in result:
            print(f"  {w}: {sim:.4f}")
        print()

```

4.6 神经网络语言模型(NNLM)

NNLM是利用神经网络对N元条件进行概率估计的一种方法，其基本结构如下图所示：



输入：前N-1个词语的独热向量(虽然没有语义信息, 但是经过训练,副产品叫词嵌入[算出来的权重矩阵叫词嵌入])

输出：第N个词语的一组概率

目标函数：

$$f(w_t, w_{t-1}, \ldots, w_{t-n+1}) = p(p_t|w_1^{t-1})$$

该模型计算包括两部分：特征映射、计算条件概率

神经网络向前传播计算公式为：

$$h = tanh(Hx + b)$$
$$y = Uh + d$$

H 为隐藏层权重矩阵，U 为隐藏层到输出层的权重矩阵。

输出层加入softmax函数，将y转换为对应的概率。

θ 表示模型的所有参数，它是一个集合 ， 包括：

$$\theta = (b, d, H, U, C)$$

以下是一个计算示例：设词典大小为1000，向量维度为25，N=3，先将前N个词表示成独热向量：

```
呼：[1,0,0,0,0,...,0]
伦：[0,1,0,0,0,...,0]
贝：[0,0,1,0,0,...,0]
```

输入矩阵为：[3, 1000]

权重矩阵：[1000, 25]

隐藏层：[3, 1000] * [1000, 25] = [3, 25]

输出层权重：[25, 1000]

输出矩阵：[3, 25] * [25, 1000] = [3, 1000] ==> [1, 1000]，表示预测属于1000个词的概率

4.7 原生RNN

循环神经网络（Recurrent Neural Network，RNN）能够在原有神经网络的基础上增加记忆单元，处理任意长度的序列 （理论上），并且在前后词语（或字）之间建立起依赖关系。

相比于CNN，RNN更适合处理视频、语音、文本等与时序相关的问题。

4.7.1 定义

原生 RNN（Vanilla RNN，也叫朴素 RNN、简单循环网络） 是最基础、最原始的循环神经网络。

它的结构极其简单：只有一个隐藏层，隐藏层的状态会传递到下一个时间步，形成循环连接

4.7.2 发展史

(1) 思想源头 Hopfield 网络（1982）

1982 年，美国物理学家 John Hopfield 提出了 Hopfield 网络。

这是一种全连接的循环网络，每个神经元的输出都会反馈给所有其他神经元。

网络会不断迭代，最终收敛到一个稳定的状态， 就像 "记忆" 的唤起过程。

Hopfield 网络不是用来处理序列的，但它第一次把循环连接和记忆联系在了一起，为后来的 RNN 埋下了种子。

(2) 早期探索 Jordan 网络（1986）

1986 年，Michael Jordan 提出了 Jordan 网络。

- 输入层 → 隐藏层 → 输出层
- 输出层的状态会反馈回隐藏层，作为下一步的额外输入

这是第一个专门用于序列处理的循环网络。

(3) 正式诞生 Elman 网络（1990）

1990 年，Jeffrey Elman 提出了 Simple Recurrent Network（SRN，简单循环网络），也叫 Elman 网络。

- 输入层 → 隐藏层 → 输出层
- 隐藏层的状态会复制一份，作为下一个时间步隐藏层的额外输入

4.7.3 核心思想

RNN 的解决思路非常直观，它来自对人类阅读方式的观察：

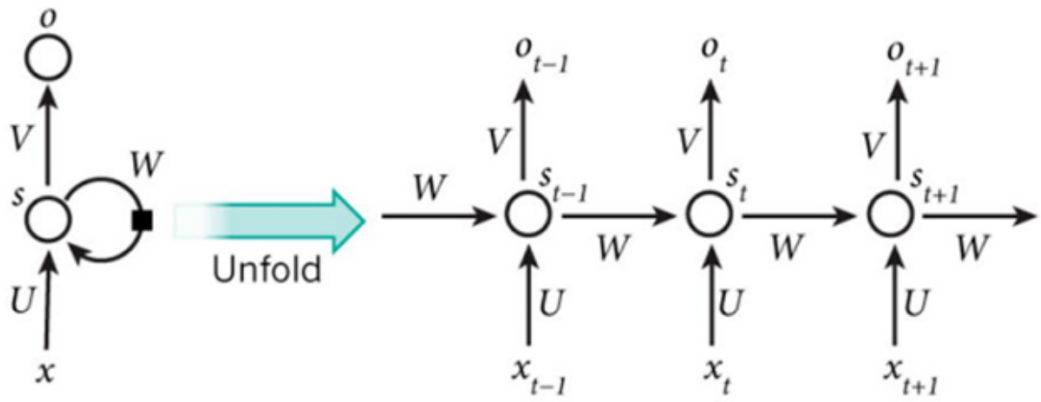
RNN 就是模仿人读句子的过程：

逐个处理：序列中的元素一个一个地输入网络，不是一下子全塞进去

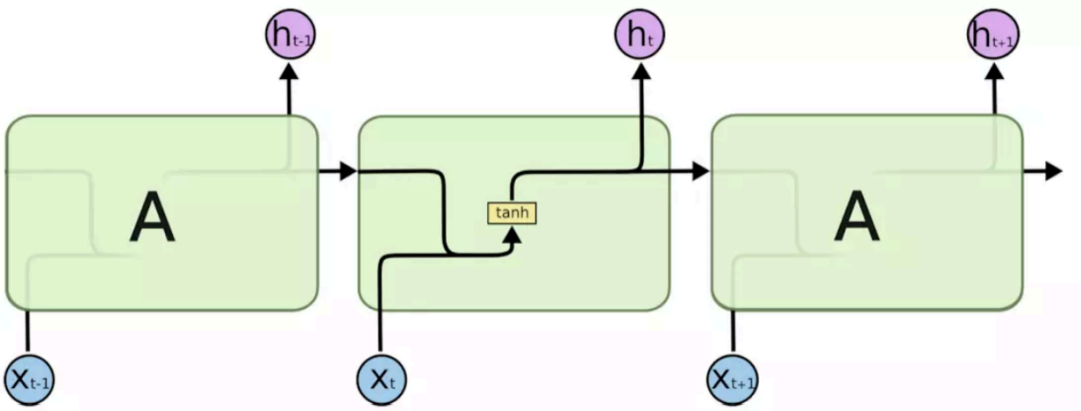
状态传递：每处理一个元素，网络都会更新自己的 "状态"（记忆），并把这个状态传给下一步

状态即记忆：隐藏状态就是网络的 "记忆"—— 里面存着到目前为止看到的所有信息

4.7.4 网络结构



左侧为不展开的画法，右侧为展开画法。内部结构如下图所示：



隐藏状态更新公式:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

输出公式:

$$y_t = W_y h_t + b_y$$

就是一个普通的全连接层，把隐藏状态映射到输出空间。

- W_y：隐藏状态到输出的权重矩阵
- b_y：输出层的偏置
- y_t：输出向量（比如词汇表大小的 logits）

如果是分类任务（比如语言模型），后面还要接一个 softmax，把输出变成概率分布。

计算公式可表示为：

$$s_t = f(Ux_t + Ws_{t-1} + b)$$
$$y_t = g(Vs_t + d)$$

其中， 表示 xt 时刻的输入；st表示 t 时刻隐藏状态；f 和 g 表示激活函数；

U ,V ,W，分别表示输入层 → 隐藏层权重、隐藏层 → 输出层权重、隐藏层 → 隐藏层权重。

对于任意时刻 ，所有权重和偏置都共享，这极大减少了模型参数量。

计算时，首先利用前向传播算法，依次按照时间顺序进行计算，再利用反向传播算法进行误差传递，和普通BP（Back Propagation）网络唯一区别是，加入了时间顺序，计算方式有些微差别，称为BPTT（Back Propagation Through Time）算法。

4.7.5 RNN缺陷

RNN善于处理跟序列相关的信息，如：语音识别，语言建模，翻译，图像字幕。它能根据近期的一些信息来执行/判 别/预测当前任务。例如：

```
1 白色的云朵漂浮在蓝色的_____
2 天空中飞过来一只____
```

根据前面输入的一连串词语，可以预测第一个句子最后一个词为"天空"、第二个句子最后一个词为"鸟"的概率最高。

因为计算的缘故，RNN容易出现梯度消失，导致它无法学习过往久远的信息，无法处理长序列、远期依赖任务。例如：

```
1 我生长在中国，祖上十代都是农民，家里三亩一分地。我是家里老三，我大哥叫大狗子，二哥叫二狗子，我叫三狗子，我弟弟叫狗剩。我的母语是_____
```

要预测出句子最后的词语，需要根据句子开头的信息"我出生在中国"，才能确定母语是"中文"或"汉语"的概率最高。原生RNN在处理这类远期依赖任务时出现了困难，于是LSTM被提出。

4.8 LSTM

4.8.1 定义

长短期记忆模型（Long Short Term Memory，LSTM）是RNN的变种，于1997年Schmidhuber和他的合作者 Hochreiter提出，由于独特的设计结构，LSTM可以很好地解决梯度消失问题，能够处理更长的序列，更好解决远期依赖任务。LSTM非常适合构造大型深度神经网络。

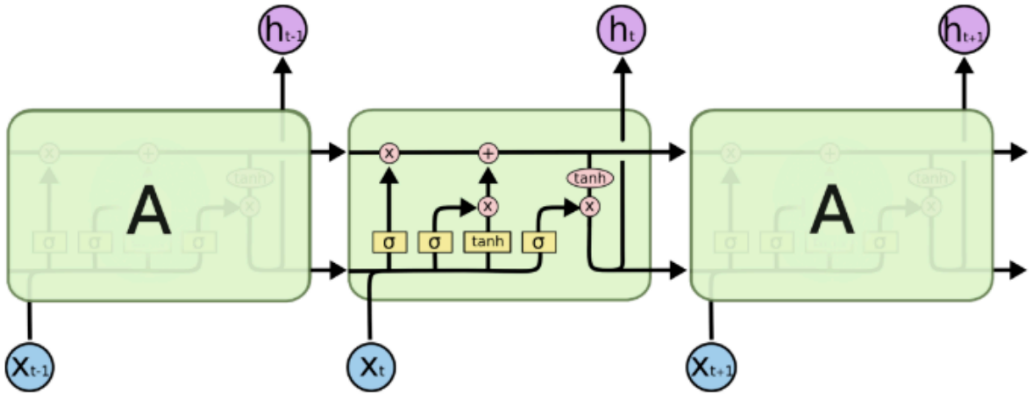
2009年，用改进版的LSTM，赢得了国际文档分析与识别大赛 （ICDAR）手写识别大赛冠军；

2014年，Yoshua Bengio的团队提出了更好的LSTM变体GRU（Gated Recurrent Unit，门控环单元）；

2016年，Google利用LSTM来做语音识别和文字翻译；同年苹果公司使用LSTM 来优化 Siri 应用。

LSTM同样具有链式结构，它具有4个以特殊方式互相影响的神经网络层

4.8.2 网络结构

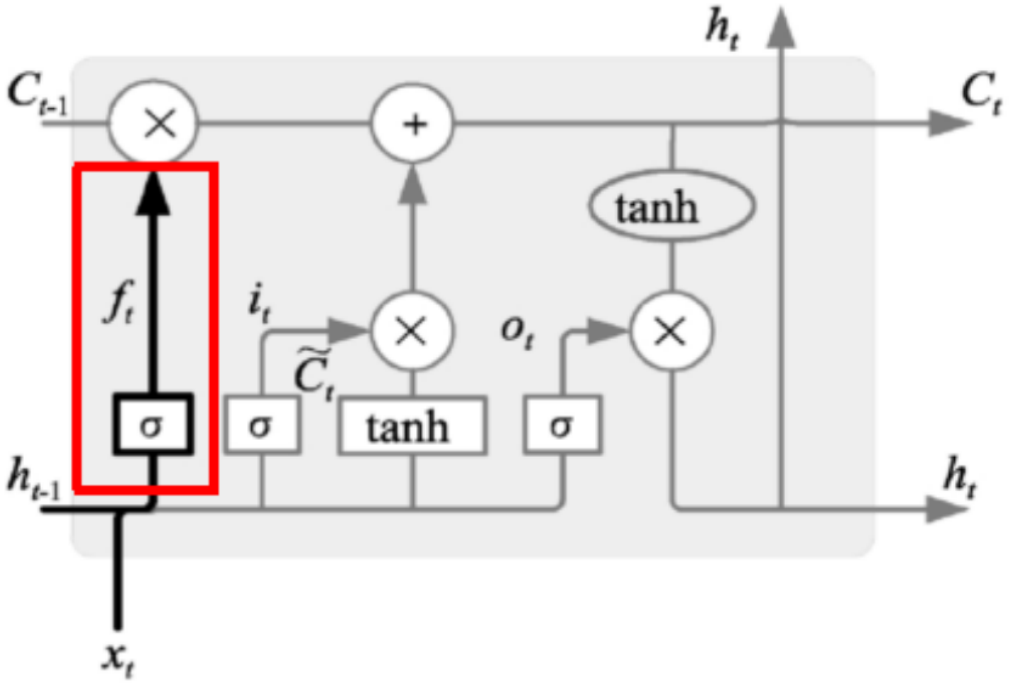


LSTM的核心是细胞状态，用贯穿细胞的水平线表示。

细胞状态像传送带一样。它贯穿整个细胞却只有很少的分支，这样能保证信息不变的流过整个结构。

同时，LSTM通过称为门（gate）的结构来对单元状态进行增加或删除，包含三扇门：遗忘门、输入门、输出门

遗忘门：决定哪些信息丢弃

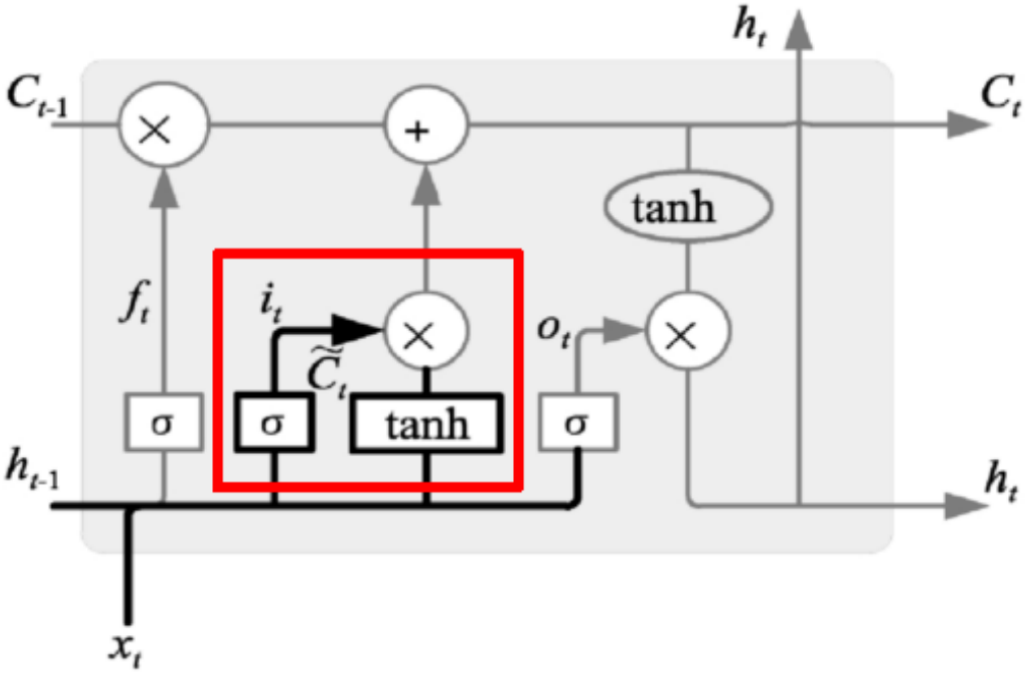


公式

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

当输出为1时表示完全保留，即将 $Ct-1$ 中的值完整传递到 Ct，输出为0是表示完全丢弃，即不会将 $Ct-1$ 的信息传递给 Ct

输入门：决定把哪些新信息存入细胞状态



公式：

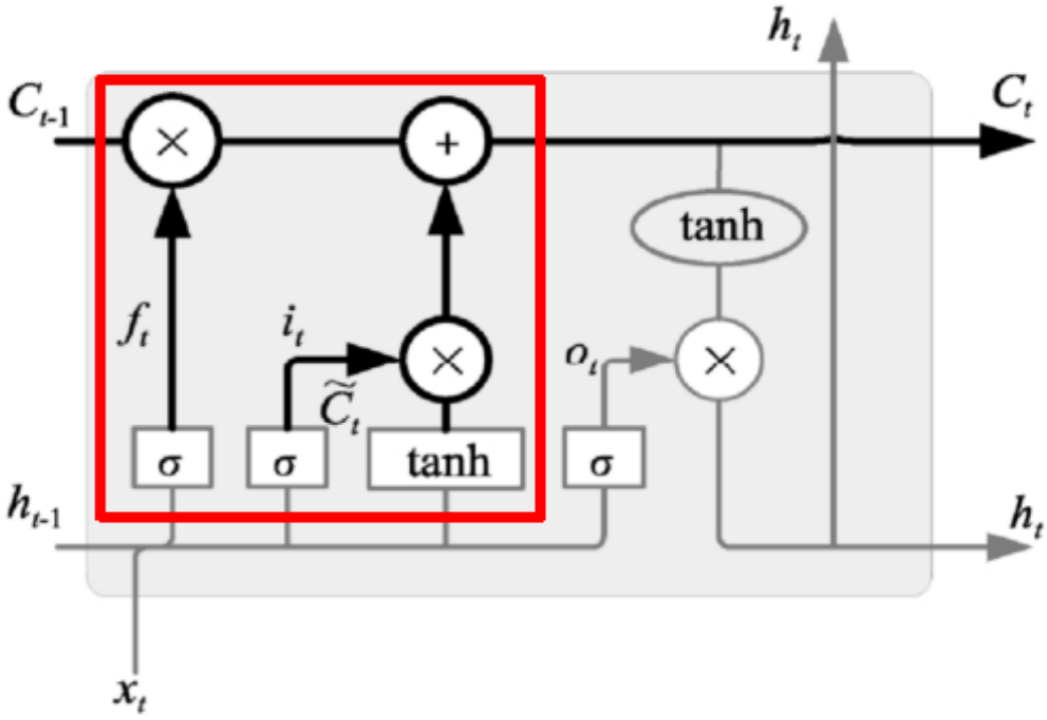
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

上述第一个公式计算输出0的 it 表示来自当前输入的信息将不会进入细胞状态，值为1的 it 意味着来自当前输入的所有信息都会进入细胞状态

再根据第二个公式计算出当前的细胞状态，根据输入门、遗忘门作用结果，可以对细胞状态进行更新

细胞状态更新（Cell State Update）

> 更新长期记忆， 忘掉该忘的，记住该记的



公式

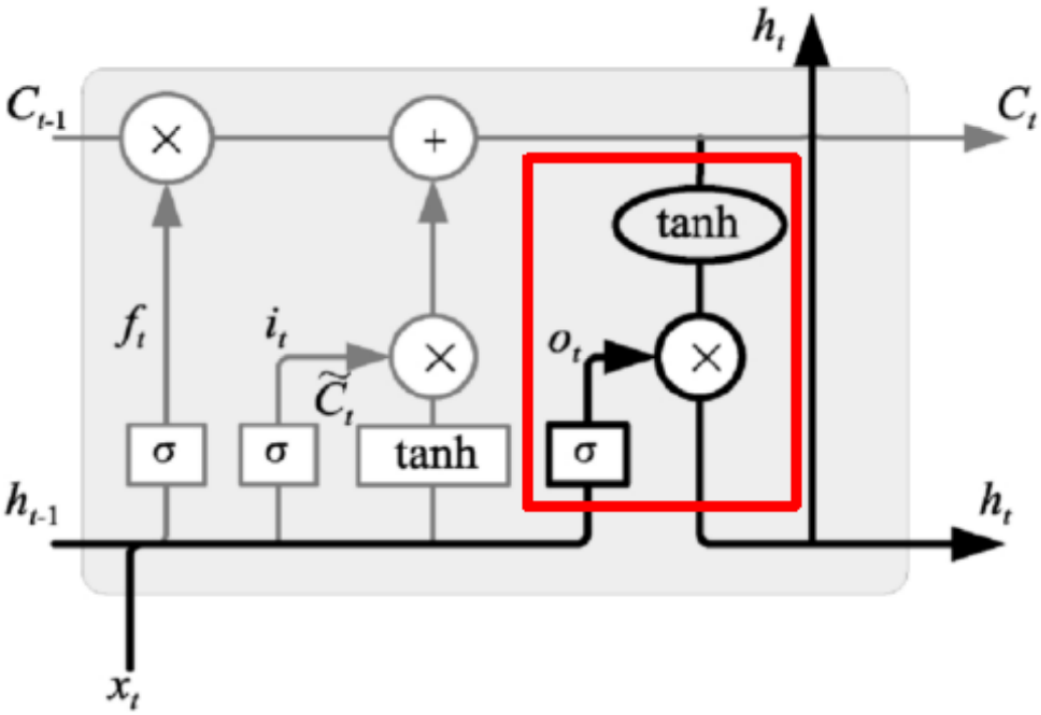
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

遗忘门找到了需要忘掉的信息后，再将它与旧状态相乘，丢弃掉确定需要丢弃的信息。

再将结果加上使细胞状态获得新的信息，这样就完成了细胞状态的更新。

输出门：决定输出哪些信息

> 决定从细胞状态中输出哪些信息作为隐藏状态。



公式：

$$O_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = O_t \cdot tanh(C_t)$$

在输出门中，通过一个Sigmoid层来确定哪部分的信息将输出，接着把细胞状态通过Tanh进行处理（得到一个在-1～1 之间的值）并将它和Sigmoid门的输出相乘，得出最终想要输出的那部分。

公式汇总

$$\begin{aligned}
f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad // \text{遗忘门} \\
i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad // \text{输入门} \\
\tilde{C}_t &= \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad // \text{候选细胞状态} \\
C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad // \text{细胞状态更新} \\
o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad // \text{输出门} \\
h_t &= o_t \odot \tanh(C_t) \quad // \text{隐藏状态输出}
\end{aligned}$$

4.9 GRU 门控单元

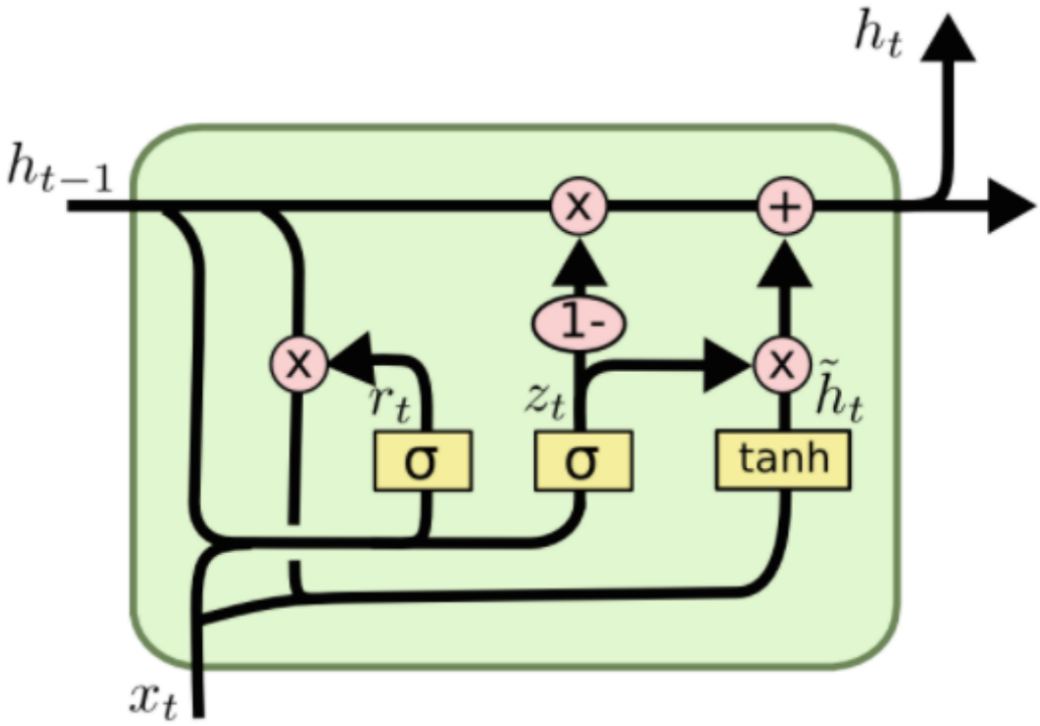
4.9.1 定义

LSTM很好地解决了原生RNN梯度消失的问题，能处理更长的序列问题，但内部结构较为复杂。

2014年，Yoshua Bengio的团队提出了一种更好的LSTM变体GRU（Gated Recurrent Unit，门控环单元），将LSTM内部由四个子神经网络减少为三个，性能和LSTM接近。

GRU包含两个门（重置门和更新门），去除了单元状态，转而使用隐藏状态来传输信息，因此其参数减少，效率更高。

4.9.2 网络结构



重置门决定 "忘掉多少过去"，更新门决定 "保留多少过去、加入多少现在"，两者共同控制隐藏状态的更新。

重置门 r_t

决定 "忘掉多少过去的信息

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

更新门 z_t

决定 "保留多少旧状态、加入多少新信息"

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

公式

$$
\begin{aligned}
z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\
r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\
\tilde{h}_t &= tanh(W \cdot [r_t * h_{t-1}, x_t]) \\
h_t &= (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t \\
y_t &= softmax(W_o h_t)
\end{aligned}
$$

其中， zt为更新门， rt为重置门， ht为节点状态， ht为隐藏层输出， yt为输出

4.10 综合项目

(1) 基于RNN的文本分类

项目概述

目标：利用训练数据集，对模型训练，从而实现对中文新闻信息有效的分类。

数据集：中文新闻信息

步骤:



代码

```
"""
pytorch + RNN 系列完成文本情感分析
pip install torchmetrics
"""
# 引入依赖
import os
import re
import string
import pickle
import jieba
from collections import Counter
import torch
```

```python
from torch.utils.data import Dataset, DataLoader
from torch import nn
from torchmetrics import Accuracy
from sklearn.model_selection import train_test_split

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# 1.文本处理
def process_text(text):
    unwanted = string.punctuation + ', 。 ！ ： ； ？ 、 ‘’“”《》【】（）…— \n\r\xa0'
    clean_text = re.sub(f'[{re.escape(unwanted)}]', '', text)

    return jieba.lcut(clean_text.strip())


# 2.解析数据
def load_data(file_path):
    texts, labels, label2name = [], [], {}
    with open(file_path, 'r', encoding="utf-8") as f:
        for line in f:
            parts = line.split("_!_")
            if len(parts) < 4:
                continue

            label = int(parts[1])
            labels.append(label)
            label2name[label] = parts[2]
            texts.append(parts[3])

    return texts, labels, label2name

# 3.构建词表
def build_vocab(texts, vocab_size=12000):
    word_counter = Counter()
    for text in texts:
        word_counter.update(process_text(text))

    vocab = {'<PAD>': 0, '<UNK>': 1}

```

```
    # 词频从高到低排序
    for word, _ in word_counter.most_common(vocab_size - 2):
        vocab[word] = len(vocab)

    return vocab

# 4.文本转向量,文本对齐
def text_to_idx(text, vocab):
    return [vocab.get(w, 1) for w in process_text(text)]


def pad_seq(seq, max_len=30):
    seq = seq[:max_len]
    return seq + [0] * (max_len - len(seq))


# 5.自定义数据集
class NewsDataset(Dataset):
    def __init__(self, texts, labels, vocab):
        self.texts = [pad_seq(text_to_idx(t, vocab)) for t in texts]
        self.labels = labels

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        return torch.LongTensor(self.texts[idx]), torch.tensor(self.labels[idx])


# 6.定义模型
class RNNNet(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        # (batch,seq_len,feature) (seq_len,batch,feature)
```

```
        self.rnn = nn.RNN(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = self.embedding(x)
        out, _ = self.rnn(x)
        return self.fc(out.mean(dim=1))


# LSTM
class LSTMNet(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = self.embedding(x)
        out, _ = self.lstm(x)
        return self.fc(out.mean(dim=1))


# 7.训练评估
def train_model(model, train_loader, val_loader, epochs=10, lr=0.001):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(epochs):
        # 训练
        model.train()
        total_loss, correct, total = 0, 0, 0
        for batch_x, batch_y in train_loader:
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)
            optimizer.zero_grad()
```

```
            output = model(batch_x)
            loss = criterion(output, batch_y)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()
            correct += (output.argmax(1) == batch_y).sum().item()
            total += len(batch_y)

        train_acc = correct / total
        avg_loss = total_loss / len(train_loader)

        # 验证
        model.eval()
        val_correct, val_total = 0, 0
        with torch.no_grad():
            for batch_x, batch_y in val_loader:
                batch_x, batch_y = batch_x.to(device), batch_y.to(device)
                output = model(batch_x)
                val_correct += (output.argmax(1) == batch_y).sum().item()
                val_total += len(batch_y)

            val_acc = val_correct / val_total

            print(f"Epoch {epoch + 1}/{epochs} | loss: {avg_loss:.4f} | 训练acc:{train_acc:.4f} | 验证acc: {val_acc:.4f}")


# 8.完整流程
# 8.1 加载数据
texts, labels, label2name = load_data("./data/news_classify_data.txt")

# 8.2 划分数据集
train_texts, val_texts, train_labels, val_labels = train_test_split(texts, labels, test_size=0.2, random_state=42,

             stratify=labels)
```

```
158
159 # 8.3 加载词表
160 vocab = build_vocab(train_texts, vocab_size=12000)
161
162 # 8.4 数据加载器
163 train_loader = DataLoader(NewsDataset(train_texts,
    train_labels, vocab), batch_size=32, shuffle=True)
164 val_loader = DataLoader(NewsDataset(val_texts,
    val_labels, vocab), batch_size=32, shuffle=False)
165
166 # 8.5训练
167 # print("训练RNN")
168 # rnn_model = RNNNet(len(vocab), 128, 128,
    len(label2name))
169 # train_model(rnn_model, train_loader, val_loader,
    epochs=20)
170
171 print("训练LSTM")
172 lstm_model = LSTMNet(len(vocab), 128, 128,
    len(label2name))
173 train_model(lstm_model, train_loader, val_loader,
    epochs=20)
174
175 # 8.6 测试
176 print("测试")
177 test_texts = [
178     "沪指跌破3000点关口，两市成交额不足万亿",
179     "央行宣布降准0.5个百分点，释放长期资金约万亿",
180     "教育部发布新规：中小学课后服务全面推行5+2模式",
181     "考研报名人数首破五百万，专硕占比持续攀升",
182     "国足世预赛1-1战平沙特，武磊点球破门救主",
183     "巴黎奥运会倒计时一百天，中国代表团成立大会在京召开",
184     "华为发布全新折叠屏手机，售价六千起支持卫星通信",
185     "英伟达市值突破三万亿美元，AI芯片需求持续爆发",
186     "一线城市二手房成交量回暖，业主报价开始企稳",
187     "多地出台公积金新政，最高贷款额度上调二十万",
188     "新能源汽车渗透率突破四成，国产品牌占据半壁江山",
189     "某新势力车企宣布降价三万，老车主可获现金补偿",
190 ]
191 lstm_model.eval()
192 for text in test_texts:
193     idx = pad_seq(text_to_idx(text, vocab))
194     x = torch.LongTensor([idx]).to(device)
```

```
    with torch.no_grad():
        output = lstm_model(x)
        pred = output.argmax(dim=1).item()
        prob = torch.softmax(output, dim=1)[0][pred].item()

    print(f"文本:{text} --> 预测:{label2name[pred]}, 置信度:{prob:.4f}")

```

4.11 多层RNN

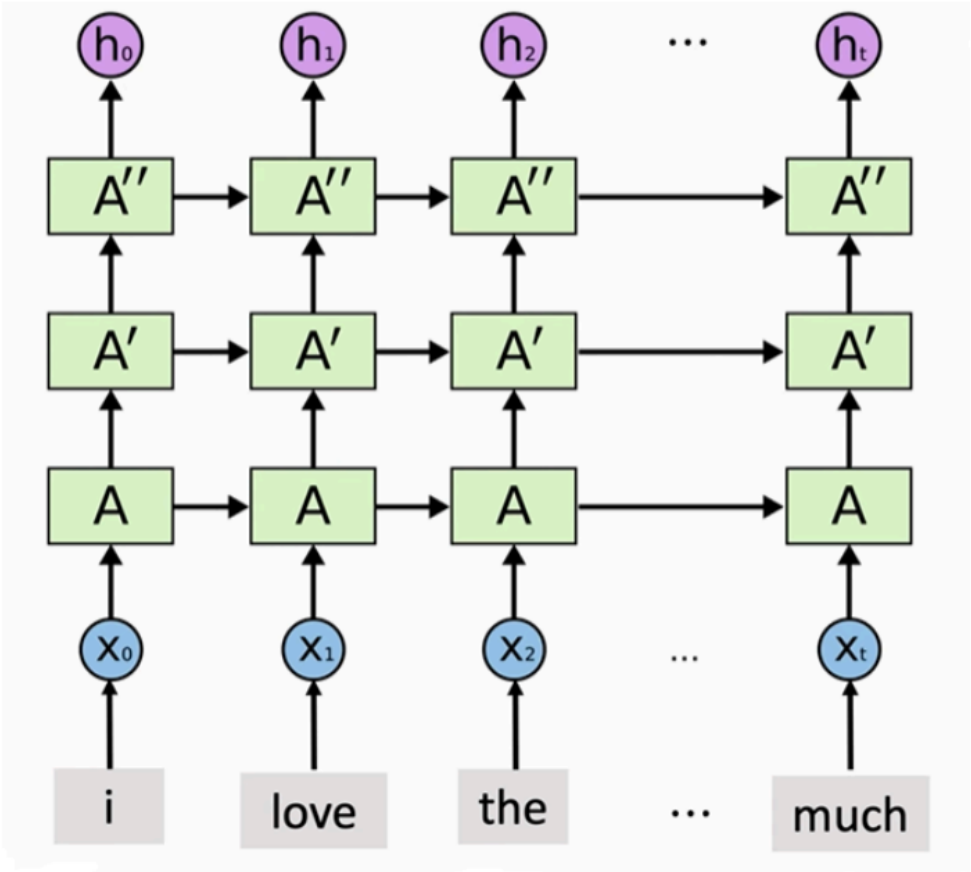
可以将许多RNN层堆叠，构成一个多层RNN网络。

RNN每读取一个新的输入，就会生成状态向量作为当前时刻的输出和下一时刻的输入状态。

将T个输入依次输入RNN，相应地会产生T个输出。

第一层RNN输出的T个状态向量可以作为第二层RNN的输入，第二层RNN拥有独立的参数，依次读取T个来自第一层RNN的状态向量，产生T个新的输出。

第二层RNN输出的T个状态向量可以作为第三层RNN的输入，依此类推，构成一个多层RNN网络。



一般认为，多层RNN层次越高，学习到的语义特征越抽象（类比CNN中的卷积操作）。

公式

$$h_t^{(l)} = \sigma\left(W_h^{(l)} h_{t-1}^{(l)} + W_x^{(l)} h_t^{(l-1)} + b^{(l)}\right)$$

4.12 双向循环神经网络

双向循环神经网络（BRNN）由两个循环神经网络组成，一个正向、一个反向，两个序列连接同一个输出层。

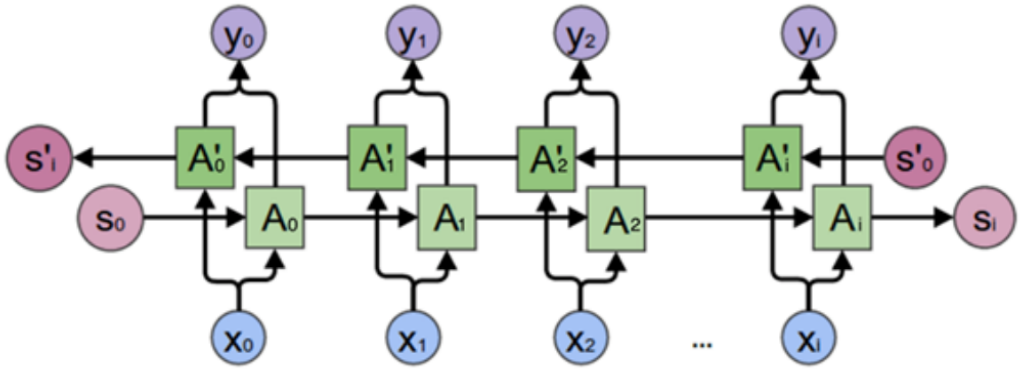
正向 RNN提取正向序列特征，反向RNN提取反向序列特征。

例子

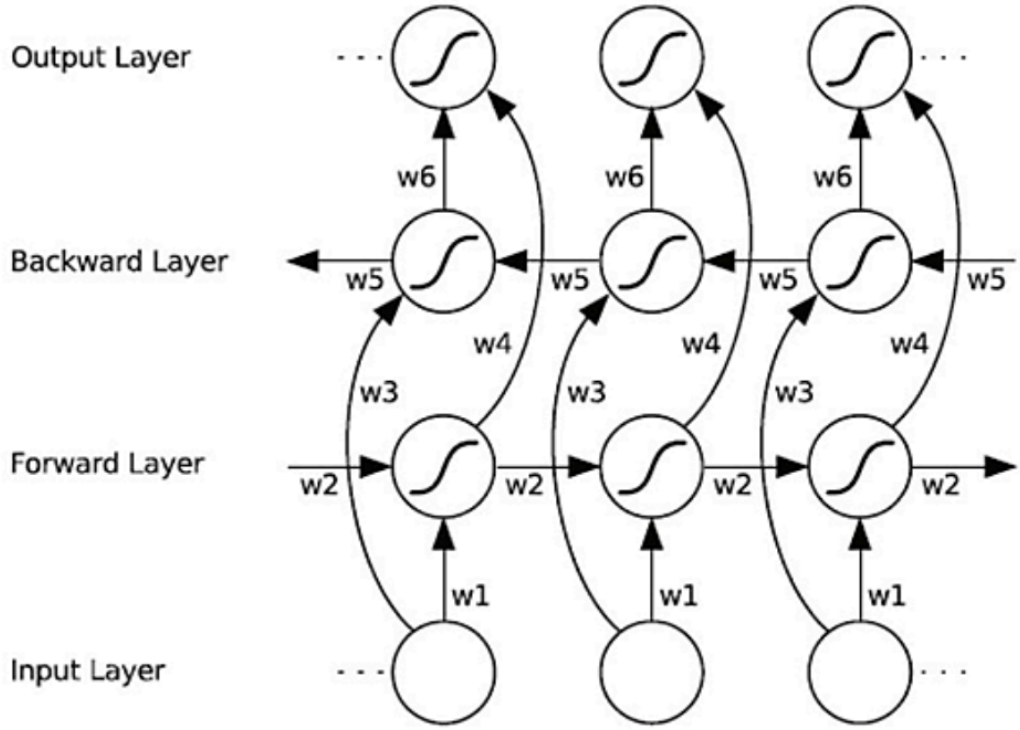
```
我喜欢苹果，比安卓用起来更流畅些
我喜欢苹果，基本上每天都要吃一个
```

根据后面的描述，我们可以得知，第一句中的"苹果"指的是苹果手机，第二句中的"苹果"指的是水果。

双向循环神经网 络结构如下图所示：



权重设置



公式

$$
\begin{aligned}
h_t &= f(w_1 x_t + w_2 h_{t-1}) \\
h'_t &= f(w_3 x_t + w_5 h'_{t+1}) \\
o_t &= g(w_4 h_t + w_6 h'_t)
\end{aligned}
$$

其中， ht为 t 时刻正向序列计算结果， ht' 为 t 时刻反向序列的计算结果，将正向序列、反向序列结果和各自权重矩阵相乘，相加后结果激活函数产生 t 时刻的输出。

通常情况下，双向循环神经网络能获得比单向网络更好的性能。

4.13 语言模型与自回归

4.13.1 定义

语言模型（Language Model, LM）是对自然语言序列概率分布的建模。

给定一个词序列 $w_1, w_2, \ldots, w_T$，语言模型给出这个序列出现的概率：$P(w_1, w_2, \ldots, w_T)$

4.12.2 链式法则与自回归

直接建模 $P(w_1, \ldots, w_T)$ 很难，因为序列是变长的、组合爆炸。

利用概率链式法则分解：

$$P(w_1, w_2, \ldots, w_T) = P(w_1) \cdot P(w_2|w_1) \cdot P(w_3|w_1, w_2) \cdot \ldots \cdot P(w_T|w_1, \ldots, w$$

$$= \prod_{t=1}^{T} P(w_t|w_1, w_2, \ldots, w_{t-1})$$

自回归（Autoregressive）的含义：每一步的预测都依赖于之前已经生成的结果。

"自" = 自己生成的历史，"回归" = 基于历史预测下一步。

4.12.3 RNN 语言模型（RNN-LM）

用 RNN 来参数化条件概率 $P(w_t|w_{<t})$，就是 RNN 语言模型。

结构图

```
输入:     <sos>     w₁       w₂       w₃      ...
           |        |        |        |
           ▼        ▼        ▼        ▼
Embedding:e₀       e₁       e₂       e₃
           |        |        |        |
           ▼        ▼        ▼        ▼
RNN:  ──→ h₀ ──→ h₁ ──→ h₂ ──→ h₃ ──→ ...
                    |        |        |
                    ▼        ▼        ▼
Output:           ŷ₁       ŷ₂       ŷ₃
                (预测w₁)  (预测w₂)  (预测w₃)
```

公式

$$h_t = \text{RNN}(h_{t-1}, e(w_t))$$

$$P(w_{t+1}|w_1, \ldots, w_t) = \text{Softmax}(Wh_t + b)$$

其中 $e(w_t)$ 是词 w_t 的嵌入向量，h_t 是第 t 步的隐藏状态。

训练目标是最大化训练数据的对数似然：

$$\mathcal{L} = -\frac{1}{T} \sum_{t=1}^{T} \log P(w_t|w_1, \ldots, w_{t-1})$$

训练时，每一步的输入是真实的前一个词 w_{t-1}，而不是模型自己预测的 $\hat{w}_{t-1}$。这就是 Teacher Forcing。

Teacher Forcing

假设不用 Teacher Forcing，训练时也用模型自己的预测：

- 第1步：输入 `<sos>` → 预测 $\hat{w}_1$（大概率是错的，因为模型还没训练好）
- 第2步：输入 $\hat{w}_1$（错的）→ 预测 $\hat{w}_2$（更错）
- 第3步：输入 $\hat{w}_2$（错的）→ 预测 $\hat{w}_3$（错上加错）

误差累积：早期的错误会被不断放大，模型永远学不好。梯度也很难有效传播。

Teacher Forcing 的本质是用真实的历史代替预测的历史，保证每一步都在正确的输入分布上学。

暴漏偏差

训练时的条件分布是 $P(w_t|w_{<t}^{\text{real}})$，推理时是 $P(w_t|\hat{w}_{<t})$。这两个分布不一致， 这就是暴露偏差。

缓解方案

方法	思想	优缺点
Scheduled Sampling	以概率p用真实词，1-p用预测词，p随训练衰减	简单有效，但训练不稳定
Professor Forcing	用GAN让隐藏状态分布对齐	理论优雅，训练复杂
Beam Search Training	训练时就用beam路径	计算量大
Sequence-Level Training	直接优化序列级指标（如BLEU）	不可微，需强化学习

从概率分布到序列

(1) Greedy Decoding（贪心解码）

每一步都选概率最大的词：

$$\hat{w}_t = \arg\max_{w_t} P(w_t|\hat{w}_1, \ldots, \hat{w}_{t-1})$$

问题：局部最优 ≠ 全局最优。每一步都选最好的，组合起来不一定是最好的序列。

```
示例：
  Step 1: P("他")=0.5, P("她")=0.4 → 选"他"
  Step 2: P("他是")→"老师"=0.3, P("她是")→"医生"=0.9

  贪心路径: 他→老师 (0.5×0.3=0.15)
  更优路径: 她→医生 (0.4×0.9=0.36)  ← 贪心错过了
```

(2) Beam Search（集束搜索）

每一步保留概率最高的 k 个候选序列（k = beam size），在 k 个路径上同时扩展。

算法流程：

```
1. 初始化：beams = [("<sos>", 分数=0)]，只有一条路径
2. 对每一步：
   2.1 把 beams 里的每条路径都扩展一步
       （每条路径取 top-k 个词，生成 k×k 个候选）
   2.2 从所有候选中，按总分数排序，保留前 k 条
   2.3 如果某条路径生成了 <eos>（句子结束标记），
       就把它放到"已完成"列表里，不再扩展

3. 最后，从所有已完成的路径中选分数最高的
```

关键技巧：

- **对数概率**：概率是乘法，很多个小数相乘会数值下溢。改用对数概率，乘法就变成了加法，数值稳定得多
- **长度归一化**：长序列天然概率低，需除以长度惩罚

$$\text{score}(y) = \frac{1}{T^{\alpha}} \sum_{t=1}^{T} \log P(y_t | y_{<t})$$

$\alpha \in [0.6, 1.0]$，$\alpha = 0$ 无惩罚，$\alpha = 1$ 完全归一化

语言模型局限

语言模型很强大，但它有一个根本局限：语言模型只有一个序列，就是它自己生成的那个序列。

```
语言模型能做的：
  输入：今天天气真
  输出：好，我们出去玩吧
```

现实中大量的任务不是这样的。很多任务是输入一个序列，输出另一个完全不同的序列。

举几个例子：

任务	输入	输出	特点
机器翻译	中文句子	英文句子	两种语言，语序可能不同
文本摘要	长文章	简短摘要	长度差异很大
对话系统	用户问题	机器回答	两个说话者，不是一回事
语音识别	音频序列	文本序列	模态都不一样
代码生成	自然语言描述	代码	领域完全不同
语法纠错	错误句子	正确句子	同语言但有修改

这些任务的共同特点：

- **输入是一个序列，输出也是一个序列**
- **输入和输出是两个独立的序列，不是同一个序列的前后部分**
- **输入和输出的长度可以不同**
- **输入和输出之间有对应关系，但不是简单的续写**

4.12.4 Encoder

Encoder 负责 "读" 输入序列。它把输入序列一个词一个词地读进去，最后输出一个向量，代表整个输入的 "意思"。

这个向量叫做上下文向量（Context Vector），记作 c

```
输入: x₁ → x₂ → x₃ → ... → xₙ
                ↓
          [Encoder]
                ↓
          上下文向量 c
     （代表整个输入序列的意思）
```

4.12.5 Decoder

Decoder 负责 "写" 输出序列。

Decoder 本质上也是语言模型，它也是自回归地逐词生成。但它跟普通语言模型有一点不一样：普通语言模型只看自己生成的历史，而 Decoder 除了看自己生成的历史，还要看 Encoder 给它的上下文向量 c。

所以 Decoder 是一个"有条件的"语言模型，条件就是 Encoder 编码出来的输入序列的意思。

把 Encoder 和 Decoder 合起来，就叫 Seq2Seq（Sequence to Sequence），也叫 Encoder-Decoder 框架。

4.14 Seq2Seq

4.14.1 定义

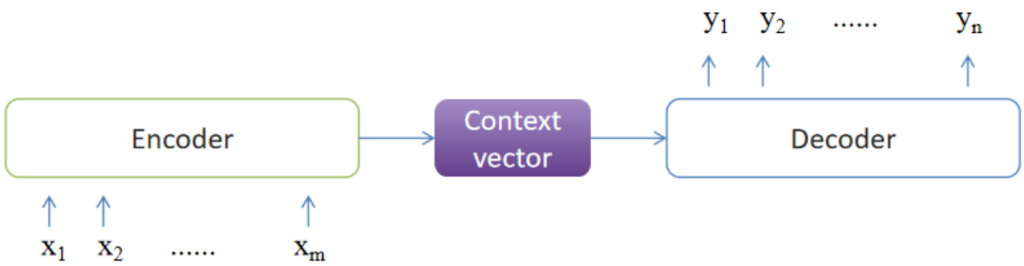
Seq2Seq（Sequence to Sequence，序列到序列）用于输入一个序列，输出一个序列，且输入输出序列长度不相等的情况，例如机器翻译、聊天机器人、文本摘要、智能问答等

Seq2Seq属于典型的Encoder-Decoder结构。

Encoder （编码器）和Decoder（解码器）分别代表两个子模型，Encoder负责将输入转换为一种中间表达(语义向量)， Decoder根据中间表达产生输出值。

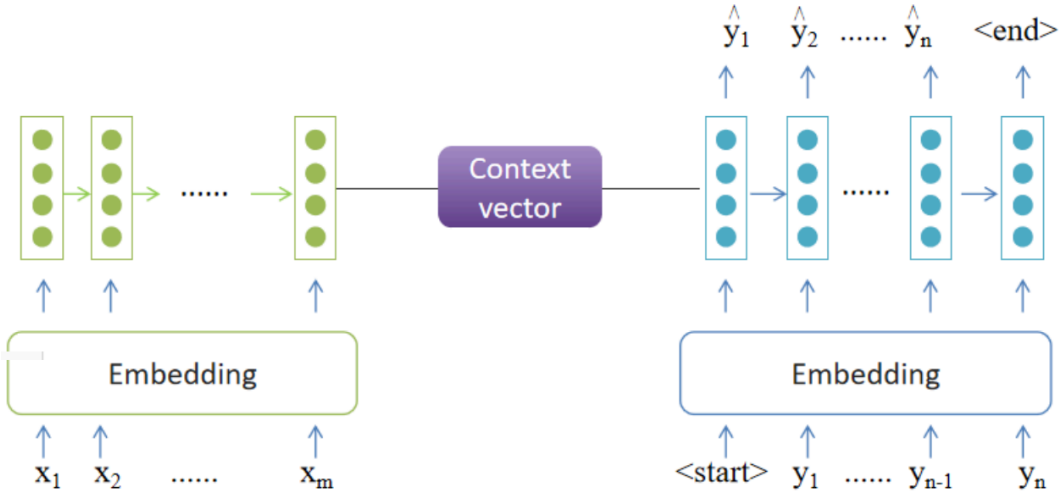
4.14.2 网络结构

整体结构



Encoder-Decoder结构示意图

训练阶段



Encoder-Decoder训练阶段详细结构

在Encoder端，将原文本先经过embedding层转换成词向量（包含语义信息），然后输入到一个RNN结构中（可以是RNN，LSTM， GRU，也可以是多层或双向结构），经过Encoder转换，将原文本压缩到一个语义向量中（图中Context Vector部分）。

Decoder部分稍复杂一些，训练阶段，因为有标签数据，所以以标签数据作为输入，以特殊字符 <start>表示句子开始，每一步根据当前正确的数词和上一步的隐藏状态来预测下一个词

$$P(y_1, y_2, \ldots, y_{T'}|x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T'} p(y_t|y_{t-1}, \ldots, y_2, y_1, x_T, \ldots, x_2, x_1)$$
$$= \prod_{t=1}^{T'} p(y_t|y_{t-1}, \ldots, y_2, y_1, c)$$

其中，x1,x2,.....xt 表示输入， y1,y2,...,yt'表示对应输出，c输入和输出长度可能不一样，表示经过编码器计算 的固定长度语义向量。

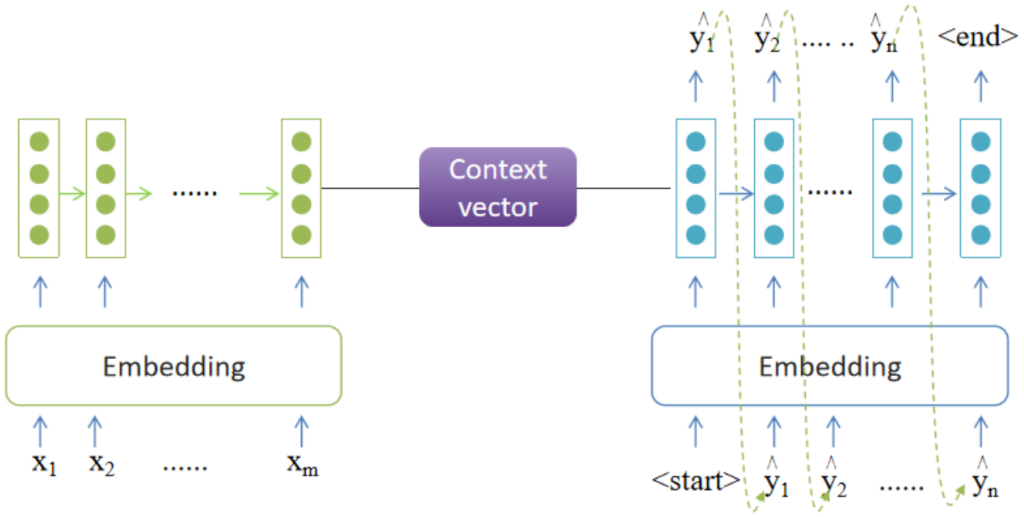
在计算损失时，可以使用词语级交叉熵作为损失函数，将每一步损失值求和，即得到总体损失函数：

$$J = -log(p(\hat{y}_1)) - log(p(\hat{y}_2)) - \ldots - log(p(\hat{y}_n)) - log(p([< end >]))$$
$$= -\frac{1}{T}\sum_{i}^{T} log(p(\hat{y}_i))$$

其中，T代表Decoder有多少步，<end>代表结束标记

推理阶段

推理阶段，Encoder端没有什么变化，而Decoder端，由于此时没有标签值，所以先根据语义向量和<start>标志预测出第一个词，然后将第一个词作为第二步输入，预测出第二个词，以此类推



推理阶段优化

① 训练方式的优化

在训练阶段，Decoder输入是标签序列y1,y2,y3....yn ，这个"标准答案"就相当于引导信息，根据“标准答案”来做解码 的方式称为 ”teacher forcing“。

如果推理阶段没有标签序列引导，会导致模型没有线索情况下进行“自由发挥”，前面的词预测错误，后面的词会跟着错误，造成“一步错， 步步错”的现象，这样导致“误差爆炸”问题（累积误差太大）。

在推理阶段，Decoder根据上一步的输出作为下一步的输入来做解码（没有标准答案），这种方式称为“free running”。

训练阶段Decoder有标签序列引导，推理阶段则“独闯江湖”，可能造成模型性能下降，所以，更好的办法是在训练阶段，只给予部分引导，让解码器提前“独闯江湖”更好适应推理阶段的环境。

通过设置一个概率p，每一步解码时，以概率p通过上一步输入来预测，以概率1-p根据标签序列引导，这种方式称为“计划采样”（scheduled sampling）。

② 解码方式的优化

在解码阶段，每一步根据前面的输入，预测概率最大的下一个词，然后再作为下一步的输入，这种方式称为"Greedy Decoding"（贪婪解码）,但每一步最优不一定是全局最优（贪婪算法的特点）。

对于这种情况，可以采用Beam Search方法改进：每一步，多选几个作为输入，然后综合考虑，选出最优的组合。

首先，设定一个候选集大小beam_size = k

每一步解码，从当前输入对应所有的输出，计算每一条路径序列组合的得分

保留上一步得分最高的k个序列组合作为下一步的输入

不断重复第二步、第三步，直至结束，选择得分最高的序列组合作为最终结果

4.14.3 优缺点

优点

- 处理序列任务具有较高的精度和较好的适应性
- 能处理输入、输出长度不相等的问题，无需对序列进行对齐和归一化处理
- 是一个端对端模型，在实际任务中使用方便

缺点

- 训练和推理速度较慢，因为每一步都是以上一步的输出作为输入，导致并行化计算困难
- Encoder将整个输入，压缩到一个定长的语义向量中，如果输入序列过长，模型参数较小，会出现语义损失； 如果模型参数过大，会出现信息过载的问题

4.14.4 信息瓶颈

信息瓶颈（Information Bottleneck）：上下文向量 c 的维度是固定的，但输入序列长度是变化的。序列越长，信息越多，固定维度的向量必然丢失信息。

长序列质量骤降: 短序列翻译得还不错，序列一长质量就暴跌。因为长序列的信息太多，固定向量装不下，很多细节就丢了。

静态上下文，无法动态关注: 解码的每一步都用同一个上下文向量 c。

长距离依赖衰减: 即使没有信息瓶颈，RNN 本身也有问题，长距离依赖衰减。

5. 注意力机制

5.1 定义

Attention中文译作注意力，诞生于机器翻译领域，2015年由Bahdanau等人在论文中提出，用来提升机器翻译模型的精度。

Attention在Encoder-Decoder模型中，单独添加一组权重（注意力权重），用来确定输出词和输出词之间的关联程度, 后来又发展出自注意力机制，用来考察同一个句子中词语之间的关联程度。

5.2 核心思想

在使用Seq2Seq模型进行翻译过程中，需要将源序列的全部内容压缩为一个固定大小的向量中（语义向量），然后通 过该向量进行解码，产生输出序列。这种方式无法并行化训练，输入元素和输出元素之间没有考虑相关性。

注意力机制会对每个输入和输出之间，单独使用一组注意力权重，这组注意力权重，允许Decoder在进行解码时，回头看整个输入序列，输入元素、输出元素之间通过权重确定其相关性。

Attention机制传统Encoder-Decoder模型相比，主要有以下几点不同：

（1）Encoder提供了更多数据给Decoder，Encoder会把所有节点的隐藏状态提供给Decoder，而不仅仅是最后一个节点的隐藏状态；

（2）Decoder并不是把所有Encoder提供的隐藏状态作为输入，而是采用一种选择机制，把最符合当前位置的隐藏状态选取出来。

跟原始 Seq2Seq 的关键区别

组件	原始 Seq2Seq	Seq2Seq + Attention
编码器输出	只用最后一个 h_n	保留所有 $h_1, h_2, \ldots, h_n$
上下文向量	固定的 c	动态的 c_t（每步不同）
信息传递	单向量瓶颈	动态检索所有位置
对齐能力	无	有（注意力权重即软对齐）

5.3 计算步骤

注意力计算可以拆成三步：打分 → 归一化 → 加权求和。

第一步：打分（Scoring）

假设现在解码器处于第 t 步，隐藏状态是 s_t。编码器有 n 个隐藏状态 $h_1, h_2, \ldots, h_n$。

我们要算的是：当前解码器状态 s_t 和每个编码器状态 h_i 之间的"相关度"。

$$e_{ti} = \text{score}(s_t, h_i)$$

第二步：归一化（Softmax）

有了分数之后，用 Softmax 把它们变成概率分布，也就是权重加起来等于 1：

$$\alpha_{ti} = \frac{\exp(e_{ti})}{\sum_{j=1}^{n} \exp(e_{tj})}$$

α_{ti} 就是注意力权重，生成第 t 个词的时候，分配给输入序列第 i 个位置的权重。

权重都是正数，加起来等于1，就像一个"注意力分配方案"。

第三步：加权求和（Weighted Sum）

有了权重之后，用这些权重对编码器所有隐藏状态加权求和，就得到了当前步的上下文向量：

$$c_t = \sum_{i=1}^{n} \alpha_{ti} h_i$$

c_t 就是"当前这一步，我应该关注的输入信息的汇总"。

完整公式合起来

$$c_t = \sum_{i=1}^{n} \frac{\exp(\text{score}(s_t, h_i))}{\sum_{j=1}^{n} \exp(\text{score}(s_t, h_j))} \cdot h_i$$

5.4 Q/K/V

5.4.1 Query（查询）

Decoder 的状态 s， 它是去 "查询" 的，去问 "谁和我最相关"。

我们给它起个名字，叫 **Query（查询）**，简称 **Q**。

5.4.2 Key（键）

Encoder 的那些状态 h_1、h_2、h_3，它们是被查询的对象，就像字典里的 "键"。

我们给它们起个名字，叫 **Key（键）**，简称 **K**。

5.4.3 Value（值）

最后加权求和的时候，用的也是 Encoder 的状态， 它们是真正的内容，就像字典里的 "值"。

我们给它们起个名字，叫 **Value（值）**，简称 **V**。

5.5 注意力类型

5.5.1 加性注意力（Bahdanau Attention）

加性注意力（Additive Attention，又称 Bahdanau Attention），计算相似度中， 使用了加法进行计算，归一化后做加权求和。

公式

$$e_{ti} = v_a^\top \tanh(W_a s_t + U_a h_i)$$

优点:

- 表达能力强（有非线性 tanh，可以学习复杂的相关性模式）
- 不要求 s_t 和 h_i 维度相同（因为各自有投影矩阵）

缺点:

- 计算量大（每个位置都要过一个 MLP）
- 不容易高效并行化

5.5.2 乘性注意力（Luong Attention）点积/点乘

两个向量越相似，点积就越大，用点积算相似度的注意力，统称为乘性注意力，乘性注意力不是一个具体的算法，而是一个家族。

最简单的乘性注意力，就是直接做点积：

$$e_{ti} = s_t^\top h_i$$

优点：

- 最简单，没有参数
- 计算最快 ， 就是矩阵乘法，可以高度并行

缺点：

- 要求 Query 和 Key 维度必须相同， 不同就没法点积
- 表达能力弱，只能算线性相似度

5.5.3 双线性注意力

加一个投影矩阵 W，先把 Query 投影到 Key 的维度，再点积。

$$e_{ti} = s_t^\top W_a h_i$$

这就是双线性注意力，很多地方直接叫它 "乘性注意力"，因为点积太简单了，通常单独叫 "点积注意力"，而带 W 的这个就代表 "乘性注意力"。

和点积的关系：

- 点积是双线性的特例，当 W 是单位矩阵时，双线性就退化成了点积
- 双线性是点积的通用版，多了一个可训练的 W

优点：

- 比加性注意力快
- Query 和 Key 维度不一样也能用
- 表达能力比点积强，W 可以学习不同维度之间的对齐方式

缺点：

- 还是线性的 ， 没有非线性，表达能力不如加性
- 多了一个参数矩阵 W

假设 s_t 和 h_i 的每个分量都是均值为 0、方差为 1 的独立随机变量。

$$\mathrm{Var}(s_t^\top h_i) = \sum_{j=1}^{d_k} \mathrm{Var}(s_j \cdot h_j) = d_k$$

5.5.4 缩放点积注意力

除以 $sqrtd_k$， 也就是 Key 维度的平方根。

$$e_{ti} = \frac{s_t^\top h_i}{\sqrt{d_k}}$$

是Transformer 中使用的注意力。

乘性注意力家族

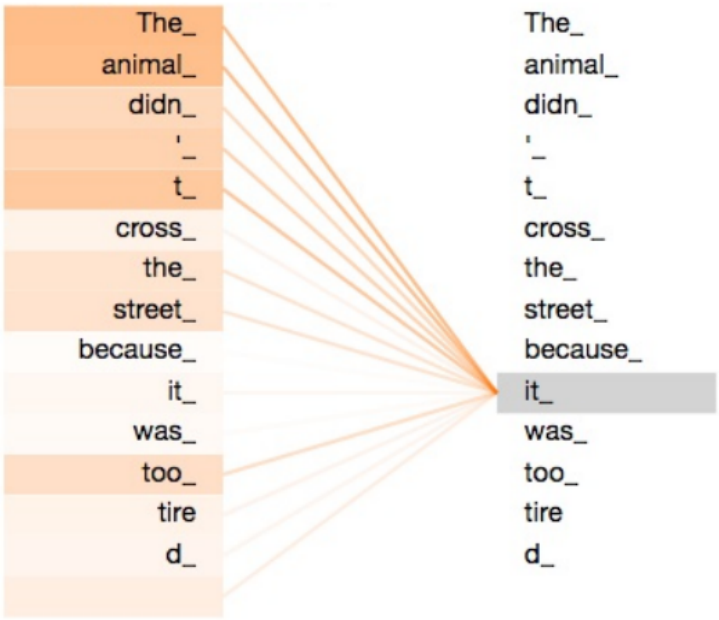
```
乘性注意力家族
    |
    ├─ 点积注意力（Dot-Product）
    |     公式：s^T h
    |     特点：最简单、无参数、要求维度相同
    |
    ├─ 双线性注意力（General / Bilinear）
    |     公式：s^T W h
    |     特点：带投影矩阵、维度可以不同、表达力更强
    |
    └─ 缩放点积注意力（Scaled Dot-Product）
          公式：s^T h / √d_k
          特点：加了缩放、解决高维梯度消失、Transformer用的
```

5.5.5 自注意力（Self-Attention）

自注意力就是：Query、Key、Value 都来自同一个序列。

也就是序列中的每个词，都去和序列中所有词（包括自己）算注意力。

自注意力可以捕捉句子内部词与词之间的依赖关系。



5.5.6 全局注意力（Global Attention）

全局注意力就是：计算注意力的时候，看所有的输入位置。

比如句子有 100 个词，就和 100 个词都算一遍注意力。

优点：信息全面，效果好

缺点：计算量大，序列长了很慢

5.6.7 局部注意力（Local Attention）

局部注意力就是：只看一部分输入位置。

比如句子有 100 个词，但我只看当前位置附近的 10 个词。

优点：计算快，适合长序列

缺点：可能漏掉远处的重要信息

5.6.8 软注意力（Soft Attention）

软注意力就是：给每个位置一个权重，加权求和。

就是我们前面讲的所有注意力，Softmax 之后，每个位置都有一个权重，然后把所有 Value 加权加起来。

特点：

- 每个位置都看，只是权重不同

- 是可微的，可以用反向传播直接训练
- 平时说 "注意力" 默认就是软注意力

5.6.8 硬注意力（Hard Attention）

硬注意力就是：只选一个位置，直接取那个位置的 Value。

特点：

- 只看一个位置，计算快
- 但是不可微，没法直接用反向传播训练
- 需要用强化学习等方法训练

5.6 综合项目-机器翻译

项目总览

项目名称：基于 Seq2Seq + Attention 的中英神经机器翻译系统

核心任务: 中文 → 英文 / 英文 → 中文 双向神经机器翻译

数据来源: OPUS-100 大规模多语言平行语料库(中文数据质量参差不齐，需要清洗)

数据集划分：训练集（最多 1M / 语言对）+ 验证集（2000 条）+ 测试集（2000 条）

评估指标: BLEU Score + 人工抽样评估

代码

由于代码数量过多, 不方便在笔记中直接显示, 请直接参见完整项目

6. Transformer模型

2017年Google机器翻译团队发表了《Attention is all you need》这篇论文，该论文也是Encoder-Decoder框架，但用 Attention 机制代替了RNN搭建了整个模型，这篇论文将Attention思想发挥到极致，抛弃了以往深度学习任务里面使用到的 CNN 和 RNN，提出一个全新的模型Transformer，这个模型广泛应用与机器翻译、问答系统、文本摘要、 语音识别的NLP领域。

6.1 发展史

6.1.1 Transformer 前时代的困境

在 2017 年 Transformer 诞生之前，序列建模领域经历了从 RNN 到 LSTM/GRU，再到 Seq2Seq 的演进。每一步都在解决前一步的问题，但每一步又都留下了新的瓶颈。

(1) RNN / LSTM / GRU 的序列建模思路与本质缺陷

循环神经网络（Recurrent Neural Network, RNN）是一类专门处理序列数据的神经网络。

其核心思想是：网络在每个时间步 t 接收当前输入 x_t 和上一时间步的隐藏状态 h_{t-1}，通过循环计算产生当前隐藏状态 h_t。过隐藏状态 h_t 将过去的信息"携带"到当前时刻，从而让网络能够利用历史信息来处理当前输入。这就像人在阅读一句话时，会记住前面读过的词，来理解当前词的含义。

本质缺陷一：串行计算，无法并行化

这是 RNN 最根本的架构缺陷。由于第 t 步的计算必须依赖第 t-1 步的结果 h_{t-1}，整个序列只能逐个时间步串行计算。这意味着：

- 序列长度为 n 时，计算的时间步长为 O(n)
- 无法利用现代 GPU 的并行计算能力
- 序列越长，训练越慢

本质缺陷二：长距离依赖衰减

RNN 的第二个致命问题是梯度消失/梯度爆炸导致的长距离依赖困难。

LSTM 和 GRU 的改进与局限

为了解决梯度消失问题，研究者提出了 LSTM（Long Short-Term Memory, 1997）和 GRU（Gated Recurrent Unit, 2014），通过门控机制来控制信息的流动。

但 LSTM/GRU 仍然没有解决根本问题：

- 串行计算的本质没有改变，依然无法并行化
- 长距离依赖只是"缓解"而非"解决"，当序列足够长时，信息仍然会衰减
- 信息路径长度仍然是 O(n)，第 1 个词和第 n 个词之间的信息传递需要经过 n 步

(2) Seq2Seq 架构的瓶颈

无论输入序列有多长（10 个词还是 1000 个词），都要被压缩到同一个固定维度的向量中。

6.1.2 Transformer 之后的时代

Transformer 的诞生开启了一个全新的时代

第一波：NLP 领域的全面革新（2018-2019）

- **BERT**（2018）：Encoder-Only 架构，双向预训练，刷新 11 项 NLP 任务纪录
- **GPT-1/2**（2018/2019）：Decoder-Only 架构，自回归预训练，展示了生成能力
- **T5 / BART**：Encoder-Decoder 架构，统一框架处理各种任务

第二波：跨领域扩展（2020-2021）

- **ViT**（2020）：Vision Transformer，将 Transformer 应用到计算机视觉
- **CLIP**（2021）：多模态，图文对比学习
- **Whisper**（2022）：语音识别

第三波：大语言模型时代（2022-至今）

- **GPT-3.5 / ChatGPT**（2022）：对话式 AI 走入大众视野
- **GPT-4**（2023）：多模态大模型
- **LLaMA、Qwen、DeepSeek 等**：开源大模型蓬勃发展

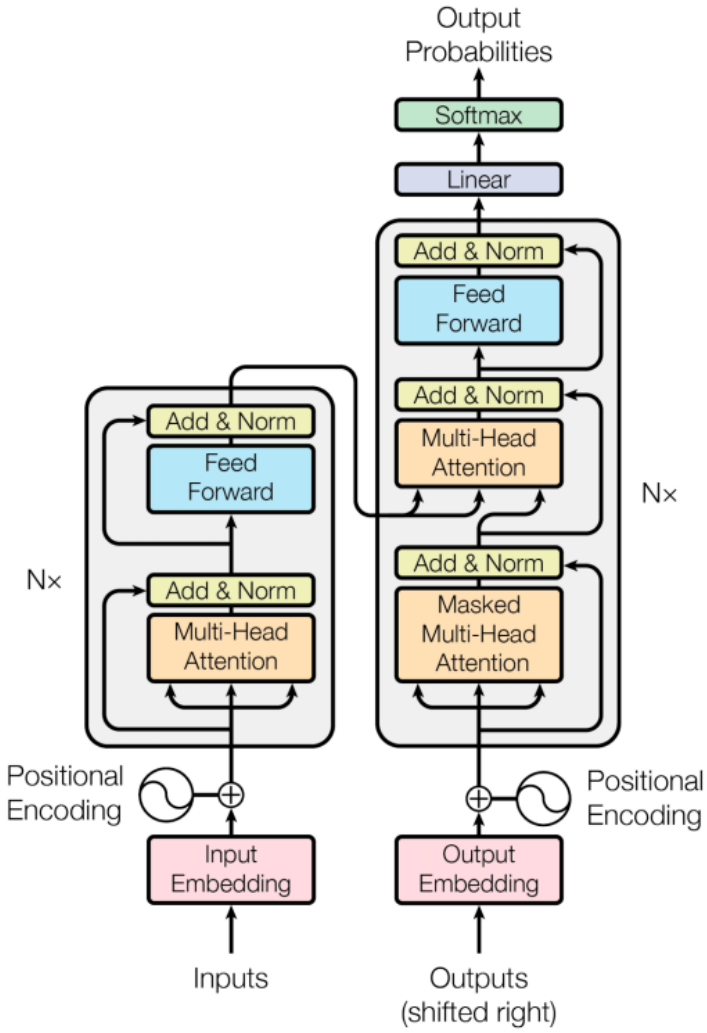
这一切的起点，都是 2017 年那篇论文——《Attention Is All You Need》。

6.2 Transformer 架构

Transformer 是一种完全基于自注意力机制的编码器-解码器架构。

编码器负责将输入序列编码为上下文化的向量表示，解码器负责基于编码器的输出生成目标序列。整个模型不包含任何循环或卷积结构。

整体架构总览:



区域	模块	功能
左侧灰色框	**Encoder（编码器）**	负责 "理解" 输入序列，将其编码为上下文化的特征表示
右侧灰色框	**Decoder（解码器）**	负责 "生成" 输出序列，基于 Encoder 的输出逐步生成目标 token
顶部	**输出层**	Linear + Softmax，将 Decoder 输出转换为词表上的概率分布

左边为Encoder部分，右边为Decoder部分，主要部分介绍如下：

输入：模型的训练基于单向、多对多，不要求输入和输出信息长度相等，当两者长度不等时，将空缺部分填充为0向量；

输出：在一般任务中，模型训练的目的为预测下一个词的概率，从而保持输入和输出信息长度相等，输出的结果相对于输入序列右移了一个位置，即右滑；

N层：代表连续N个计算单元（block），文章中N=6；

多头注意力：上图中Multi-Head Attention即多头注意力，采用了多组注意力权重；

词嵌入：Input Embedding和Output Embedding表示词嵌入层，使用预训练词向量表示文本内容，维度为 计算Q,K,V 512维；

Positional Encoding：位置编码信息，因为缺少RNN结构，所以Transformer输入位置编码来作为词语位置信息，词语位置信息能在一定程度上反映出词语的相似度或关联性；

全连接前馈神经网络：即图中的Feed Forward部分，对每个位置的词嵌入单独进行变换，使其上下文的维度统 一；

Add & Norm：残差结构，将输入和输出相加，再做归一化处理；

Linear：全连接输出层，输出一组概率，其维度为预测目标的词汇表大小. 整个模型中

Self-Attention和Multi-Head Attention是其核心，下面详细进行论述。

6.3 数据流向

```
源序列 Inputs
    → Input Embedding
    → + Positional Encoding
    → N × Encoder（自注意力 → FFN，每层都有残差+LN）
    → Encoder 输出（作为 K, V）

目标序列 Outputs (shifted right)
    → Output Embedding
    → + Positional Encoding
    → N × Decoder:
        掩码自注意力（看已生成的目标序列）
        → 交叉注意力（看 Encoder 输出）
        → FFN
        （每层都有残差+LN）
    → Linear（投影到词表）
    → Softmax
    → 输出概率分布
```

让我们以机器翻译为例，跟踪一个完整的数据流向：

编码器端（输入："I love you"）

```
Step 1: 输入 token 化
        "I love you" → [I, love, you] → [101, 203, 542]（假设的 token ID）

Step 2: 词嵌入
        [101, 203, 542] → 查嵌入矩阵 → 3 个 d_model 维向量

Step 3: 加上位置编码
        嵌入向量 + 位置编码向量 → 带位置信息的向量

Step 4: 经过 6 层 Encoder
        每层：自注意力 → 残差+LN → FFN → 残差+LN
        输出：3 个 d_model 维的上下文化表示
```

解码器端（输出："我爱你"）

```
Step 1: 输入 shifted right 的目标序列
        （训练时）"我爱你" → [<start>, 我, 爱, 你] → 词嵌入 + 位置编码

Step 2: 经过 6 层 Decoder
        每层：
        - 掩码自注意力（只能看到当前及之前的位置）
        - 交叉注意力（Query 来自 Decoder，Key/Value 来自 Encoder 输出）
        - FFN
        （每个子层都有残差+LN）

Step 3: 输出层
        Decoder 输出 → Linear 投影到词表维度 → Softmax → 概率分布

Step 4: 生成下一个 token
        选择概率最大的 token 作为下一个输出
```

这就是 Transformer 的完整数据流，Encoder 负责理解，Decoder 负责生成，交叉注意力负责对齐，自注意力负责上下文建模。

6.4 输入表示层详解

与 RNN 不同，Transformer 本身没有任何内置的序列顺序概念。因此，我们需要做两件事：

1. 把离散的 token 变成连续的向量（词嵌入）

2. 给向量加上位置信息（位置编码）

这两件事共同构成了 Transformer 的输入表示层。

6.4.1 Token Embedding（词嵌入层）

(1) 定义

词嵌入（Token Embedding / Word Embedding）是将**离散的 token ID 映射为连续的低维稠密向量**的操作。

在 Transformer 中，输入不是直接用 one-hot 向量表示的（那样维度太高了，比如词表有 3 万个词，one-hot 就是 3 万维），而是用一个低维的稠密向量来表示每个 token。

(2) 核心思想

- 语义相似的词，在向量空间中距离也近
- 向量的不同维度编码了词的不同语义特征
- 通过学习，嵌入向量能够捕捉词的语法和语义信息

(3) 嵌入维度 d_model 的选择与意义

d_{model} 是 Transformer 中所有隐藏状态的维度，包括嵌入向量、注意力输出、FFN 中间层之外的所有向量。原论文中 $d_{model} = 512$。

d_{model} 决定了模型的"容量"，维度越高，模型能表达的信息越多，但计算量和参数量也越大。

(4) 词表大小与嵌入矩阵的维度关系

嵌入矩阵 E 的形状为 (V, d_{model})，其中：

- V：词表大小（vocabulary size）
- d_{model}：嵌入维度

$$\text{Embedding 参数量} = V \times d_{model}$$

6.4.2 Positional Encoding（位置编码）

(1) 概述

由于Transformer中不包含卷积和循环结构，为了让模型能利用词语顺序信息，所以单独加了一个额外的位置编码向量，维度和embedding一样. 这个向量代表了当前词的位置信息，或在一个句子中不同次之间的距离。

论文中采用的计 算方法如下：

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

其中，pos是位置，i是维度（这里是词的位置索引），偶数位置采用正弦函数编码，奇数位置采用余弦函数编码，然后将这个编码值和embedding相加，作为下一层的输入。

自注意力本身是置换等变（permutation equivariant）的，改变输入顺序，输出也只是相应地改变顺序，但每个位置的表示只由内容决定，不由位置决定。

而语言是有顺序的，"我爱你"和"你爱我"是完全不同的意思。因此，我们必须给模型注入位置信息。

如何注入位置信息?

Transformer 的做法是：给每个位置生成一个独特的位置向量，然后把这个向量加到词嵌入向量上。

$$\text{输入表示} = \text{词嵌入} + \text{位置编码}$$

这样，每个位置的向量就既包含了语义信息，又包含了位置信息。

(2) 正弦位置编码（Sinusoidal Positional Encoding)

这是原论文中使用的位置编码方式，也是最经典的一种。

结构图

```
位置 pos:    0      1      2      3      4      ...
             |      |      |      |      |
             ▼      ▼      ▼      ▼      ▼
          ┌──────────────────────────────────┐
          |   正弦/余弦函数生成位置向量      |
          └──────────────────┬───────────────┘
                             ▼
          每个位置对应一个 d_model 维的向量

维度示意（d_model=4）:
pos=0: [sin(0/10000^0), cos(0/10000^0), sin(0/10000^1), cos(0/10000^1)]
pos=1: [sin(1/10000^0), cos(1/10000^0), sin(1/10000^1), cos(1/10000^1)]
...
```

定义

正弦位置编码使用正弦和余弦函数的不同频率来为每个位置生成唯一的编码向量。

完整公式

对于位置 pos 和维度 i（偶数维度用 sin，奇数维度用 cos）：

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$$

其中：

- pos：位置索引（0, 1, 2, ..., n-1）
- i：维度索引（0, 1, 2, ..., d_model/2 - 1）
- d_{model}：模型维度
- 2i 和 2i+1 表示第 i 对维度（偶数维用 sin，奇数维用 cos）

公式解读

让我们把公式拆开来理解：

1. 波长（周期）：对于第 i 对维度，波长为：

$$\lambda_i = 2\pi \times 10000^{\frac{2i}{d_{model}}}$$

2. 维度 i 越小，波长越短，频率越高——编码精细的位置差异

3. 维度 i 越大，波长越长，频率越低——编码粗略的位置差异

这就像二进制表示，低位变化快，高位变化慢。不同维度上的不同频率组合起来，可以唯一标识每个位置。

(3) 为什么用正弦/余弦函数

它可以让模型很容易地学习到相对位置关系。

定理

对于任意固定的偏移量 k，PE(pos+k) 可以表示为 PE(pos) 的线性函数。

证明

我们需要证明：存在一个只依赖于 k 的线性变换矩阵 M_k，使得：

$$PE(pos + k) = M_k \cdot PE(pos)$$

让我们来看第 i 对维度（2i 和 2i+1）：

$$PE(pos, 2i) = \sin(\omega_i \cdot pos)$$

$$PE(pos, 2i+1) = \cos(\omega_i \cdot pos)$$

其中 $\omega_i = \frac{1}{10000^{2i/d_{model}}}$。

现在考虑 pos+k：

$$PE(pos + k, 2i) = \sin(\omega_i(pos + k)) = \sin(\omega_i pos + \omega_i k)$$

利用三角恒等式：

$$\sin(a + b) = \sin a \cos b + \cos a \sin b$$

$$\cos(a + b) = \cos a \cos b - \sin a \sin b$$

代入：

$$\sin(\omega_i pos + \omega_i k) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k)$$

$$\cos(\omega_i pos + \omega_i k) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k)$$

写成矩阵形式：

$$\begin{bmatrix}\sin(\omega_i(pos+k)) \\ \cos(\omega_i(pos+k))\end{bmatrix} = \begin{bmatrix}\cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k)\end{bmatrix} \begin{bmatrix}\sin(\omega_i pos) \\ \cos(\omega_i pos)\end{bmatrix}$$

左边就是 PE(pos+k) 的第 i 对维度，右边的矩阵只依赖于 k，不依赖于 pos。

这就证明了：对于每对维度，相对位置 k 的偏移可以通过一个线性变换来实现。 整个向量也是如此（各对维度独立）。

(4) 可学习位置编码（Learned Positional Embedding)

除了正弦位置编码，另一种常见的方式是**可学习的位置嵌入**。

定义

可学习位置嵌入就是直接为每个位置初始化一个向量，然后通过训练来学习这些向量的值。

实现方式

- 创建一个形状为 (max_len, d_model) 的可学习参数矩阵
- 对于位置 pos，直接取矩阵的第 pos 行作为位置编码

与正弦位置编码的对比

特性	正弦位置编码	可学习位置编码
是否可训练	否（固定的）	是（通过训练学习）
外推能力	好（可以处理训练时没见过的长度）	差（超出 max_len 就没有了）
表达能力	受限于正弦函数形式	更灵活，可以学习任意模式
参数量	0（无参数）	max_len × d_model
实现复杂度	稍复杂（需要计算 sin/cos）	简单（直接查表

(5) 后续演进：相对位置编码、RoPE、ALiBi 等

正弦位置编码和可学习位置编码都是绝对位置编码，它们编码的是每个位置的绝对位置。

但在很多场景下，相对位置更重要，"A 在 B 前面"比"A 在第 3 位、B 在第 5 位"更有意义。

后续研究者提出了多种相对位置编码方案：

1. 相对位置编码（Relative Position Embedding）：在注意力计算中直接考虑相对位置

2. RoPE（Rotary Position Embedding）：通过旋转来注入位置信息，LLaMA 等模型使用

3. ALiBi（Attention with Linear Biases）：给注意力分数加上线性偏置，简单有效

6.4.3 嵌入 + 位置编码的融合

相加方式与维度匹配

$$x = \text{Embedding}(token) + \text{PositionalEncoding}(pos)$$

为什么是相加而不是拼接?

原因有三：

1. 维度一致性：相加保持了 d_model 维度不变，后续层不需要调整维度

2. 参数效率：如果拼接，维度会变成 2×d_model，后续所有层的参数都要翻倍

3. 实验效果：相加在实践中效果很好——模型可以学会在向量的不同维度中分别编码语义和位置信息

维度匹配

词嵌入的维度是 d_model，位置编码的维度也是 d_model，两者可以直接相加。

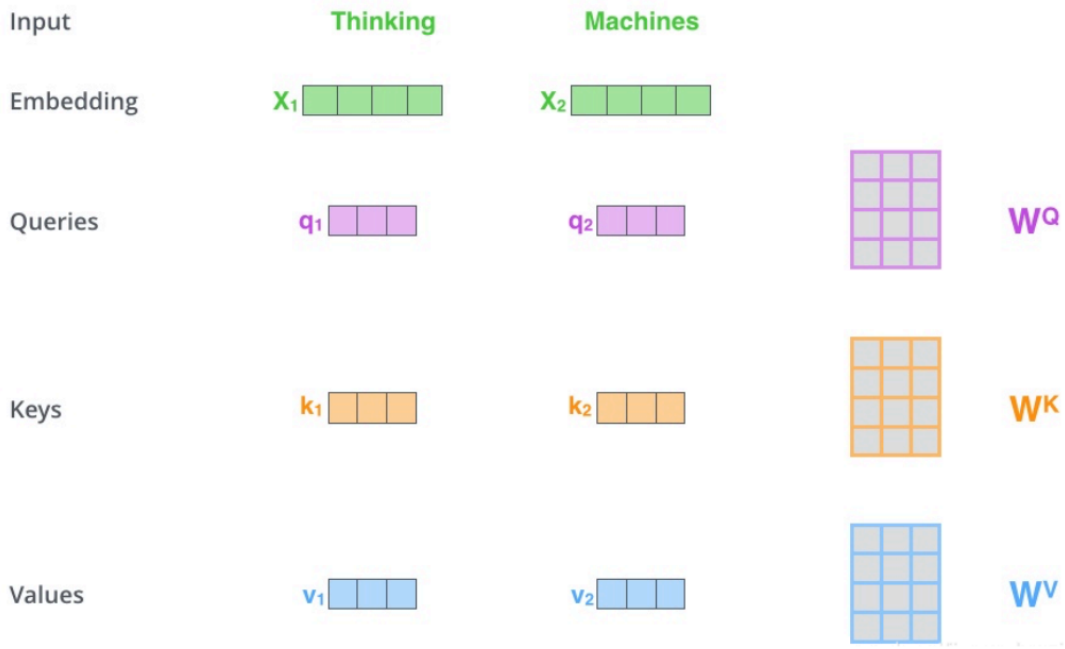
6.5 自注意力机制

如果说 Transformer 是一座大厦，那么自注意力就是这座大厦的地基和钢筋骨架。它是 Transformer 最核心、最本质的创新，也是理解整个架构的关键所在。

Self-Attention（自注意力），又称”内部注意力“（intra-attention），输入和输出是同一个序列，用于发现序列数据 内部的关联性及特征。

详细计算过程

(1) 每个词计算出3个不同的向量，它们分别是Query向量（Q），Key向量（K）和 Value向量（V），在论文中，向量是512维。这三个向量是用embedding向量与一个矩阵相乘得到的结果，这个矩阵是随机初始化的，维度为（64, 512）。 利用Query跟Key比较相似度，把相似度作为权重，再跟Value做Attention分数计算. 例如，有两个词 Thinking、Machines，计算过程如下所示：

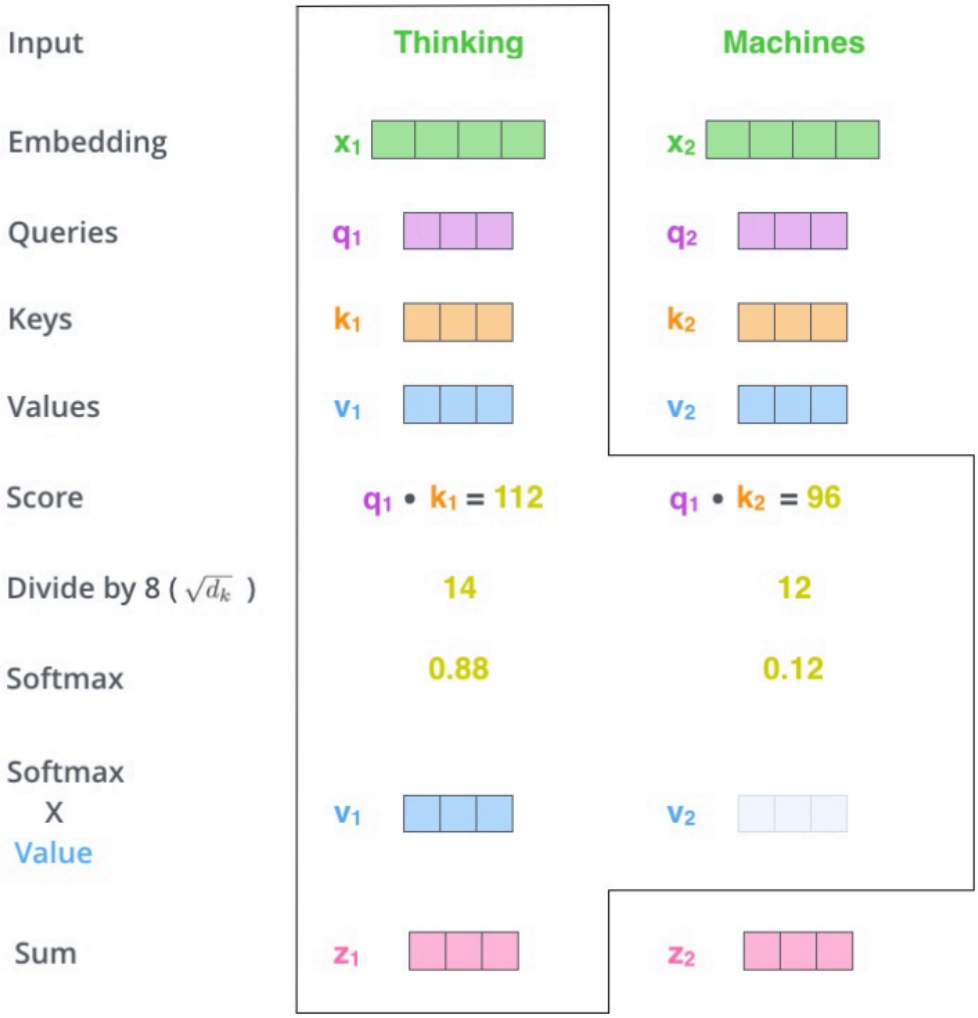


计算Q,K,V

(2) 计算Self-attention分数值，该分数值决定了我们在某个位置对一个词进行编码时，对句子其它部分的关注程度。

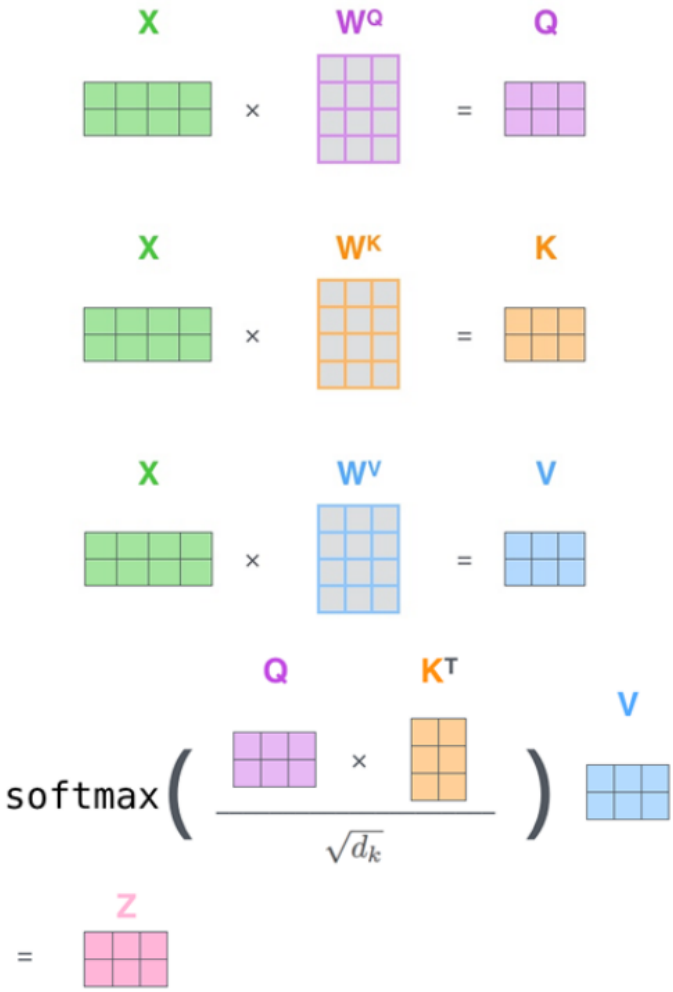
计算方法是：先用Query和Key做点乘， 是针对自己的点乘， 是针对第二个词的点乘，这个分值代表两个词语的相似度。

然后，把点乘结果除以一个值（下图除以8，这个值一般是采用上面提到的矩阵第一个维度的开方），对数值大小做缩放调整，然后做softmax（这个分数可以认为是每个词对当前词的贡献），并和Value相乘后相 加，得到的结果即是Self-attention在当前节点的值。



Self-attention的完整计算过程

在实际应用场景中，为了提高计算速度，采用矩阵方式，直接计算出Query, Key, Value的矩阵，然后把embedding的 值与三个矩阵相乘，得到的新矩阵Q与K相乘，乘以一个常数，接softmax操作，最后乘上V矩阵.如下图所示：



所以，表达式可以写成：

$$Attention(Q,K,V)=softmax(\frac{QK^T}{\sqrt{d_k}})V$$

6.6 Multi-Head Attention多头自注意力

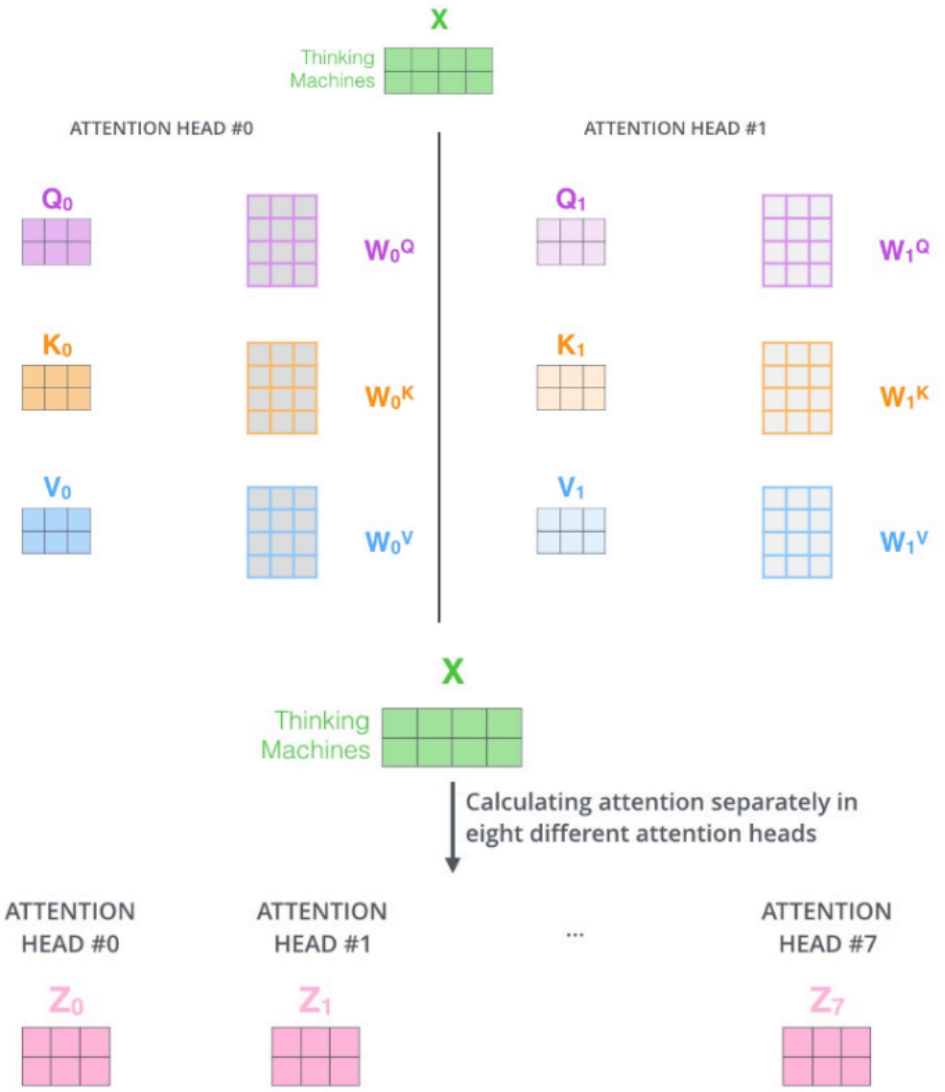
该模型还引入了Multi-Head Attention（多头注意力）机制。所谓“多头”，指有多组Q、K、V矩阵，Transformer模型中使用了8组，所以计算结果可以得到8组矩阵。

多头的核心思想：在不同子空间学习不同的注意力模式

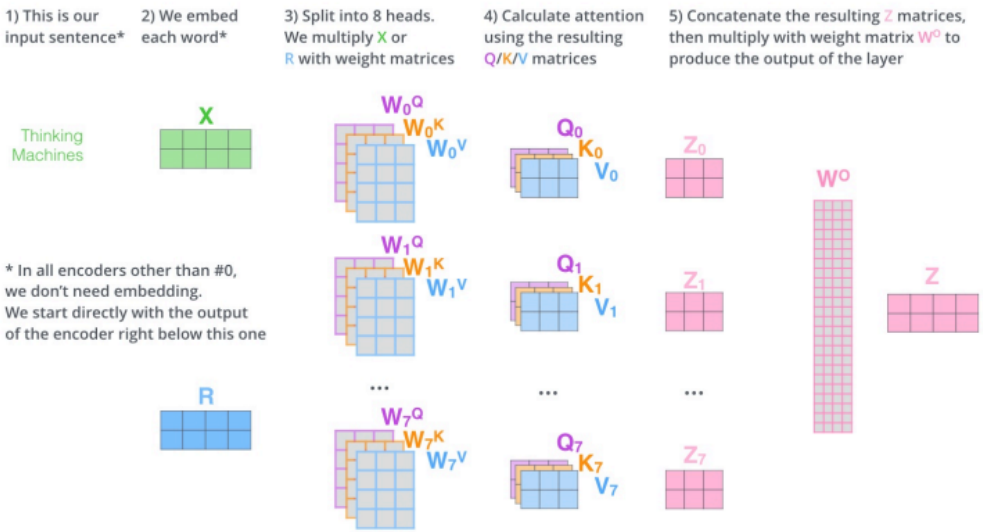
1. 将 Q、K、V 分别投影到 h 个不同的子空间
2. 在每个子空间中独立计算注意力
3. 将 h 个头的输出拼接起来
4. 通过一个线性投影得到最终输出

每个头的维度是 $d_k = d_{model}/h$，这样拼接后维度仍然是 d_{model}。

如下图所示：

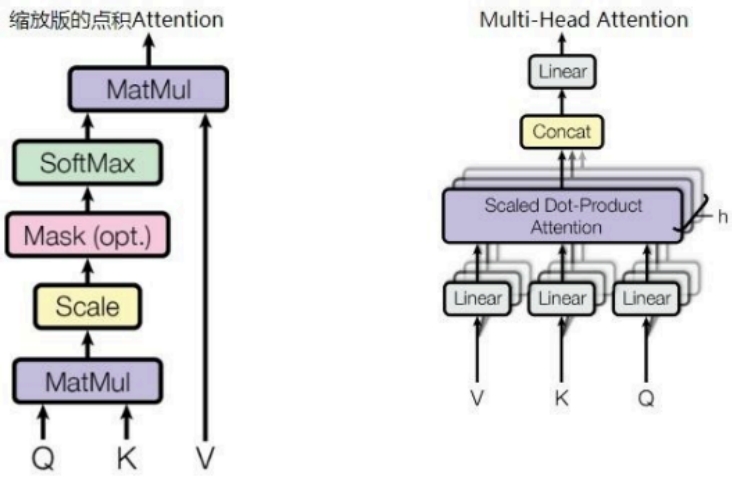


实际计算时，采用了矩阵合并的方式，一次性计算出最终的结果。



多头注意力整个过程可以简述为：Query, Key, Value首先经过一个线性变换，然后做缩放点积计算（做h次，即所谓 多头），然后将h次缩放点积计算结果进行拼接，再进行一次线性变换，得到的值作为注意力计算结果。

如下图所示：



(左)缩放点积运算. (右)多头注意力

各头实际学到的模式分析

多头注意力的各个头到底学到了什么？这是研究者们非常感兴趣的问题。

通过对训练好的模型进行分析，研究者发现不同的头确实学到了不同的模式：

语法头（Syntactic Heads）

- 关注语法结构关系
- 例如：动词关注它的主语或宾语
- 常见于模型的底层

语义头（Semantic Heads）

- 关注语义相似性
- 例如：同义词、相关词之间有较高的注意力
- 常见于模型的中层

位置头（Positional Heads）

- 关注位置关系
- 例如：每个位置主要关注它前面或后面的几个位置
- 类似于局部卷积

指代头（Coreference Heads）

- 关注指代关系
- 例如：代词（"它"、"他"）关注它所指代的名词

分隔符头（SEP Heads）

- 在 BERT 等模型中，有些头主要关注 <[BOS_never_used_51bce0c785ca2f68081bfa7d91973934]> 或 [SEP] 标记
- 可能用于汇聚全局信息

注意：这些模式不是绝对的，不同模型、不同层、不同头学到的模式都不同。而且很多头的模式很难用人类的语言来解释，这也是深度学习的"黑箱"问题。

6.7 注意力掩码（Mask）机制详解

掩码（Mask）是一种控制注意力可见范围的技术。

它通过在 softmax 之前将某些位置的注意力得分设为负无穷，使得这些位置的注意力权重变为 0。

核心思想

注意力机制默认是"全可见"的——每个位置可以看到所有位置。但在某些场景下，我们需要限制可见范围：**1. 填充位置（Padding）**：为了批处理，不同长度的序列会被填充到相同长度。填充的位置不应该被关注。

2. 因果约束（Causality）：在自回归生成中，当前位置不能看到未来位置的信息。

2. 其他约束：比如局部注意力、稀疏注意力等。

6.8 Padding Mask（填充掩码）

在实际训练中，我们通常以 batch（批次）的方式处理数据。同一个 batch 中的序列长度可能不同，为了能够组成矩阵进行并行计算，我们需要把短序列用特殊的 token（通常是 `<pad>`）填充到相同的长度

作用：忽略序列中的 padding 位置

例如：

```
序列 1: 我 爱 你 <pad> <pad>   （长度 5，有效长度 3）
序列 2: 今 天 天 气 不 错       （长度 6，有效长度 6）
```

问题：填充的 `<pad>` 位置没有实际意义，不应该参与注意力计算。如果模型关注了这些无意义的位置，会引入噪声。

解决方案：用 Padding Mask 让模型忽略这些填充位置。

核心思路: 在计算完注意力得分后、做 softmax 之前，将 padding 位置对应的得分设为一个非常小的数（如 -1e9 或 -∞）。这样，经过 softmax 后，这些位置的权重就会趋近于 0。

数学表达

$$\text{score}_{ij} = \begin{cases} \frac{q_i \cdot k_j}{\sqrt{d_k}} & \text{如果 j 不是 padding 位置} \\ -\infty & \text{如果 j 是 padding 位置} \end{cases}$$

$$\alpha_{ij} = \text{softmax}(\text{score}_i)_j$$

当 score = -∞ 时：

$$\text{softmax}(-\infty) = \frac{e^{-\infty}}{\sum e^s} = \frac{0}{\sum e^s} = 0$$

这样，padding 位置的注意力权重就是 0，不会对结果产生任何影响。

6.9 Look-ahead / Causal Mask（前瞻 / 因果掩码）

这是 Transformer 中另一种非常重要的掩码，也是 Decoder 自注意力的核心。

作用：防止当前位置看到未来位置的信息

因果性原则：输出第 t 个 token 时，只能依赖第 1 到 t 个 token，不能依赖 t 之后的 token。这就是因果性（Causality）。

解决方案：用 Look-ahead Mask（也叫 Causal Mask）遮住未来的位置。

6.10 Masked Multi-Head Attention（掩码多头注意力）

定义：应用了因果掩码的多头自注意力，Masked Multi-Head Attention 就是在多头自注意力的基础上加上因果掩码。

作用：

1.对目标序列进行自注意力计算: 让目标序列中的每个位置融合前面位置的信息

2. 保证因果性: 通过掩码确保每个位置只能看到自己和之前的位置

与 Encoder 自注意力的区别

特性	Encoder 自注意力	Decoder 掩码自注意力
是否有因果掩码	否	是
可见范围	所有位置互相可见	只能看到当前及之前的位置
作用	编码输入的上下文表示	自回归地生成目标序列

6.11 为什么 Transformer 效果好

(1) 长距离依赖的直接建模

自注意力让序列中任意两个位置可以直接交互，信息路径长度为 O(1)。这意味着：

- 长距离依赖和短距离依赖一样容易学习
- 梯度可以直接传播，不会消失
- 模型可以捕捉全局依赖关系

这是 Transformer 相比 RNN 最大的优势。

(2) 并行计算带来的可扩展性

自注意力完全并行，可以充分利用 GPU/TPU 的计算能力。这意味着：

- 训练速度快，可以用更多数据训练
- 模型可以做得很大（参数多），因为训练速度跟得上
- 数据规模和模型规模可以同时扩大

(3) 多头注意力的多模式建模能力

多头注意力让模型可以同时关注多种类型的关系：

- 语法关系（主谓、动宾）
- 语义关系（同义、上下位）
- 位置关系（相邻、远距离）
- 指代关系（代词-名词）

每个头专注于一种模式，最后汇总——这比单一的相似度度量强大得多。

(4) 残差连接带来的训练深度

残差连接让 Transformer 可以堆叠很多层而不退化。深层网络可以学习更抽象、更高级的特征表示。

- 底层：低级特征（词汇、语法）
- 中层：中级特征（短语、句子结构）
- 顶层：高级特征（语义、语用）

层次化的特征表示是深度学习成功的关键，而残差连接是训练深层网络的基础。

6.12 Transformer 的局限性

尽管 Transformer 非常强大，但它也不是完美的。它有几个显著的局限性。

(1) 二次复杂度（$O(n^2)$）与长序列瓶颈

问题

自注意力的计算量和内存占用都是 $O(n^2)$——和序列长度的平方成正比。

这对于长文档、基因组序列、高分辨率图像等长序列任务来说是不可接受的。

解决方案

针对这个问题，研究者提出了很多高效 Transformer：

- 稀疏注意力（Longformer、BigBird）：只关注部分位置
- 线性注意力（Linformer、Performer）：用低秩近似或核方法将复杂度降为 $O(n)$
- 状态空间模型（Mamba 等）：用结构化的状态空间代替注意力

(2) 位置编码的局限

问题

自注意力本身没有位置信息，需要额外的位置编码来注入。但现有的位置编码方案都有各自的问题：

- **正弦位置编码**：固定的，不能学习任务相关的位置模式
- **可学习位置编码**：不能外推到训练时没见过的长度
- **相对位置编码**：实现复杂，计算开销大

(3) 数据 hungry（需要大量数据）

问题

由于自注意力的归纳偏置很弱，Transformer 需要大量的训练数据才能学好。

- 小数据场景下，Transformer 可能不如 RNN/CNN
- 低资源语言、小样本任务上，Transformer 的优势不明显

解决方案

- 预训练 + 微调：先用大规模无标注数据预训练，再在小数据上微调
- 数据增强：回译、同义词替换等
- 更强的归纳偏置：在架构中引入更多先验知识

(4) 推理效率问题（自回归生成慢）

问题

Transformer 的 Decoder 是自回归的——生成一个长度为 m 的序列需要 m 次前向传播，每次的复杂度是 $O(m^2 \cdot d)$。

总复杂度：$O(m^3 \cdot d)$

这使得 Transformer 的推理速度比训练慢得多——特别是生成长文本时。

对比

- 训练：一次前向传播，并行计算所有位置 → $O(n^2 \cdot d)$
- 推理：n 次前向传播，每次计算量递增 → $O(n^3 \cdot d)$

解决方案

- **KV Cache**：缓存之前的 K 和 V，避免重复计算
- **量化**：用更低精度的计算（INT8、INT4）
- **speculative decoding**：用小模型"猜"，大模型验证

6.13 Transformer 家族与后续演进

原始的 Transformer 架构只是一个开始。在它之后，研究者们基于 Transformer 进行了大量的改进和扩展，形成了一个庞大的 Transformer 家族。

三大架构分支

原始 Transformer 是 Encoder-Decoder 架构。但后来的研究发现，根据任务类型的不同，可以只使用 Encoder 或只使用 Decoder，也能取得很好的效果。

由此形成了 Transformer 的三大架构分支。

(1) Encoder-Only（BERT 系列）：理解类任务

代表模型：BERT、RoBERTa、ALBERT、ELECTRA、DeBERTa 等

架构特点：

- 只使用 Transformer 的 Encoder 部分
- 双向注意力——每个位置可以看到所有位置
- 适合需要"理解"的任务

预训练任务：

- **MLM（Masked Language Modeling）**：随机 mask 掉一些 token，让模型预测被 mask 的 token
- 这是一个双向任务——预测需要看左右两边的上下文

适用任务：

- 文本分类（情感分析、主题分类）
- 命名实体识别（NER）
- 问答系统（抽取式 QA）
- 句子对任务（自然语言推理、相似度计算）
- 各种需要"理解"文本的任务

(2) Decoder-Only（GPT 系列）：生成类任务

代表模型：GPT-1、GPT-2、GPT-3、GPT-4、LLaMA、Qwen、DeepSeek 等

架构特点：

- 只使用 Transformer 的 Decoder 部分
- 单向（因果）注意力——每个位置只能看到自己和之前的位置
- 适合需要"生成"的任务

预训练任务：

- **自回归语言建模**：给定前面的 token，预测下一个 token
- 这是一个单向任务——只能用左边的上下文预测右边

适用任务：

- 文本生成（故事、文章、代码）
- 对话系统（ChatGPT 类）
- 摘要生成
- 翻译
- 各种需要"生成"文本的任务

(3) Encoder-Decoder（T5 / BART 系列）：转换类任务

代表模型：T5、BART、mBART、PEGASUS 等

架构特点：

- 完整的 Encoder-Decoder 架构
- Encoder 做双向理解，Decoder 做自回归生成
- 适合"输入到输出"的转换任务

预训练任务：

- **Span Corruption**（T5）：随机 mask 掉一段文本，让模型还原
- **Denoising**（BART）：各种噪声 + 还原
- 本质是"破坏-重建"的模式

适用任务：

- 机器翻译
- 文本摘要
- 文本改写
- 问答（生成式）
- 各种"输入序列 → 输出序列"的转换任务

三大分支对比

特性	Encoder-Only	Decoder-Only	Encoder-Decoder
代表模型	BERT	GPT、LLaMA	T5、BART
注意力类型	双向自注意力	单向（因果）自注意力	Encoder 双向 + Decoder 单向 + 交叉注意力
预训练任务	MLM（掩码语言建模）	自回归语言建模	序列到序列的去噪
核心能力	理解	生成	转换
典型任务	分类、NER、QA	生成、对话	翻译、摘要
参数量效率	高（只有 Encoder）	高（只有 Decoder）	较低（两部分都有）
大模型趋势	较少（小模型为主）	主流（GPT 系列）	中等（特定任务）

7. BERT

7.1 预训练语言模型进化史

7.1.1 词向量革命（Word2Vec 2013）

在 Word2Vec 之前，NLP 中表示词的方式是"one-hot 编码"，每个词是一个 V 维向量，只有自己那个位置是 1，其余都是 0。这种表示的问题显而易见：

- 维度爆炸（词表有多大，向量就有多长）
- 词之间没有任何语义关系（"猫"和"狗"的向量距离和"猫"和"桌子"一样远）

Word2Vec 用一个简单的想法彻底改变了这一切：通过预测上下文来学习词的稠密向量表示。有两种方式：

- **CBOW**：根据上下文预测中心词（"我__吃苹果"→"喜欢"）
- **Skip-gram**：根据中心词预测上下文（"喜欢"→"我""吃""苹果"）

Word2Vec 在于它发现了"向量空间中的语义关系"：

$$v(\text{国王}) - v(\text{男人}) + v(\text{女人}) \approx v(\text{女王})$$

7.1.2 上下文相关的词向量（ELMo 2018.02）

ELMo（Embeddings from Language Models）的核心想法是：词向量不应该是静态的，而应该根据上下文动态生成。

ELMo 用了一个双向 LSTM 来做语言模型预训练：

- 前向 LSTM：根据前面的词预测下一个词
- 后向 LSTM：根据后面的词预测前一个词
- 把两个方向的隐藏状态拼接起来，作为词的上下文相关表示

这样，"苹果"在"我吃苹果"和"苹果手机"中就有了不同的向量。

7.1.3 生成式预训练（GPT-1 2018.06）

GPT-1 由 OpenAI 于 2018 年 6 月发布，比 BERT 早 4 个月。它做了两个关键改进：

1. 用 Transformer Decoder 替代 LSTM：特征提取能力更强，并行化更好
2. 端到端微调：不是把预训练模型当"特征提取器"，而是在下游任务上微调整个模型

GPT-1 的预训练任务是自回归语言模型：根据上文预测下一个词。这是一个"单向"的任务，每个词只能看到左边的内容。

GPT-1 的局限：

- 单向注意力，在理解任务上有天然缺陷
- 预训练数据量相对小（只有 BookCorpus，约 4.5GB）
- 模型规模小（117M 参数）

7.1.4 双向预训练（BERT 2018.10）

BERT 的直接技术前身是 ELMo（双向上下文建模），同时借鉴了 GPT-1 的"预训练 + 端到端微调"范式。BERT 的核心创新是：把 ELMo 的双向思想和 Transformer 的强大特征提取能力结合起来。

BERT 之后，预训练语言模型成为 NLP 的标配，"预训练 + 微调"成为新的范式，"小作坊时代"（每个任务从零训练一个小模型）彻底结束，进入"一超多强"（一个大模型微调适配所有任务）的时代。

7.2 BERT 定义

BERT（Bidirectional Encoder Representations from Transformers）是 Google 于 2018 年 10 月发布的预训练语言模型。它基于 Transformer Encoder 架构，通过掩码语言模型（MLM）任务进行双向预训练，然后在下游任务上端到端微调。

核心贡献：首次证明"双向预训练 + 大规模数据 + 端到端微调"的组合可以在 11 项 NLP 任务上全面超越当时的 SOTA，标志着 NLP 进入"预训练+微调"的统一范式时代。

设计哲学

(1) 双向优于单向: 语言理解需要同时看左右上下文：

"这部电影**不**好看"——必须同时看到"不"和"好看"才能判断负面

GPT-1 的单向注意力在处理这些情况时，当前词只能看到左边，看不到右边，理解必然不充分。

BERT 的双向注意力让每个词都能同时看到左右，理解更全面。

(2) 预训练任务要和微调任务"对齐"

预训练的目标是学习通用的语言表示，然后在下游任务上微调。如果预训练任务和微调任务差异太大，迁移效果就会打折扣。

BERT 的 MLM 任务和下游的"理解型"任务高度对齐：

- 分类任务：需要理解整句语义 → MLM 需要理解上下文来预测掩码词
- 序列标注：需要理解每个词的上下文 → MLM 正是在做这件事
- 问答：需要理解问题和段落的关系 → MLM 的跨句子上下文建模有帮助

而 GPT-1 的自回归任务和"理解型"任务对齐度较低，自回归是"生成"，不是"理解"。

(3) 大规模

BERT 的成功不仅来自架构，更来自规模：

- 训练数据：BookCorpus（8 亿词）+ 英文维基百科（25 亿词）= 33 亿词，是 GPT-1 的 4 倍

- 模型规模：BERT-large 有 3.4 亿参数，是 GPT-1 的 3 倍
- 训练时长：BERT-base 在 4 块 Cloud TPU 上训练了 4 天，large 训练了 4 天（16 块 TPU）

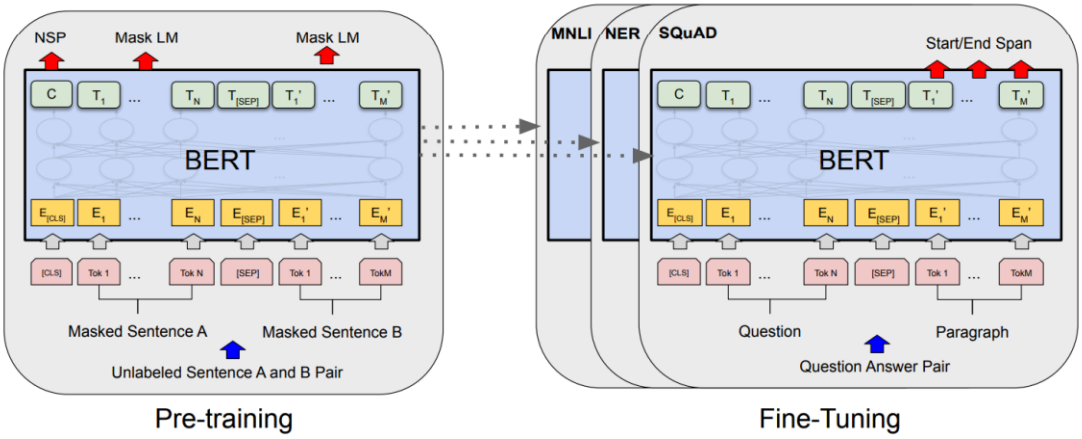
BERT 用实验证明了：在双向架构的基础上，更大的模型 + 更多的数据 = 更好的效果。这为后来 GPT-3 的"缩放定律"埋下了伏笔。

7.3 BERT 网络结构

BERT 的架构非常"朴素"，它就是一个堆叠了 N 层的 Transformer Encoder。每一层由"双向多头自注意力 + 前馈网络 + 残差连接 + Post-LN"四个组件串联而成。核心是双向自注意力，每个位置可以同时看到左右所有位置的信息，这是 BERT 理解能力的来源。

BERT 有两个标准配置：

参数	BERT-base	BERT-large	说明
层数 L	12	24	Transformer Encoder 的层数
隐藏维度 H	768	1024	每个 token 的向量维度
注意力头数 A	12	16	多头注意力的头数
FFN 维度	3072	4096	通常是 H 的 4 倍
参数量	110M	340M	large 约是 base 的 3 倍
最大序列长度	512	512	位置编码的最大长度



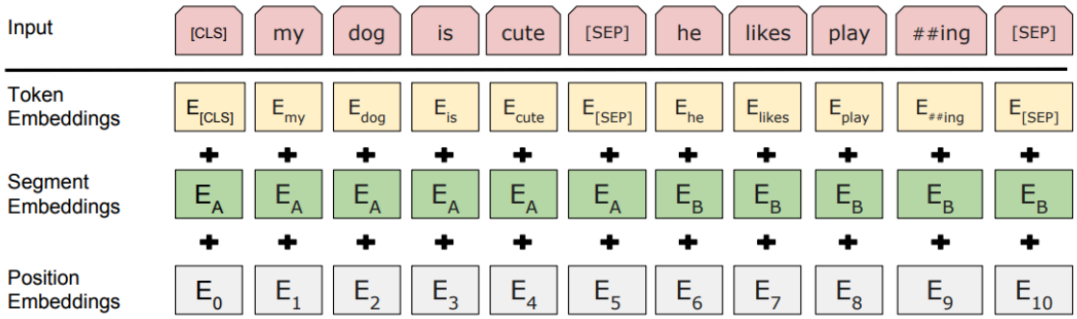
BERT预训练模型(左)和微调程序(右)

7.3.1 输入嵌入层

> 预训练使用无标注数据，让模型学习通用语言表示。

BERT 的输入嵌入由三部分相加而成：

$$\text{emb}_i = \text{TokenEmb}(w_i) + \text{SegmentEmb}(s_i) + \text{PositionEmb}(p_i)$$



BERT输入表示. 输入为词嵌入、分段嵌入和位置嵌入的总和

Token Embedding（词嵌入）

用 WordPiece 分词器把文本切成子词（subword），然后查表得到词嵌入。词表大小为 30522。

WordPiece 把词切成子词，比如 "playing" → "play" + "##ing"，这样词表可以控制在 3 万左右，同时能覆盖任意文本。

Segment Embedding（段落嵌入）

只有两种取值：句子 A 的所有 token 加同一个向量，句子 B 的所有 token 加另一个向量。这是为了支持句子对任务（如问答、自然语言推理）。

Position Embedding（位置嵌入）

可学习的位置编码，最大长度 512。

BERT 没有用原始 Transformer 的正弦编码，而是用了可学习编码。实验表明，在 512 的长度范围内，可学习编码和正弦编码效果差不多，但可学习编码更简单。

7.3.2 双向多头自注意力

(1) 双向 vs 单向：注意力矩阵的对比

BERT 的双向自注意力，注意力矩阵是完整矩阵，每个位置可以关注所有位置：

```
        我    爱    编    程
我      ●     ●     ●     ●     ← 左:无  右:爱编程  全可见
爱      ●     ●     ●     ●     ← 左:我  右:编程  全可见
编      ●     ●     ●     ●     ← 左:我爱  右:程  全可见
程      ●     ●     ●     ●     ← 左:我爱编  右:无  全可见
```

GPT 的单向自注意力，注意力矩阵是下三角矩阵（因果掩码），每个位置只能看到自己和左边：

```
        我    爱    编    程
我      ●     ✗     ✗     ✗     ← 左:无  右:爱编程  全被遮  看不到
爱      ●     ●     ✗     ✗     ← 左:我  右:编程  被遮  看不到
编      ●     ●     ●     ✗     ← 左:我爱  右:程  被遮  看不到
程      ●     ●     ●     ●     ← 左:我爱编  右:无
```

双向自注意力的计算流程

与标准缩放点积注意力一致，但不加因果掩码：

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

注意：Softmax 的输入 $QK^\top/\sqrt{d_k}$ 是完整的 $n \times n$ 矩阵，没有任何位置被设为 $-\infty$。

多头双向自注意力

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

每个头都是双向的，在不同的表示子空间里关注不同的关系（语法/语义/指代等）。

双向自注意力的代价

优势	代价
完整上下文，理解能力强	不能直接用于生成（会看到"答案"）
每个位置信息最充分	预训练必须用 MLM（不能用自回归）
参数全部投入理解	生成任务需额外设计解码机制

7.3.3 前馈网络（FFN）

注意力层负责"序列内信息交互"，FFN 负责"每个位置的特征变换"：

$$\mathrm{FFN}(x) = W_2 \cdot \mathrm{GELU}(W_1 x + b_1) + b_2$$

BERT 用 GELU 激活（不是 ReLU），FFN 中间维度为 $4H$（base: 3072, large: 4096）。

7.3.4 残差连接与 Post-LN

每个子层（注意力/FFN）都有残差连接和 Post-LN：

$$x_{\text{out}} = x + \mathrm{LayerNorm}(\mathrm{Sublayer}(x))$$

BERT 用 **Post-LN**（LN 在子层之后），与 GPT/T5 的 Pre-LN 不同。

7.3.5 输出表示与特殊标记

经过 N 层 Encoder 后，每个位置得到一个 H 维的上下文相关表示。

(1) <[BOS]> 标记：插入在序列最前面，其最终隐藏状态被视为整句的"聚合表示"，用于分类任务。

(2) [SEP] 标记：分隔两个句子。

单句任务格式 `<[BOS> 句子 [SEP]`，

句子对任务格式 `<[BOS]> 句子A [SEP] 句子B [SEP]`。

(3) 注意力掩码：batch 内句子长度不同，短句子后面 padding 填充。在注意力分数上把 padding 位置设为 $-\infty$，Softmax 后权重为 0，padding 不参与计算。

7.3.6 如何协作?

```
一个 token 在 BERT 中的"旅程"（以"爱"为例）：文本是: 我爱编程

Step 1: 嵌入层
  "爱" → TokenEmb("爱") + SegmentEmb + PositionEmb(位置2)
  → 一个 768 维向量，包含"词是什么+属于哪个句子+在什么位置"

Step 2: 第 1 层双向自注意力
  "爱"作为 Query，去和 <[BOS]>/我/爱/编/程/[SEP] 所有 Key 算相似度
  → 加权聚合所有位置的 Value
  → "爱"的新表示融合了"我喜欢编程"的完整上下文

Step 3: 第 1 层 FFN
  对"爱"的新表示做非线性变换，提取更高级的特征

Step 4: 重复 Step 2-3 共 12 次（base）
  底层关注局部/语法（"爱"和"我"的主谓关系）
  高层关注语义/全局（整句的情感倾向）

Step 5: 输出
  "爱"的最终表示 = 融合了完整双向上下文的 768 维向量
  <[BOS]> 的最终表示 = 整句的聚合表示
```

7.4 掩码语言模型（MLM）

7.4.1 MLM 的定义：

随机选择输入序列中 15% 的 token，用特殊标记 `[MASK]` 替换（或随机替换/保持不变），然后让模型根据双向上下文预测这些被遮住的原始词。
MLM 是 BERT 的灵魂。

7.4.2 80/10/10 策略

MLM 有一个根本矛盾：预训练时输入中有 `[MASK]`，但微调时输入中从来没有 `[MASK]`，这导致预训练和微调的分布不匹配。

比例	处理方式	目的
80%	替换为 `[MASK]`	主要训练信号，学会"根据上下文预测缺失词"
10%	替换为随机词	强迫模型"纠错"——看到不合理的词时，根据上下文判断应该是什么
10%	保持原词不变	让模型知道"有时候这个位置就是对的，不需要改"，平衡前两种情况

7.4.3 损失函数

MLM 的损失只计算被掩码位置(被掩码的 15% 位置)的交叉熵：

$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|M|}\sum_{i \in M} \log P(w_i | w_{\setminus M})$$

其中：

- M 是被掩码的位置集合
- $|M|$ 是被掩码的 token 数
- w_i 是位置 i 的原始词
- $w_{\setminus M}$ 是除掩码位置外的所有上下文
- θ 是模型参数

具体的预测概率：

$$P(w_i \mid w_{\setminus M}; \theta) = \text{softmax}(h_i W^T + b)_{w_i}$$

其中 h_i 是 Encoder 在位置 i 的输出向量，W 是输出投影矩阵（通常和词嵌入矩阵共享权重，称为"权重绑定"），减少参数量，词嵌入和输出投影在同一个空间，语义更一致。

7.4.4 MLM 的问题

1. 训练效率低：只有 15% 的 token 参与损失计算，每个样本的训练信号密度低

2. 掩码位置独立性假设：被掩码的位置之间相互独立预测，没有建模掩码词之间的依赖关系

3. 预训练-微调差异：虽然 80/10/10 缓解了，但 `[MASK]` 仍然只在预训练中出现

7.5 下一句预测（NSP）

NSP 是 BERT 的第二个预训练任务。很多下游任务（如问答、自然语言推理）需要理解两个句子之间的关系，而 MLM 只建模句子内部的词级关系。

NSP 的做法：

- 给模型两个句子 A 和 B
- 50% 的概率 B 是 A 的真实下一句（正例，标签 IsNext）
- 50% 的概率 B 是语料中随机抽取的句子（负例，标签 NotNext）
- 用 <[BOS]> 的输出接一个二分类器来预测

但是后来 RoBERTa 的实验发现，去掉 NSP 后，模型在大多数任务上的表现不降反升。

原因一：负例质量太低

NSP 的负例是从语料中随机抽取的句子。这导致正例和负例的"主题"差异巨大：

- 正例：A 是"我喜欢编程"，B 是"我每天写代码"（同一主题）
- 负例：A 是"我喜欢编程"，B 是"今天天气很好"（完全不同主题）

原因二：MLM 已经能学到句子间关系

在 BERT 的预训练中，输入经常包含两个句子（用 [SEP] 分隔），MLM 的掩码可能出现在两个句子中。

模型在预测句子 B 中的掩码词时，自然会利用句子 A 的上下文。所以 MLM 已经隐式地学到了句子间关系，NSP 提供的额外信号有限。

原因三：NSP 引入了噪声

低质量的负例不仅没有帮助，反而引入了训练噪声，模型为了区分"主题是否一致"这种简单特征，可能会"浪费"模型容量，影响更精细的语言理解。

7.6 ALBERT SOP（句子顺序预测）

ALBERT 没有完全抛弃句子级任务，而是用 SOP（Sentence Order Prediction） 替代了 NSP。

SOP 的做法：

- 正例：A 然后 B（正确顺序）
- 负例：B 然后 A（交换顺序）

SOP 的负例和正例来自同一段落，主题完全一致，只是顺序不同。

模型必须理解"句子间的逻辑顺序"才能区分，而不是简单判断"主题是否一致"。

7.7 BERT 微调

BERT 的预训练学习了通用的语言表示，微调则是把这些表示"适配"到具体任务上。

7.7.1 "灾难性遗忘"问题

微调有一个潜在问题：在下游任务上微调后，模型可能"忘记"预训练学到的通用知识，这叫"灾难性遗忘"（Catastrophic Forgetting）。

BERT 的微调通常不会严重遗忘，因为：

- 微调学习率很小，参数变化不大
- 下游任务和预训练任务都是"语言理解"，有共性
- 微调数据量通常不大，训练轮数少（2-4 轮）

但如果微调学习率太大、训练轮数太多，确实会出现遗忘。

7.7.2 微调

(1) 句子级分类

取 <[BOS]> 的输出，接一个线性分类层。适用于情感分析、主题分类等。损失函数：交叉熵。

(2) 序列标注

每个 token 的输出都接一个分类器。适用于 NER、词性标注等。

$$\hat{y}_i = \text{softmax}(W_t \cdot h_i + b_t)$$

注意：如果用了 WordPiece 分词，一个词可能被切成多个子词，通常只取第一个子词的输出做预测，或者对所有子词的输出做池化。

(3) 抽取式问答

给定问题和段落，从段落中抽取答案片段。预测两个分数：每个 token 作为答案起点的概率和作为终点的概率。

$$P_{\text{start}}(i) = \text{softmax}(W_s \cdot h_i + b_s)_i$$

$$P_{\text{end}}(i) = \text{softmax}(W_e \cdot h_i + b_e)_i$$

推理时，选择 start ≤ end 且 P_start(start) × P_end(end) 最大的片段作为答案。

(4) 句子对分类

输入 `<[BOS]> 句子A [SEP] 句子B [SEP]`，取 <[BOS> 输出做分类。适用于自然语言推理、句子相似度等。

1.6.4 微调的超参选择

BERT 微调对超参比较敏感，常用配置：

- 学习率：2e-5, 3e-5, 5e-5（需要网格搜索）
- Batch size：16, 32
- 训练轮数：2, 3, 4
- 最大序列长度：128, 256, 512
- 权重衰减：0.01
- 学习率调度：线性预热 + 线性衰减

一个经验法则：**学习率和 batch size 要一起调。**大 batch 可以用稍大的学习率，小 batch 要用小学习率。

7.8 BERT 的演进家族

7.8.1 RoBERTa

核心发现：BERT 的架构没问题，但训练策略太保守。

RoBERTa 做了四个改动：

1. **去掉 NSP**：如前所述，NSP 有害无益
2. **更大的 batch**：从 256 增加到 8000。大 batch 训练更稳定，梯度噪声更小
3. **更多的数据**：从 16GB 增加到 160GB（增加了 CC-News、OpenWebText 等）

4. **更长的训练**：从 100K 步增加到 500K 步

结果：在相同架构下，RoBERTa 在 GLUE 上比 BERT 提升了约 2-3%。

7.8.2 ALBERT：

核心问题：BERT-large 有 3.4 亿参数，训练成本高，而且很多参数可能是"冗余"的。

ALBERT 用两个技巧大幅降参：

(1) 因子化词嵌入（Factorized Embedding)

BERT 的词嵌入矩阵是 V×H（30522×768 = 2340 万参数）。

ALBERT 把它分解为 V×E + E×H，其中 E 远小于 H（如 E=128）。

$$\text{emb}(w) = \text{LayerNorm}(E_w \cdot V_w) \quad \text{其中} \quad E_w \in \mathbb{R}^{E\times V}, V_w \in \mathbb{R}^{H\times E}$$

参数从 V×H 降到 V×E + E×H。当 V=30522, H=1024, E=128 时，参数从 3125 万降到 390 万 + 131 万 = 521 万，减少了约 83%。

(2) 跨层参数共享（Cross-Layer Parameter Sharing)

BERT 的 12/24 层各有各的参数。ALBERT 让所有层共享同一组参数——注意力层共享，FFN 层共享。

这样，不管堆多少层，参数量都和一层一样。ALBERT-xxlarge 堆了 12 层（实际是重复共享 12 次），但参数量只有 233M，比 BERT-large（340M）还少。

7.8.3 ELECTRA：MLM 效率太低

核心问题：MLM 每次只有 15% 的 token 参与损失，训练效率低。

ELECTRA 的解决方案：**生成器-判别器架构。**

```
输入：我 爱 编 程
          |
     ┌────▼────┐
     | 生成器   |  ← 小的 MLM 模型，预测被掩码的词
     └─────┬───┘
          | 把掩码位置替换为生成器的预测（可能是错的）
          ▼
损坏输入：我 爱 写 程    (生成器把"编"预测成了"写")
          |
```

```
        ┌──────▼──────┐
        │  判别器     │  ← 判断每个 token 是"原始的"还是"被替换的"
        └─────────────┘
              │
        输出: 原始  原始  替换  原始
```

生成器是一个小的 MLM 模型（通常是判别器的 1/4 大小），负责把掩码位置替换为"合理的假词"。

判别器则判断每个 token 是真还是假。

7.8.4 DeBERTa：位置信息建模不够好

核心问题：BERT 把位置编码和内容编码直接相加，丢失了"内容"和"位置"之间的细粒度交互。

DeBERTa 的改进：**解耦注意力（Disentangled Attention）**。

在标准注意力中，Q 和 K 都只包含内容信息（位置编码是加在输入上的，和内容混在一起）。

DeBERTa 把 Q 和 K 都拆成"内容向量"和"位置向量"两部分，注意力分数同时考虑四种交互：

$$\text{scores} = Q_c K_c^T + Q_c K_p^T + Q_p K_c^T$$

其中 Q_c是内容 Query，Q_p 是位置 Query，K_c 是内容 Key，K_p 是位置 Key。

7.9 BERT 的局限性

不擅长生成: BERT 是 Encoder-Only，没有自回归机制，不能直接生成文本。虽然可以用"掩码填充"的方式做一些生成（如把要生成的位置全部掩码，然后逐个预测），但效果远不如 GPT/T5。

长度限制: BERT 的最大序列长度是 512（由可学习位置编码决定）。处理长文档时需要截断或分段，丢失信息。后续的 Longformer、BigBird 等模型用"稀疏注意力"来扩展长度，但那已经不是标准 BERT 了。

预训练-微调分布不匹配: MLM 的 `[MASK]` 在微调时不存在，80/10/10 只是缓解而非根治。

对知识的"记忆"能力有限: BERT 的参数量相对小（110M/340M），能记住的世界知识有限。在需要事实性知识的问答任务上，BERT 不如大模型。

8. GPT

GPT（Generative Pre-traning Transformer）是Open AI公司提出的一种语言模型，它的核心是一个Transformer的解码器。

在GPT诞生之前，传统NLP模型大量使用有监督学习方法进行训练, 这种有监督学习任务存在明显的缺点： （1）需要大量人工标注数据，耗时且昂贵；

（2）泛化能力和迁移性差

GPT提出了一种两阶段半监督训练方法，先通过大规模未标注数据训练一个无监督语言模型，再根据下游任务进行有监督微调.

8.1 诞生背景

2018 年 6 月，OpenAI 发布了 GPT-1。在当时的语境下，这个选择有些"反直觉"：

- 机器翻译的 SOTA 是 Encoder-Decoder 架构（原始 Transformer）
- 语言理解任务正在被双向模型主导（ELMo 已经证明了双向的价值）
- 为什么 OpenAI 偏偏选了"只有 Decoder"这种"看起来最弱"的架构？

答案藏在 OpenAI 的愿景里：他们要做的不是"更好的理解模型"，而是"通用的生成模型"。

8.2 自回归语言建模优势

优势一：预训练和生成完全一致，没有分布不匹配

BERT 的 MLM 有一个根本问题：预训练时输入有 `[MASK]`，微调/推理时没有。这导致预训练和实际使用之间存在"分布鸿沟"。

GPT 的自回归语言模型则完全没有这个问题：

- 预训练时：给定前 t-1 个词，预测第 t 个词
- 推理生成时：给定前 t-1 个词，采样第 t 个词

两者的计算方式完全一致，预训练学到的能力可以直接用于生成。这就像"平时练习就是考试内容"，而不是"平时练填空，考试考写作"。

优势二：100% 的 token 都有训练信号

MLM 只有 15% 的 token 参与损失计算，而自回归语言模型的每个位置都在预测下一个词，100% 的 token 都产生梯度。同样的计算量，GPT 的训练信号密度是 BERT 的约 6.7 倍。

优势三：天然支持生成，不需要额外解码器

BERT 要做生成需要额外加一个 Decoder（如 BART），而 GPT 本身就是一个生成器。对话、写作、代码生成、翻译……所有生成任务都可以用同一个框架搞定。

8.3 GPT演进路线

```
GPT-1 (2018): 预训练+微调 → 证明范式可行
    | 问题: 每个任务都要微调，不够通用
    ▼
GPT-2 (2019): 零样本 → 不微调也能干活
    | 问题: 零样本效果不够好，需要示例引导
    ▼
GPT-3 (2020): 少样本/ICL → 给几个示例就能做任务
    | 问题: 不"听话"，生成质量不稳定
    ▼
InstructGPT (2022): RLHF → 让模型遵循人类指令
    | 问题: 只有文本，推理能力有限
    ▼
GPT-4 (2023): 多模态+推理 → 接近人类水平
    | 问题: 交互方式有限，延迟高
    ▼
GPT-4o (2024): 实时多模态 → 全能交互
```

> 每一代都在解决上一代的"体验瓶颈"，从"能不能用"到"好不好用"到"像不像人"。

8.4 GPT-1（2018年6月）

GPT-1 的论文标题是《Improving Language Understanding by Generative Pre-Training》（通过生成式预训练改进语言理解）。

核心思想一句话：先用无标注数据预训练一个生成式语言模型，再在有标注数据上做有监督微调，用两阶段范式解决 NLP 任务。

在 GPT-1 之前，NLP 任务大多是从零开始训练每个任务的模型，或者用词向量（Word2Vec、GloVe）作为特征。

GPT-1 第一次系统地证明了："预训练 + 微调"的范式在 NLP 领域同样有效，而且效果远超之前的方法。

8.4.1 无监督预训练

目标函数：标准的自回归语言建模，最大化给定前文条件下下一个 token 的似然。

$$\mathcal{L}_{\text{pretrain}} = \sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta)$$

其中 k 是上下文窗口大小，Θ 是模型参数，P是条件概率，模型使用SGD进行优化。

训练过程：

- 在大规模无标注文本上，用滑动窗口截取长度为 512 的序列
- 每个序列输入模型，模型预测每个位置的下一个 token
- 用交叉熵损失计算所有位置的预测误差
- 用 Adam 优化器反向传播更新参数
- 学习率从 0 线性预热 2000 步到 2.5×10^{-4}，然后余弦退火到 0
- 训练 100 个 epoch

关键设计：

- 因为是自回归，所以用**因果掩码（下三角矩阵）**，每个位置只能看到自己和左边的 token，看不到右边
- 这就是 GPT 和 BERT 最本质的区别：GPT 是单向（从左到右），BERT 是双向

8.4.2 有监督微调

目标函数：在预训练模型基础上，加任务特定的输出层，用标注数据端到端微调。

$$\mathcal{L}_{\text{finetune}} = \sum \log P(y \mid x^1, \ldots, x^m; \Theta)$$

其中 x 是输入 token 序列，y 是任务标签。

具体做法：

- 取预训练好的模型，去掉语言建模头

- 根据任务类型加不同的输出层：
  - 分类任务：取最后一个 token 的输出，接线性分类层
  - 蕴含任务：把前提和假设用分隔符拼起来，取最后 token 输出分类
  - 相似度任务：两个句子分别输入，取最后 token 输出后做元素相乘，再接分类层
  - 问答任务：上下文和问题拼起来，每个位置输出一个分数，预测答案起止位置
- 用标注数据计算任务损失，反向传播更新所有参数（包括预训练部分）
- 同时保留语言建模损失作为辅助损失，权重为 0.5，防止微调破坏预训练知识

总损失函数：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{LM}}$$

其中 $\lambda = 0.5$。

微调超参数：

- 学习率：6.25×10^{-5}
- Batch size：32
- Epoch：3
- Dropout：0.1（分类器层）

8.4.3 训练数据

数据集：BooksCorpus

数据来源：7000 多本未出版的书籍，来自 Smashwords、Bookshare 等平台。

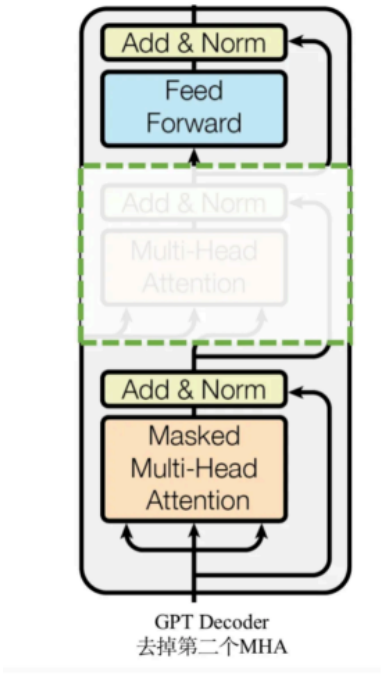
数据特点：

- 规模：约 8 亿个单词（约 1GB 文本）
- 体裁多样：冒险、奇幻、爱情、科幻等多种小说类型
- 文本连续性强：书籍是长篇连续文本，有利于模型学习长程依赖关系
- 对比：当时 ELMo 用的 1B Word Benchmark 虽然大小相近，但句子是打乱的，没有长程结构

8.4.5 模型结构总览

参数	值
架构	Decoder-only Transformer
层数	12 层
隐藏维度	768
注意力头数	12 个
FFN 中间维度	3072
总参数量	约 1.17 亿
上下文窗口	512 tokens
词表大小	40,000（BPE）
激活函数	GELU

8.4.6 网络结构



GPT-1 的每一层解码器包含以下组件，按顺序执行：

（1）掩码多头自注意力（Masked Multi-Head Self-Attention）

注意力计算公式：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

其中 d_k 是每个注意力头的维度。

多头注意力公式:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

（2）残差连接 + LayerNorm

```
x = x + Dropout(attention_output)
x = LayerNorm(x)
```

注意：GPT-1 用的是 **Post-LN**（先残差再加 LayerNorm），和原始 Transformer 一致。GPT-2 开始改成 Pre-LN。

LayerNorm 公式:

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中 μ 和 σ^2 是每个样本的均值和方差，γ 和 β 是可学习的缩放和偏移参数。

（3）前馈网络（Feed-Forward Network）

```
第一步：升维
  h = GELU(x · W_1 + b_1)  [batch, seq_len, 3072]

第二步：降维
  output = h · W_2 + b_2    [batch, seq_len, 768]
```

激活函数用 **GELU**（高斯误差线性单元），不是 ReLU。

GELU 公式:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

其中 $\Phi(x)$ 是标准正态分布的累积分布函数：

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{x} e^{-t^2/2} dt$$

GELU 相比 ReLU 的优势：平滑可微，不会有"死亡神经元"问题，在深层网络中训练更稳定。

（4）残差连接 + LayerNorm

```
x = x + Dropout(ffn_output)
x = LayerNorm(x)
```

（5）嵌入层

输入 embedding = Token Embedding + Position Embedding

- Token Embedding：可学习矩阵，形状 $[40000, 768]$
- Position Embedding：可学习矩阵，形状 $[512, 768]$（不是正弦编码，是可学习的）

两者逐元素相加，得到最终输入。

（6）输出层

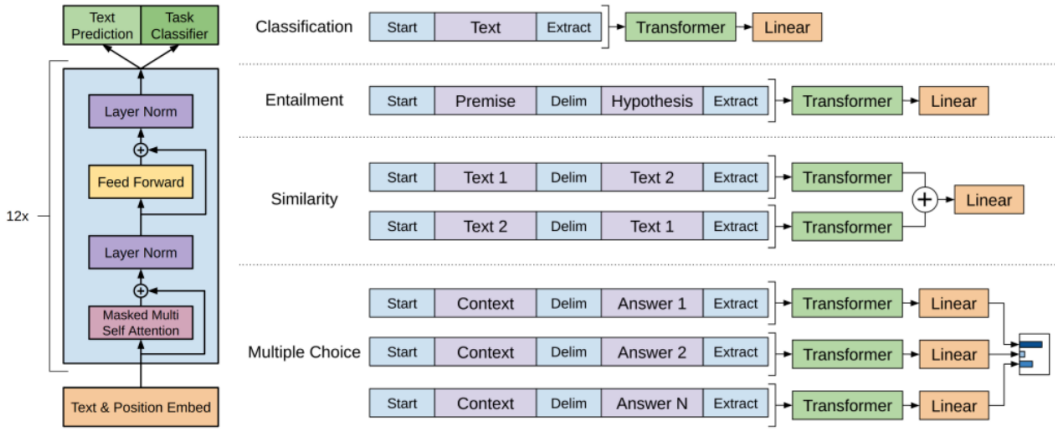
预训练阶段：最后一层的隐藏状态乘 Token Embedding 的转置（权重绑定），得到每个词表位置的 logits，过 Softmax 得到下一个 token 的概率分布。

$$P(u_{i+1} \mid u_{<i}) = \mathrm{Softmax}(h_i W_e^\top)$$

其中 h_i 是第 i 个位置的隐藏状态，W_e 是 Token Embedding 矩阵。

8.4.7 特定务的输入变换

对于文本分类任务而言，可以直接对预训练模型进行微调. 由于预训练模型是在连续文本上训练的，对于某些具有结构 化的输入（例如有序句子对，或是文档、问题、答案的三元组）任务，需要对输入进行一些修改，将结构化输入转换 成预训练模型可以处理的有序序列.



8.5 GPT-2（2019年2月）

GPT-2 的论文标题是《Language Models are Unsupervised Multitask Learners》（语言模型是无监督的多任务学习者）。

核心思想一句话：把模型和数据规模都放大，语言模型可以在不做任何微调的情况下，直接完成多种任务（零样本学习，Zero-Shot）。

GPT-1 证明了"预训练 + 微调"有效。GPT-2 更进一步说：模型足够大、数据足够多、足够多样的话，连微调都不需要，直接用自然语言提示就能让模型做任务。

比如：

- 翻译：输入"翻译成法语：我爱你 →"，模型直接输出法语
- 摘要：输入"[文章] TL;DR:"，模型直接输出摘要
- 问答：输入"问题：xxx 答案："，模型直接输出答案

8.5.1 无监督预训练

目标函数：和 GPT-1 完全一样，标准自回归语言建模。

$$\mathcal{L} = \sum_i \log P(u_i \mid u_{<i}; \Theta)$$

训练过程：

- 和 GPT-1 基本相同，但规模大了一个数量级
- 上下文窗口从 512 扩展到 1024
- Batch size 512
- 学习率预热到 2.5×10^{-4}，然后退火
- 用了梯度累积来模拟大 batch

8.5.2 不做微调

GPT-2 不做任何有监督微调，所有任务都通过零样本提示完成。这是和 GPT-1 最大的方法论区别。

论文的核心论点就是：无监督预训练本身就包含了多任务学习的能力，因为训练数据里天然存在各种任务的示范（比如网页里有翻译对照、有问答、有摘要）。

模型足够大时，就能隐式学会这些任务，不需要显式的监督微调。

8.5.3 训练数据

数据集：WebText（OpenAI 自己构建的，未公开）

数据来源：

- 从 Reddit 爬取所有获得至少 3 个 karma（点赞）的外部链接
- 共约 800 万个文档
- 排除了维基百科（因为很多评测数据集包含维基百科内容，避免数据泄露）

数据规模：约 40GB 文本，约 800 万网页。

数据特点：

- 质量高：Reddit 点赞机制相当于人工筛选，只有被认为有价值的内容才会被收录
- 多样性强：涵盖新闻、小说、论坛、编程文档、技术博客等各种风格
- 包含自然的任务示范：网页里天然有"问题-答案"、"原文-摘要"、"英文-法文对照"等结构
- 长文本：很多网页是长篇文章，有利于长上下文学习

8.5.4 模型结构总览

GPT-2 有四个版本：

版本	层数	隐藏维度	注意力头	参数量
Small	12	768	12	1.24 亿
Medium	24	1024	16	3.55 亿
Large	36	1280	20	7.74 亿

版本	层数	隐藏维度	注意力头	参数量
XL	48	1600	25	15.42 亿

最大的 XL 版：

- 48 层 Decoder
- 隐藏维度 1600
- 25 个注意力头（每个头 64 维）
- FFN 中间维度 6400
- 上下文窗口 1024
- 词表大小 50,257

8.5.6 网络结构

GPT-2 的基础结构和 GPT-1 一样，都是 Decoder-only Transformer，但做了几个关键的工程改进：

（1）LayerNorm 位置改为 Pre-LN

GPT-1 用的是 Post-LN：

$$x = \mathrm{LayerNorm}(x + \mathrm{Attention}(x))$$

GPT-2 改成了 Pre-LN：

$$x = x + \mathrm{Attention}(\mathrm{LayerNorm}(x))$$

FFN 部分同理：

$$x = x + \mathrm{FFN}(\mathrm{LayerNorm}(x))$$

（2）额外LayerNorm

GPT-1 最后一层输出后直接接输出头。GPT-2 在最后一层之后、输出头之前，额外加了一个 LayerNorm，进一步稳定训练。

（3）修改初始化方式

考虑到残差路径的累积效应，把残差分支上的线性层初始化为：

$$W_{\text{init}} \sim \mathcal{N}\left(0, \frac{1}{\sqrt{2N}}\right)$$

其中 N 是层数。这样随着层数加深，残差路径上的信号不会被放大太多，训练更稳定。

（4）词表扩展

从 GPT-1 的 40,000 扩展到 50,257。

（5）上下文窗口扩展

从 512 扩展到 1024 tokens。

（6）注意力机制

和 GPT-1 完全一样，还是因果掩码的多头自注意力，没有结构创新。

8.6 GPT-3（2020年5月）

GPT-3 的论文标题是《Language Models are Few-Shot Learners》（语言模型是少样本学习者）。

核心思想一句话：把规模推到极致（1750 亿参数），模型会涌现出"上下文学习"（In-Context Learning）能力，在输入提示里给几个示例，模型就能学会任务，不需要更新任何参数。

8.6.1 无监督预训练

目标函数：还是标准自回归语言建模，和 GPT-1、GPT-2 完全一样。

$$\mathcal{L} = \sum_i \log P(u_i \mid u_{<i}; \Theta)$$

训练过程：

- 总训练 token 数：约 3000 亿（5000 亿语料，部分数据重复 epoch）
- Batch size：从 32K 逐渐增加到 3.2M tokens（梯度累积）
- 学习率：预热到 0.6×10^{-4}（175B 版），然后余弦退火
- 训练了 8 个不同大小的模型（125M 到 175B），用来验证 scaling laws

关键发现：模型性能和参数量、数据量、计算量之间遵循平滑的幂律关系（Scaling Laws）。

Scaling Law 公式（Chinchilla 之前的经验公式）：

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} + L_{\text{non-generative}}$$

其中 N 是参数量，D 是训练 token 数，$N_c, D_c, \alpha_N, \alpha_D$ 是经验常数。

这意味着可以通过扩大规模来可预测地提升性能。

8.6.2 不做微调

GPT-3 **不做微调**（论文的核心设定）。

所有任务都通过上下文学习完成：

- Zero-Shot：提示 = 任务描述 + 输入
- One-Shot：提示 = 任务描述 + 1个示例 + 输入
- Few-Shot：提示 = 任务描述 + 几个示例 + 输入

模型看到示例后，在推理时隐式学习任务模式，然后生成答案。整个过程不更新任何参数。

8.6.4 训练数据

GPT-3 用了一个混合数据集，总大小约 5000 亿 token：

数据集	Token 数	占比	说明
Common Crawl（过滤后）	4100 亿	60%	互联网网页爬取，经过严格质量过滤
WebText2	190 亿	19%	WebText 的扩展版，Reddit 高赞链接
Books1	120 亿	8%	互联网书籍语料
Books2	550 亿	8%	更大规模的书籍语料
Wikipedia	30 亿	3%	英文维基百科

Common Crawl 的质量过滤三步法：

1. **基于相似度过滤**：训练一个逻辑回归分类器，用 WebText、Wikipedia、书籍作为正例，Common Crawl 随机样本作为负例，然后过滤掉 Common Crawl 中被分类器判为负例的低质量内容

2. **模糊去重**：在数据集内部和跨数据集之间，用 MinHash 等方法做文档级去重，减少冗余
3. **加入高质量参考语料**：把 WebText2、Books、Wikipedia 显式加入训练混合，提升整体质量

数据上采样：

- 高质量数据集（Books、Wikipedia）被上采样，即在训练中被多次看到
- Common Crawl 虽然占 60%，但实际 epoch 数少
- 文本数据约 2 个 epoch，代码数据约 4 个 epoch

8.6.5 模型结构总览

GPT-3 训练了 8 个不同大小的模型，从 1.25 亿到 1750 亿参数，跨越三个数量级。

训练这么多尺寸的目的是验证 Scaling Laws（缩放定律），证明模型性能随规模平滑增长。

所有 8 个模型共享以下统一配置：

- 架构：Decoder-only Transformer
- 上下文窗口：2,048 tokens
- 词表大小：50,257（BPE）
- 激活函数：GELU
- 归一化：Pre-LN
- 注意力：因果掩码 + 交替密集/稀疏注意力（175B 版用稀疏，小模型全密集）

8 个模型完整参数对照表：

模型	参数量	层数	隐藏维度	注意力头数	每头维度	FFN 中间维度	Batch tokens	峰值学习率
Small	125 M	12	768	12	64	3,072	0.5 M	6.0×10^{-4}
Medium	350 M	24	1,024	16	64	4,096	0.5 M	3.0×10^{-4}
Large	760 M	24	1,536	16	96	6,144	0.5 M	2.5×10^{-4}
XL	1.3 B	24	2,048	16	128	8,192	1 M	2.0×10^{-4}
2.7B	2.7 B	32	2,560	32	80	10,240	1 M	1.6×10^{-4}

模型	参数量	层数	隐藏维度	注意力头数	每头维度	FFN 中间维度	Batch tokens	峰值学习率
6.7B	6.7 B	32	4,096	32	128	16,384	2 M	1.2×10^{-4}
13B	13.0 B	40	5,120	40	128	20,480	2 M	1.0×10^{-4}
175B	175 B	96	12,288	96	128	49,152	3.2 M	0.6×10^{-4}

8.6.6 网络结构

GPT-3 的基础架构和 GPT-2 几乎完全相同，只有一个结构上的变化：

（1）交替使用密集注意力和局部带状稀疏注意力

这是 GPT-3 唯一的架构创新（来自 Sparse Transformer 论文）。

在 96 层中，交替使用两种注意力模式：

- **偶数层**：标准密集注意力（每个位置可以关注所有位置）
- **奇数层**：局部带状稀疏注意力（每个位置只能关注自己附近的一个窗口内的位置）

（2）其他架构细节

和 GPT-2 完全一致：

- Pre-LN（LayerNorm 在子层输入）
- 最后一层后额外加 LayerNorm
- 修改的初始化（考虑残差累积）
- 可学习位置编码
- 因果掩码（下三角）
- GELU 激活函数

（3）训练并行策略

175B 参数的模型无法放在单张 GPU 上，必须用各种并行技术：

- **张量并行（Tensor Parallelism）**：在每层内部，把矩阵乘法切分到多张 GPU
- **流水线并行（Pipeline Parallelism）**：把不同层放到不同 GPU
- **数据并行（Data Parallelism）**：整个模型复制多份，处理不同数据

- 实际用了上万张 V100 GPU

8.7 InstructGPT（2022年3月）

InstructGPT 的论文标题是《Training language models to follow instructions with human feedback》（用人类反馈训练语言模型遵循指令）。

核心思想一句话：GPT-3 虽然能力强，但经常不"听话"

GPT-3 是一个"续写机器"，你给它开头，它就往下续写。

InstructGPT 就是把 GPT-3 从"续写机器"调教成"听话助手"。

核心原则：3H 原则

- **Helpful**（有用）：能帮助用户完成任务
- **Honest**（诚实）：不编造信息，知道就说知道，不知道就说不知道
- **Harmless**（无害）：不输出有害、歧视、危险的内容

8.7.1 预训练

InstructGPT **不从头预训练**，直接用已经预训练好的 GPT-3 作为起点。

也就是说，InstructGPT = GPT-3 + 三阶段人类反馈微调。

8.7.2 微调怎么做的（三阶段 RLHF）

这是 InstructGPT 的核心，分三个阶段：

第一阶段：监督微调（SFT, Supervised Fine-Tuning）

目标：让 GPT-3 初步学会"听指令、按格式回答"。

数据：约 13,000 条"指令-示范答案"对。

- 一部分是标注员自己编写的各种指令和理想回答
- 一部分是从 OpenAI API 收集的真实用户提示，标注员写理想回答

SFT 损失函数：

$$\mathcal{L}_{\text{SFT}} = -\sum_{(x,y)\in\mathcal{D}} \sum_{t=1}^{|y|} \log \pi_{\text{SFT}}(y_t \mid y_{<t}, x)$$

其中 x 是指令，y 是示范答案，π_{SFT} 是 SFT 模型。

效果：SFT 模型已经能基本遵循指令，但回答质量不稳定，有时候好有时候差。

第二阶段：奖励模型训练（RM, Reward Modeling）

目标：训练一个"打分器"，输入一个指令和一个回答，输出一个分数，表示这个回答有多符合人类偏好。

数据：约 33,000 条提示，每条提示用 SFT 模型生成 4-9 个不同的回答，由人类标注员从最好到最差排序。

损失函数：对比损失（Pairwise Loss）

$$\mathcal{L}_{\mathrm{RM}} = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(r_\phi(x,y_w) - r_\phi(x,y_l)\right)\right]$$

其中 y_w 是人类偏好的回答（winner），y_l 是不偏好的回答（loser），r_ϕ 是奖励模型，σ 是 sigmoid 函数。

对于多个回答的排序，可以扩展为 Listwise 损失：

$$\mathcal{L}_{\mathrm{RM}} = -\mathbb{E}\left[\sum_{i<j}\log\sigma\left(r(x,y_i) - r(x,y_j)\right)\right]$$

其中 y_i 排序在 y_j 前面（y_i 更好）。

第三阶段：强化学习微调（RL, Reinforcement Learning）

目标：用奖励模型作为奖励信号，用 PPO 算法微调 SFT 模型，让模型生成更高分的回答。

算法：PPO（Proximal Policy Optimization，近端策略优化）

数据：约 31,000 条 API 提示，不需要人工标注。

训练过程：

1. 随机采样一个提示 x
2. 当前策略模型（SFT 初始化）生成回答 y
3. 奖励模型给 (x,y) 打分 $r_\phi(x,y)$
4. 加上 KL 散度惩罚：$\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{SFT}})$，防止策略模型偏离 SFT 太远
5. 最终奖励：

$$R(x,y) = r_\phi(x,y) - \beta\cdot\mathrm{KL}\left(\pi_\theta(\cdot\mid x)\|\pi_{\mathrm{SFT}}(\cdot\mid x)\right)$$

6. 用 PPO 算法更新策略模型参数

KL 散度公式：

$$\mathrm{KL}(\pi_\theta \| \pi_{\mathrm{ref}}) = \sum_t \pi_\theta(y_t \mid y_{<t}, x) \cdot \log \frac{\pi_\theta(y_t \mid y_{<t}, x)}{\pi_{\mathrm{ref}}(y_t \mid y_{<t}, x)}$$

PPO 的核心目标函数：

$$\mathcal{L}_{\mathrm{PPO}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t, \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) A_t \right) \right]$$

其中：

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}$$

是新旧策略的概率比，A_t 是优势函数，ϵ 是裁剪范围（通常 0.2）。

PPO 的核心思想：

- 每次更新参数时，限制新策略和旧策略的差距不能太大
- 用 clipped objective 防止更新步长过大导致训练崩溃
- 这样可以稳定地用强化学习优化语言模型

额外技巧：在 PPO 训练时，混入预训练梯度（预训练损失乘以一个小权重），防止模型在 RL 过程中丢失通用语言能力。这叫做 PPO-ptx。

PPO-ptx 总目标：

$$\mathcal{L}_{\mathrm{total}} = \mathcal{L}_{\mathrm{PPO}} + \gamma \cdot \mathcal{L}_{\mathrm{pretrain}}$$

8.7.4 训练数据

三个阶段用三种不同的数据：

阶段	数据量	数据来源	标注方式
SFT	~13K 条	标注员编写 + API 真实提示	标注员写理想回答
RM	~33K 条提示	API 真实提示为主	标注员对多个回答排序
PPO	~31K 条提示	API 真实提示	无人工标注，用 RM 打分

标注员：OpenAI 雇佣了约 40 名标注员，经过严格培训和筛选，主要来自美国和东南亚。标注员需要理解 3H 原则，对敏感内容有判断力。

数据特点：

- 指令类型多样：问答、写作、编程、翻译、摘要、推理、聊天等
- 覆盖真实用户需求：大部分来自 API 实际调用，不是人工编造的
- 质量高：经过多轮审核和一致性检验

8.7.5 模型结构总览

InstructGPT 不是新模型，是 GPT-3 的微调版本。有三个尺寸：

版本	参数量	说明
1.3B	13 亿	最小版本，用于实验对比
6B	60 亿	奖励模型用这个尺寸
175B	1750 亿	最大版本，效果最好

基础架构和 GPT-3 完全一样：96 层 Decoder，12288 隐藏维度，96 注意力头，交替密集/稀疏注意力。

8.7.6 网络结构

InstructGPT 的网络结构和 GPT-3 完全相同，没有任何架构改动。所有变化都在训练方法上。

但有几个和结构相关的细节值得注意：

（1）奖励模型的结构

```
输入：指令 x + 回答 y，拼接成一个序列
  ↓
和 GPT-3 一样的 96 层 Transformer（Decoder，但这里只做编码）
  ↓
取最后一个 token 的隐藏状态 [1, 12288]
  ↓
线性层 [12288 → 1]
  ↓
输出标量奖励分数 r
```

（2）PPO 训练时的模型角色

PPO 训练涉及三个模型：

- **策略模型（Policy）**：被训练的模型，从 SFT 初始化，175B
- **奖励模型（Reward）**：固定不动，给回答打分，6B
- **参考模型（Reference）**：固定不动，就是 SFT 模型，用来算 KL 散度，175B

训练时需要同时加载这三个模型，显存压力很大。

（3）KL 惩罚的作用

KL 散度衡量当前策略和 SFT 模型的输出分布差异。如果没有 KL 惩罚，模型可能会：

- 发现某些奇怪的输出模式能从奖励模型那里拿到高分
- 比如重复输出某些关键词、输出特别长的回答
- 这叫做"奖励黑客"（Reward Hacking）

KL 惩罚让模型不能偏离 SFT 太远，保持输出的自然性和通用性。

8.8 ChatGPT / GPT-3.5（2022年11月）

ChatGPT 没有正式论文，OpenAI 只发了博客。它本质上是 **InstructGPT 的对话优化版本**，基于 GPT-3.5 系列模型。

核心思想一句话：把 InstructGPT 的 RLHF 技术应用到对话场景，训练一个能进行多轮对话、回答问题、承认错误、质疑错误前提、拒绝不当请求的对话助手。

8.8.1 预训练

ChatGPT 不从头预训练，基础是 GPT-3。但有一个关键的中间步骤：代码继续预训练。

OpenAI 发现，在 GPT-3 基础上用大量代码数据继续预训练，可以大幅提升模型的推理能力（思维链、逻辑推理、数学能力）。这就是 code-davinci-002，也是 GPT-3.5 系列的起点。

8.8.2 微调怎么做的

ChatGPT 的微调和 InstructGPT 基本一样，也是三阶段 RLHF，但数据更偏向对话：

第一阶段：SFT 监督微调

- 数据：大量的多轮对话示范，包括人类对话、客服对话、标注员编写的对话
- 目标：让模型学会对话格式——理解用户消息、生成助手回复、记住上下文
- 输入格式：用特殊 token 分隔用户和助手的消息，比如 `<|user|>...` `<|assistant|>...`

第二阶段：RM 奖励模型

- 数据：同一个对话提示，模型生成多个回答，人类标注员排序
- 排序标准更侧重对话质量：是否连贯、是否有用、是否记住了之前的对话内容
- 奖励模型输入完整的对话历史，而不只是单轮指令

第三阶段：PPO 强化学习

- 和 InstructGPT 一样，用 PPO + KL 惩罚
- 额外优化对话的连贯性、安全性、有用性

8.8.3 训练数据

OpenAI 没有公开 ChatGPT 的训练数据细节，但已知包括：

- **对话数据**：大量人类多轮对话，可能来自社交媒体、客服记录、标注员编写
- **指令数据**：和 InstructGPT 类似的指令-回答对
- **安全数据**：专门标注的有害请求和拒绝示例，用于训练模型拒绝不当请求
- **对比数据**：人类对模型多个回答的偏好排序

数据截止时间：最初版本截止到 2021 年 9 月，后来版本有更新。

8.8.4 模型结构总览

ChatGPT（gpt-3.5-turbo）的具体参数 OpenAI 没有公开

8.8.5 网络结构

ChatGPT 的网络结构和 GPT-3 基本相同，没有重大架构改动。主要的工程优化：

（1）推理优化

为了支持实时对话，ChatGPT 做了大量推理优化：

- 模型蒸馏：用大模型蒸馏出小模型，提升推理速度
- 量化：可能用 INT8 或 FP8 量化，减少显存和计算
- 连续批处理（Continuous Batching）：动态合并请求，提升 GPU 利用率
- 推测解码（Speculative Decoding）：用小模型提前生成候选，大模型验证，提升解码速度

（2）对话格式处理

输入不是纯文本，而是结构化的对话消息：

```
<|system|>你是一个有用的助手
<|user|>你好
<|assistant|>你好！有什么可以帮你的？
<|user|>今天天气怎么样？
<|assistant|>
```

模型在 `<|assistant|>` 之后生成回答，特殊 token 帮助模型区分角色。

8.9 GPT-4（2023年3月）

GPT-4 的技术报告标题就是《GPT-4 Technical Report》。

核心思想一句话：一个大规模多模态模型，可以接受图像和文本输入，输出文本；在推理、创意、专业知识、代码等方面达到甚至超过人类水平。

GPT-4 相比 GPT-3.5 的核心提升：

- **推理能力大幅提升**：数学、逻辑、代码、专业考试（律师资格、医师执照、AP 考试等）表现优异

- **多模态**：支持图像输入（不是生成图像，是理解图像）
- **更长上下文**：8K 和 32K 两个版本（后来扩展到 128K）
- **更安全**：经过大量安全对齐训练，有害输出大幅减少
- **支持工具调用**：可以调用函数、搜索、代码解释器等

下面的架构信息主要来自业内泄露和推测!!!!!!

8.9.1 预训练

目标函数：标准自回归语言建模，和之前所有版本一样。

$$\mathcal{L} = \sum_i \log P(u_i \mid u_{<i}; \Theta)$$

训练规模（泄露信息，来自 SemiAnalysis）：

- 总参数量：约 1.8 万亿（但不是每次都全激活）
- 训练 token 数：约 13 万亿
- 文本数据：约 2 个 epoch
- 代码数据：约 4 个 epoch
- 训练计算量：约 2.15×10^{25} FLOPs
- 训练成本：约 6300 万美元

多模态预训练：

- 文本和图像统一在同一个 Transformer 中处理
- 图像被编码成 token 序列，和文本 token 混合输入
- 预训练目标还是预测下一个 token（文本 token 或图像 token）

8.9.2 微调

GPT-4 的微调比之前更复杂，包括：

（1）SFT 监督微调

- 用高质量的指令-回答对做监督微调
- 数据质量比 InstructGPT 更高，覆盖更多专业领域

（2）RLHF 人类反馈强化学习

- 和 InstructGPT 一样的三阶段，但规模更大、数据更多
- 奖励模型更复杂，可能用了集成的多个奖励模型

（3）RBRM 规则奖励模型（Rule-Based Reward Model）

- 这是 GPT-4 的新增项
- 用一套基于规则的奖励模型，专门针对安全性
- 对有害输出给负奖励，对安全拒绝给正奖励
- 和人类偏好奖励模型结合使用

（4）红队测试（Red Teaming）

- 雇佣大量红队专家，专门尝试诱导模型输出有害内容
- 把这些攻击案例加入训练数据，提升模型安全性

8.9.5 训练数据

OpenAI 没有公开详细数据构成，但已知包括：

- 大规模互联网文本（Common Crawl 等，经过严格过滤）
- 大量代码数据（GitHub 等）
- 书籍、学术论文、技术文档
- 专业领域数据（法律、医学、金融等）
- 多模态数据：图像-文本配对数据
- 数据截止：最初版本到 2021 年 9 月，GPT-4 Turbo 到 2023 年 12 月

8.9.6 模型结构总览

以下是泄露/推测的 GPT-4 架构参数（来自 SemiAnalysis 等业内来源，非官方确认）：

参数	推测值
架构	Decoder-only Transformer + MoE
总参数量	约 1.8 万亿
每次前向激活参数量	约 2800 亿

参数	推测值
层数	约 120 层
专家数	16 个
每次激活专家数	2 个
隐藏维度	约 16,384
注意力头数	约 128 个
上下文窗口	8K / 32K（后来 128K）
词表大小	约 100,000

8.9.7 网络结构深入剖析

GPT-4 最大的架构变化是引入了 **MoE（Mixture of Experts，混合专家）**。

（1）MoE 混合专家架构

核心思想：不是所有参数都参与每次计算，而是根据输入动态选择一部分"专家"来计算。

门控网络公式：

$$g(x) = \text{Softmax}(xW_g)$$

其中 $W_g \in \mathbb{R}^{d_{\text{model}} \times E}$，$E$ 是专家数量。

Top-k 选择：

$$\text{TopK}(g(x), k) = \begin{cases} g_i(x) & \text{if } g_i(x) \text{ 在前 k 大} \\ 0 & \text{otherwise} \end{cases}$$

MoE 输出：

$$\text{MoE}(x) = \sum_{i=1}^{E} \text{TopK}(g(x))_i \cdot \text{Expert}_i(x)$$

MoE 的位置：MoE 通常替换 Transformer 中的 FFN 层。注意力层还是共享的，只有 FFN 是专家化的。

负载均衡损失：为了防止所有样本都涌向同一个专家，MoE 训练时通常加一个辅助损失：

$$\mathcal{L}_{\text{balance}} = \alpha \cdot E \cdot \sum_{i=1}^{E} f_i \cdot P_i$$

其中 f_i 是分配给专家 i 的 token 比例，P_i 是门控给专家 i 的平均概率。这个损失鼓励所有专家被均匀使用。

（2）多模态架构

GPT-4 的视觉理解不是单独的视觉编码器，而是：

- 图像被分割成 patch（类似 ViT）
- 每个 patch 投影成一个向量，作为一个"视觉 token"
- 视觉 token 和文本 token 混合，一起输入 Transformer
- Transformer 用统一的自注意力处理所有 token
- 这样文本和图像可以在同一个隐空间里交互

（3）其他架构细节

- Pre-LN（和 GPT-2/3 一致）
- 可能用了 RMSNorm 而不是 LayerNorm（更高效）

RMSNorm 公式：

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}} \cdot \gamma$$

RMSNorm 去掉了均值中心化，只保留 RMS 归一化，计算更快，效果和 LayerNorm 相当。

- 可能用了 SwiGLU 激活函数而不是 GELU（表达能力更强，LLaMA 等模型都用这个）

SwiGLU 公式：

$$\text{SwiGLU}(x) = (\text{Swish}(xW_1) \otimes xV)W_2$$

其中 $\text{Swish}(x) = x \cdot \sigma(x)$，$\otimes$ 是逐元素相乘。

- 可能用了 RoPE（旋转位置编码）而不是可学习位置编码（更好的长上下文外推）

RoPE 公式：

$$f(q, m) = qe^{im\theta}$$

在复数域上对位置 m 的 query 向量施加旋转，相对位置信息自然融入注意力计算。

- 多查询注意力（MQA）或分组查询注意力（GQA）：多个头共享 K/V，降低推理显存

GQA 原理：

- 标准 MHA：每个头有独立的 K/V
- MQA：所有头共享同一组 K/V
- GQA：分成 G 组，每组共享一组 K/V（介于 MHA 和 MQA 之间）

注意：以上是推测，OpenAI 没有确认。但从 GPT-4 的性能和推理速度来看，这些技术大概率都用了。

8.10 GPT-4o（2024年5月）

GPT-4o（o 代表 omni，全模态）是 OpenAI 的原生多模态模型。

核心思想一句话：一个单一的端到端神经网络，同时处理文本、语音、视觉输入和输出，实现实时语音对话，响应速度接近人类。

GPT-4 之前的多模态是"拼接式"的：

- 语音输入：ASR（语音转文字）→ 语言模型 → TTS（文字转语音）
- 图像输入：视觉编码器 → 把图像描述成文字 → 语言模型

这种流水线的问题：

- 延迟高：多个模型串联，每一步都有延迟
- 信息损失：ASR 识别错误会传递下去，图像转文字会丢失细节
- 不自然：语音没有语调、情感、停顿

GPT-4o 彻底改变了这个范式：文本、语音、图像都在同一个神经网络里端到端处理。

关键特性：

- **实时语音对话**：响应延迟最低 232ms，平均 320ms，和人类对话反应速度差不多
- **原生语音输出**：直接生成语音波形，不需要 TTS，有自然的语调、情感、停顿

- **视觉理解**：实时理解摄像头画面
- **文本能力**：和 GPT-4 Turbo 相当，非英文语言提升明显
- **推理速度**：比 GPT-4 Turbo 快 2 倍，便宜一半

8.10.1 预训练

目标：端到端多模态自回归预训练。

核心：把文本、语音、图像都映射到同一个 token 空间，然后用标准的"预测下一个 token"目标训练。

$$\mathcal{L} = \sum_{i} \log P(z_i \mid z_{<i}; \Theta)$$

其中 z_i 可以是文本 token、图像 token 或音频 token。

统一输入序列：

$$\text{input} = [z^1_{\text{text}}, \ldots, z^n_{\text{text}}, z^1_{\text{image}}, \ldots, z^m_{\text{image}}, z^1_{\text{audio}}, \ldots, z^k_{\text{audio}}]$$

模态编码：

- 文本：BPE token，和之前一样
- 图像：分割成 patch，编码成 token 序列
- 音频：波形编码成离散 token（或连续表示），可能用了类似 VQ-VAE 的音频编码器

预训练数据：

- 大规模文本数据（和 GPT-4 类似）
- 图像-文本配对数据
- 音频-文本配对数据（语音+字幕）
- 音频-图像配对数据（视频+音频）
- 纯音频数据（播客、有声书等）

关键是**配对数据**，让模型学会不同模态之间的对应关系。

8.10.2 微调

GPT-4o 的微调包括：

（1）多模态 SFT

- 用多模态指令数据做监督微调
- 包括：语音问答、图像描述、视频理解、多模态对话等

（2）多模态 RLHF

- 人类对多模态回答进行排序
- 排序标准包括：语音自然度、语调合适度、视觉理解准确性、整体有用性
- 奖励模型也是多模态的，输入语音/图像/文本，输出分数

（3）语音专项优化

- 专门优化语音的自然度、情感表达、停顿、语调
- 可能用了对抗训练或扩散模型来提升语音质量

8.10.3 训练数据

GPT-4o 的训练数据是多模态的：

- **文本**：互联网文本、书籍、代码、学术论文等
- **图像**：大规模图像-文本配对数据
- **音频**：大量语音数据，包括播客、有声书、对话录音、语音-字幕配对
- **视频**：带音频的视频数据，学习音画同步
- **对话数据**：多轮语音对话示范

数据特点：

- 模态之间有配对关系（同一个内容的文本、语音、图像版本）
- 包含真实人类对话，学习对话节奏和语调
- 覆盖多种语言、多种口音、多种场景

8.10.4 模型结构

OpenAI 没有公开 GPT-4o 的详细参数，但已知：

- 基础架构：单一 Transformer 骨干，端到端多模态
- 统一隐空间：所有模态映射到同一个向量空间
- MoE 架构：和 GPT-4 类似的混合专家

- 模态特定编码器：文本、语音、图像各有自己的输入编码器，但共享 Transformer 骨干
- 模态特定解码器：输出时根据目标模态选择不同的输出头
- 推理速度：比 GPT-4 Turbo 快约 2 倍

（1）统一多模态架构

GPT-4o 的核心是**单一神经网络处理所有模态**：

```
输入层（模态特定编码器）：
  文本 → Text Encoder → 文本 embedding
  图像 → Vision Encoder（patch + 投影） → 图像 embedding
  音频 → Audio Encoder（波形编码） → 音频 embedding
      ↓
所有 embedding 拼接到同一个序列，加上模态标记和位置编码
      ↓
共享 Transformer 骨干（MoE Decoder，几十层）
  - 自注意力可以跨模态关注：文本 token 可以关注图像 token，音频 token 可以关注文本 token
  - 这样实现真正的跨模态理解
      ↓
输出层（模态特定解码器）：
  文本输出 → LM Head → 文本 token 概率
  音频输出 → Audio Decoder → 语音波形
  （图像输出 GPT-4o 暂时不支持生成）
```

（2）跨模态注意力

这是 GPT-4o 最关键的机制。在自注意力中，不同模态的 token 可以互相注意：

- 文本 token 关注图像 token：理解"这张图里有什么"
- 音频 token 关注文本 token：把语音和文字对齐
- 图像 token 关注音频 token：理解"画面里的声音是什么"

跨模态注意力分数计算：

$$\text{scores}_{\text{text}\to\text{image}} = Q_{\text{text}} K_{\text{image}}^{\top} / \sqrt{d_k}$$

这种跨模态注意力是端到端多模态理解的基础，比"先转文字再处理"的流水线强得多，因为没有信息损失。

（3）音频编码与解码

音频处理是 GPT-4o 的难点：

- **编码**：语音波形 → 音频编码器 → 离散/连续 token 序列
  - 可能用了神经音频编码器（类似 EnCodec、SoundStream）
  - 把波形压缩成离散 token，大幅降低序列长度
- **解码**：Transformer 输出音频 token → 音频解码器 → 语音波形
  - 直接生成波形，不需要传统 TTS
  - 可以控制语调、情感、语速、停顿

VQ-VAE 风格的音频量化（推测）：

$$z_q = \text{Quantize}(e(x)) = e_k \quad \text{其中 } k = \arg\min_j \|e(x) - e_j\|$$

其中 $e(x)$ 是编码器输出，e_j 是码本中的向量。

（4）实时推理优化

为了实现 320ms 的平均延迟，GPT-4o 做了大量优化：

- **流式处理**：语音输入边说边处理，不需要等用户说完
- **低延迟解码**：优化的推理引擎，减少每 token 生成时间
- **模型蒸馏**：可能用了蒸馏来减小实际推理模型
- **量化**：FP8 或 INT8 量化，提升计算速度
- **推测解码**：用小模型草稿，大模型验证
- **连续批处理**：高效处理并发请求

（5）和 GPT-4 的架构对比

方面	GPT-4	GPT-4o
模态处理	拼接式（视觉编码器+LLM）	端到端统一
语音	不支持（需外部 ASR/TTS）	原生支持输入输出
图像	支持输入	支持输入
延迟	高（多步流水线）	低（端到端）
推理速度	基准	快约 2 倍
训练目标	文本为主	多模态联合

8.11 GPT 系列演进总结

8.11.1 核心技术路线变化

```
GPT-1：预训练 + 微调（1.17亿参数，书籍数据）
  ↓ 规模放大，零样本
GPT-2：预训练，不微调，零样本（15亿参数，WebText）
  ↓ 规模再放大，少样本涌现
GPT-3：预训练，不微调，上下文学习（1750亿参数，CommonCrawl混合）
  ↓ 对齐人类，RLHF
InstructGPT：GPT-3 + RLHF三阶段微调（1750亿，人类反馈数据）
  ↓ 对话优化，代码预训练
ChatGPT/GPT-3.5：对话版RLHF，代码继续预训练（百亿级，对话数据）
  ↓ 多模态，MoE，更强推理
GPT-4：MoE架构，多模态输入，强推理（1.8万亿总参数，多模态数据）
  ↓ 原生端到端多模态，实时语音
GPT-4o：统一多模态，端到端语音视觉文本（MoE，多模态配对数据）
```

8.11.2 每个版本的核心贡献

- **GPT-1**：确立"预训练 + 微调"两阶段范式
- **GPT-2**：证明规模带来零样本能力，引入 Pre-LN 等工程改进
- **GPT-3**：规模推到极致，涌现上下文学习，验证 Scaling Laws
- **InstructGPT**：发明 RLHF，让模型对齐人类意图
- **ChatGPT**：把 RLHF 应用到对话，引爆通用人工智能
- **GPT-4**：MoE 架构 + 多模态 + 强推理，达到专业人类水平
- **GPT-4o**：原生端到端多模态，实时语音交互，走向全模态 AI

8.11.3 不变的东西

从 GPT-1 到 GPT-4o，有几个东西始终没变：

1. **Decoder-only Transformer** 基础架构
2. **自回归语言建模** 预训练目标（预测下一个 token）
3. **因果掩码**（从左到右，看不到未来）
4. **规模是核心驱动力**（更大的模型、更多的数据、更强的计算）

变的是：规模越来越大、数据越来越多样、对齐越来越精细、模态越来越丰富、推理越来越快。

- 翻译任务：训练数据中包含大量"英文句子 = 法文句子"的对照
- 摘要任务：训练数据中包含大量"文章 + 摘要"的配对
- 问答任务：训练数据中包含大量"问题 + 答案"的格式

模型在预测下一个词时，隐式地学会了这些任务的"输入-输出映射"。在推理时，只要给出任务的"提示"（如"把英文翻译成法文：Hello"），模型就能"接着写"出答案。

- 架构：最大 48 层，1600 维，25 头，1.5B 参数（GPT-2 XL）
- 数据：WebText，从 Reddit 抓取的 40GB 高质量网页文本
- 关键技术：去掉了微调，直接用提示（prompt）做零样本推理

GPT-2 当时因为"担心被滥用"而没有完全开源（后来还是开源了），但它的零样本能力让整个领域震惊：原来语言模型可以"不微调就干活"。